法案系统建设说明书

文件版本：FINAL-V3.1 | 机器合同消歧终态版
冻结目标版本：FROZEN-FINAL-V3.1
定稿日期：2026-08-22
文件性质：治理系统源头建设规范 / Governance Bootstrap Specification
文件定位：定义空白仓库与既有仓库如何一次性建立法案正文、lawId 索引、唯一 Owner、机器施工图与规格绑定、Codex 防漂移协议、Static Audit、Exam、Dynamic Proof、修订事务、既有迁移、跨仓合同和冻结闭环
适用对象：Codex、AI Coding Agent、开发者、架构师、审计者、后续自动化执行者
交付形态：冻结 Markdown 权威源文件 + 同目录同名 PDF 非权威派生物；两者由外部 freeze manifest 分别绑定 SHA-256
当前交付状态：`SPEC_FROZEN`；本文件只允许位于法定 exact path，当前字节、固定 AGENTS、工具链和 PDF 派生物均由 `GOVERNANCE-FREEZE.json` 绑定
冻结声明：**进入 `SPEC_FROZEN` 后只读。任何普通开发、修复、重构、审计、迁移或兼容任务均无权修改本文件。**
法条标识：全部 lawId 固定使用中性前缀 `LAW-`；任何历史项目名称、旧 fence 名称或旧 lawId 前缀不得进入本文件、法案正文、generated projection、证据或报告。

0. 本说明书的唯一身份

本说明书不是业务法案，不定义某个产品应该具备什么功能，也不替代项目的基础法案、功能法案、施工图、规格、契约或代码。

本说明书只拥有一个主权：

> 定义“法案系统本身必须如何建设、如何运行、如何防止 Codex 漂移、如何阻断第二链路”。

本说明书必须让一个从未接触项目的 Codex 在读完后能够直接回答并执行：

> 法案放在哪里；
> 法条按什么结构编写；
> 什么内容属于基础法案；
> 什么内容属于功能法案；
> lawId 怎么编号和退役；
> Owner 怎么唯一绑定；
> 索引怎么生成和验真；
> 任务怎么路由到完整 lawId 闭包；
> Codex 在落笔前必须提交什么边界记录；
> 哪些范围扩张合法；
> 哪些深化、兼容、重构和新入口绝对禁止；
> Static Audit 怎么构建、怎么执行、怎么产出逐法条回执；
> Exam 和 Dynamic Proof 在什么情况下必须存在；
> 什么证据足以宣布 PASS；
> 什么情况必须停止并返回 FAIL；
> 最终如何冻结，避免本文件被 Codex 自行修改。

本说明书不是可供 Codex“参考发挥”的原则集合。所有出现“固定”“必须”“只能”“不得”“精确”的条款均为施工合同；Codex 不得补写另一种等价实现，不得把未写明的区域理解为自由发挥区。

第一部分 | 最高秩序与不可变公式

1. 法案系统的七件套

完整法案系统固定由以下七类对象组成，缺一项即未建成：

1. 冻结的《法案系统建设说明书》；
2. 一份《基础法案.md》；
3. 一份《功能法案.md》；
4. 由唯一 compiler 生成的 law registry；
5. 每个 capabilityDomain 唯一的 Owner Manifest；
6. lawId 对应的机器施工图 anchor、规格、Static/Exam/Dynamic 原子证据 binding；
7. 对当前候选逐 lawId 产生二元裁决的质量系统。

唯一公式固定为：

```
法案系统
=
权威正文
+ 明确边界
+ 唯一 Owner
+ 完整索引
+ 任务防漂移
+ 可复放证据
+ 失败关闭执法
```

以下组合均不构成法案系统：

只有文档，没有机器执法；
只有测试，没有 active lawId；
只有索引，没有权威正文；
只有 Owner 名称，没有 lawId 绑定；
只有正向目标，没有禁止面；
只有 PASS 汇总，没有逐法条回执；
只有 Codex 提示词，没有法案与施工图；
只有目录和空壳，没有真实 predicate；
只有静态检查，却宣称真实运行已经成立。

2. 每条规则必须同时回答五个问题

每条 active lawId 必须精确回答：

要什么；
不要什么；
谁负责；
怎么证明；
违反后怎么办。

任何不能写出明确“不要什么”的要求，不得进入 active 法案。它只能继续讨论，不能取得治理主权。

2.1 边界闭合的唯一判定

“说清楚”固定等于同时写清正边界和负边界，不能只写目标，也不能只写禁令：

治理对象	要：必须做	不要：绝对禁区
法案	`summary`、唯一 Owner、required evidence、违法后果	`forbidden[]`、不得被下级覆盖
施工图	终态对象、拓扑、目录、合同、状态、恢复和完成标准	不建设对象、禁止反向依赖、禁止第二 Owner/第二真相
规格	精确字段、枚举、状态、错误、surface、contract identity	未声明字段、宽松默认、兼容副本、隐式状态
索引与 projection	精确 source projection、导航、原子 binding 和 deterministic bytes	新增规范、人工维护、PASS/READY、第二索引

治理对象	要：必须做	不要：绝对禁区
质检	正确证据类型、完整分母、逐法条 receipt、失败关闭	证据互相冒充、漏项放行、warning/partial/fake PASS
运行逻辑	唯一入口、固定状态机、候选绑定、输出位置、清理和终止条件	隐式分支、第二 runner、候选漂移、未声明写入和双重终态

边界判定规则固定：

1. “没有写不要”不等于默认禁止；它表示边界未闭合，必须 FAIL。
2. “没有写要”不等于执行者可以自行补全；它表示生产义务未定义，必须 FAIL。
3. 合法可选项必须进入精确枚举并同时写明选择者、前置条件和失败语义；不得把“可选”当作自由发挥区。
4. 同一对象在两个位置出现不同的必做、禁区、状态、字段、错误码或输出位置时，不得择一实现；规范自身必须先完成 source-spec-amendment。

3. 终态原则

active 法案只描述完整生产系统持续成立的终态，不描述开发阶段。

以下词语或同义语不得在规范性句子中取得有效含义：

```
第一期、第二期、阶段一、阶段二、MVP、foundation、preview、planned、以后再做、后续接入、暂不支持、先搭壳、先跑起来、默认关闭、人工晋级、条件成熟后开放、可先留空、待完善
```

当前实现没有达到 active lawId 时，唯一合法结论为：

```
缺失 / 不合规 / FAIL / 必须继续建设。
```

不得通过修改名称、加入 feature flag、保留 fallback、建立兼容桥或降低 inspector 强度把缺失改写成合法状态。

4. 权威顺序与两类人类授权

必须把“人类授权”拆成两种不同主权，禁止混为一谈。

4.1 本次任务范围主权

人类对本次任务的原文只决定：

```
本次目标；
允许触碰的对象；
是否允许 law-amendment；
是否允许 source-spec-amendment；
是否允许 bootstrapMode=blank / existing；
是否属于 release-proof；
是否属于跨仓 change-set。
```

任务原文不得直接降低现役法案、改变 Owner、跳过证据或授权第二链路。若普通任务原文与 active law 冲突，固定结果为 `USER_REQUEST_CONFLICTS_WITH_ACTIVE_LAW`。

4.2 规范内容主权

项目规范顺序固定为：

```
冻结的《法案系统建设说明书》
-> 《基础法案.md》
-> 《功能法案.md》
-> 系统总施工图
-> capabilityDomain 施工图
-> 规格与机器契约
-> 本次 Task Envelope
-> Owner Manifest 与 generated enforcement bindings
-> 实现代码
-> 当前候选证据
```

-> 历史记录与讨论材料

解释规则固定为：

1. 人类明确授权“修订法案”只打开被点名法条及法定闭包的写权限；新内容仍必须符合本说明书。
2. 人类明确授权“修订本说明书”只打开 source-spec-amendment 事务；不得同时施工业务功能。
3. 下级对象与上级冲突时，修改或删除下级对象；不得建立 compatibility、fallback、shadow 或 bridge。
4. 当前代码永远不能反向证明法案应该降低。
5. generated artifact 只投影，报告只证明，消费端副本只缓存；三者均不得取得源头主权。
6. 历史报告、旧提示词、工单、会议纪要和 Git 历史只用于追溯，不能取得当前主权。

第二部分 | 固定技术底座与目录

5. 治理子系统的固定实现技术

5.1 一仓一套精确工具链

每个仓库只能有一个治理运行时和一个 canonical package manager。freeze manifest 必须记录完整版本，入口逐项核对。

```
语言：TypeScript，strict=true。
哈希：SHA-256。
文本：UTF-8、无 BOM、LF。
路径：仓库相对 POSIX 路径；有效 UTF-8；Unicode NFC；禁止绝对路径和反斜杠。
JSON：第 30 节 canonical JSON。
AST：TypeScript compiler API。
```

空白仓库默认：

```
Node.js：24 的完整补丁版本，写入 .nvmrc；
package manager：pnpm 的完整版本；
lockfile：pnpm-lock.yaml。
```

既有仓库迁移固定规则：

```
保留该仓当前唯一生产 package manager 和精确 Node pin；
不得把治理迁移顺带变成 Node/npm/pnpm 全仓迁移；
只有人类另行明确授权 toolchain source-spec-amendment 才能更换；
同一仓不得同时保留 npm 与 pnpm 两条治理命令主链；若 packageManager、lockfile、现役用户入口三者不能确定唯一结果，existing bootstrap 必须返回
 CANONICAL_PACKAGE_MANAGER_AMBIGUOUS，等待人类逐字点名后再迁移；
业务依赖可保留原策略，治理子系统新增或直接使用的依赖必须精确 pin。
```

允许的 canonical package manager 只有：

```
pnpm
npm
```

脚本逻辑名在所有仓一致；调用形式由 freeze manifest 唯一决定：

```
pnpm：pnpm <script-name> [-- args]
npm：npm run <script-name> -- [args]
```

5.2 治理依赖

治理子系统直接依赖集合精确为：

```
typescript
tsx
@types/node
zod
fast-glob
yaml
```

规则固定：

```
治理依赖必须使用完整精确版本，不得使用 ^、~、*、tag 或未解析 workspace range；
主业务可继续使用其它库，但治理 parser、compiler、runner、manifest parser、report writer 不得建立第二实现；
仓库存在 ts-morph 不等于治理系统可以使用它；治理 AST 只认 typescript compiler API；
普通任务不得升级治理工具链；升级必须走 source-spec-amendment、staging、全量 mutants 与新 freeze。
```

主业务系统使用其它语言时，治理子系统仍只保留这一套 TypeScript 实现；不得按业务语言复制第二治理系统。

5.3 projectChecks

`projectChecks` 只是 quality 的前置进程检查，不拥有 law verdict。每项只允许：

```
{
  "checkId": "typecheck",
  "scriptName": "typecheck",
  "requiredContexts": ["CHANGE", "RELEASE"],
  "allowedEphemeralWritePatterns": []
}
```

规则固定：scriptName 必须是 package.json 中唯一脚本；禁止 shell 拼接、网络下载、修改 source、修改 projection/report 或产生第二 PASS；允许写入的只能是 `.gitignore` 已排除且在 record 中精确列出的 cache/build path。运行前后 candidateId 必须相同。existing migration 以旧唯一质量工作流的真实调用图为分母，所有仍有效的 typecheck/lint/test/build-readiness 检查必须迁入本数组或由 active law 明确退役，不得由 Codex自行删减。

6. 固定目录树

法案系统建成后，以下路径精确存在；带“按需”的目录只有存在对应 active evidence binding 时才建立：

```
AGENTS.md
tsconfig.json

.github/instructions/agent-rules/
  基础法案.md
  功能法案.md

docs/governance/
  法案系统建设说明书-冻结定稿版.md
  GOVERNANCE-FREEZE.json

docs/blueprints/
  SYSTEM-BLUEPRINT.md
  domains/<capabilityDomain>/<CAPABILITY-DOMAIN>-BLUEPRINT.md

docs/specs/
  <capabilityDomain>/<spec-id>.spec.json

scripts/audit/
  README.md
  core/
    request-capture.ts
    entry-gate.ts
    task-start.ts
    abandon-task.ts
    propose-task-scope.ts
    law-parser.ts
    law-history-check.ts
    shadow-law-detector.ts
    manifest-parser.ts
    blueprint-parser.ts
    spec-parser.ts
    canonical-json.ts
    hash.ts
    candidate-identity.ts
    candidate-mutation-sentinel.ts
    ownership-check.ts
    artifact-compiler.ts
    build-audit-artifacts.ts
    projection-check.ts
    route-task.ts
    scope-check.ts
    surface-delta.ts
    amendment-transaction.ts
    bootstrap-existing.ts
    run-generated-plan.ts
    aggregate-quality.ts
    run-quality-check.ts
    run-governance-mutants.ts
    report-writer.ts
  shared/
    contracts.ts
    path-policy.ts
    evidence-index.ts
    evidence-ref.ts
    inspector-contract.ts
    component-receipt.ts
  surface-extractors/
    registry.ts
    <extractor-id>.ts
```

```
  domains/<capabilityDomain>/
   manifest.json
   inspector.ts
   predicates.ts
  generated/
   law-registry.generated.json
   authority-census.generated.json
   enforcement-bindings.generated.json
   audit-plan.generated.json
   inspector-registry.generated.ts
  reports/
   <capabilityDomain>.report.json

.governance/runtime/
  user-request.txt
  task-envelope.json
  task-proposal.json
  task-baseline.json
  route-receipt.json
  task-context.md
  task-lock.json
  amendment-plan.json
  migration-contract.json
  evidence/static/**
  evidence/exam/**
  evidence/dynamic/**
  evidence/final/**
  mutants/**
  cross-repo/**

scripts/exam/                          # 按需
  exam-registry.ts
  run-exam.ts
  exams/<capabilityDomain>/<exam-id>.ts

scripts/dynamic-proof/                 # 按需
  proof-registry.ts
  run-proof.ts
  proofs/<capabilityDomain>/<proof-id>.ts

tests/governance/
  parser-contract.test.ts
  compiler-determinism.test.ts
  binding-integrity.test.ts
  route-closure.test.ts
  scope-drift.test.ts
  amendment-transaction.test.ts
  existing-migration.test.ts
  composite-evidence.test.ts
  cross-repo-contract.test.ts
  inspector-mutants.test.ts
  governance-mutants.test.ts
  mutants/manifest.json
```

模板中的 `<...>` 只表示 bootstrap 时必须替换的元变量。active 文件、generated 文件、manifest、blueprint、spec、代码和报告中不得残留任何占位符。

路径规则固定为：

1. `.governance/runtime/**` 必须进入 `.gitignore`，不得提交。
2. `scripts/audit/generated/**` 只能由唯一 `audit:build` 写入。
3. `scripts/audit/reports/**` 只作为 `invocationMode=TASK` 的当前候选 report cache；只能由唯一 final aggregator 经 `report-writer.ts` 写入，必须被 `.gitignore` 排除且不得提交，不进入 candidateId、法案主权或实现 diff。`CI_CANDIDATE` 绝对不得写该目录。
4. existing migration 必须把历史已跟踪 `scripts/audit/reports/**` 从 Git index 物理删除；需要留档时只上传 candidate-bound proof package。
5. 每个 domain 的 `manifest.json`、`inspector.ts`、`predicates.ts` 只由该 domain 拥有。
6. Exam 与 Dynamic Proof 按 domain 分目录；共享 registry/runner 只有一份。
7. 不得建立任何带 v2/new/legacy/manual/compat/shadow 后缀的同职路径。
8. 不得在 `src/**`、`tools/**`、`scripts/ci/**` 或业务目录复制 compiler、Owner mapping、report writer、law parser、route 或 quality aggregator。

9. PDF 不在固定目录树中；只有 freeze manifest 明确 `derivatives.pdf.enabled=true` 时，才按其固定路径生成非权威派生物。
10. Manifest `ownedPaths` 只声明路径出现时的唯一 primary 写主权，不证明路径或能力已经存在，也不授权预建空目录、占位文件或未来 surface；物理存在必须由 active binding 和当前终态实现共同要求。
11. `TASK` 的 decision/component/LawQualityReceipt 使用 `.governance/runtime/evidence/**`；`CI_CANDIDATE` 的全部 decision、component、LawQualityReceipt、Owner report、mutant receipt 和 invocation receipt 必须写入仓库根 realpath 之外的 OS/CI ephemeral execution root。

第三部分 | 冻结文件与两份 active 法案正文

7. 本说明书的冻结机制

`docs/governance/GOVERNANCE-FREEZE.json` 顶层只允许：

```
{
  "schemaVersion": 2,
  "status": "SPEC_FROZEN",
  "specPath": "docs/governance/法案系统建设说明书-冻结定稿版.md",
  "specSha256": "<64 lowercase hex>",
  "freezeVersion": "FROZEN-FINAL-V3.1",
  "amendmentAuthority": "explicit-human-only",
  "entryFile": {
    "path": "AGENTS.md",
    "sha256": "<64 lowercase hex>"
  },
  "packageManager": {
    "name": "pnpm",
    "version": "<exact version>"
  },
  "toolchain": {
    "node": "<exact version>",
    "typescript": "<exact version>",
    "tsx": "<exact version>",
    "zod": "<exact version>",
    "fastGlob": "<exact version>",
    "yaml": "<exact version>",
    "typesNode": "<exact version>"
  },
  "projectChecks": [
    {
      "checkId": "typecheck",
      "scriptName": "typecheck",
      "requiredContexts": ["CHANGE", "RELEASE"],
      "allowedEphemeralWritePatterns": []
    }
  ],
  "derivatives": {
    "pdf": {
      "enabled": false,
      "path": null,
      "sourceSha256": null,
      "fileSha256": null
    }
  }
}
```

冻结规则：

1. `specSha256` 对本说明书原始 UTF-8 字节计算 SHA-256；文件不得自写自身 hash。
2. `entryFile.sha256` 对完整 `AGENTS.md` 原始字节计算。
3. 工具链和 package manager 必须是 lockfile 与实际运行时的完整版本，不得保留示例值。
4. `projectChecks` 按 checkId 排序且无重复；quality 只能通过 canonical package manager 调用这些 package scripts。blank 精确包含 typecheck；existing 必须包含旧唯一质量工作流实际调用的全部只读或仅写 ignored-cache 检查，不得静默删除。
5. `governance:entry` 每次任务核对全部 hash、版本和 projectChecks。
6. `SPEC_FROZEN` 只说明规范文本被锁定，不说明实现已经通过。
7. `SYSTEM_PROVEN` 与一个 candidateId 和 RELEASE context 绑定；任何 candidate 变化立即失效。
8. `SYSTEM_PROVEN` 不写回 freeze manifest，不反向取得规范主权；需要留档时只导出不可变 proof package。
9. 只有用户原文明确包含“授权修订《法案系统建设说明书》”并点名范围时，才允许 source-spec-amendment。
10. source-spec-amendment 必须走第 39.2 节 staging transaction；不得在真实工作树暴露中间不一致。
11. PDF 默认关闭。开启时只作为 Markdown 的非权威派生物并绑定 source hash；PDF 不存在或未更新不得阻断默认关闭的冻结交付。

12. Markdown 标题、文件名或版本字符串不得单独宣称 `SPEC_FROZEN`。freeze manifest 缺失、`specPath` 与 realpath 不一致、任一 hash/pin 为示例值或 entry 复验未通过时，状态只能是 `FINAL_SOURCE_VALIDATED`，不得写成已冻结或已实证。

8. active 法案正文精确为两份，shadow law 精确为零

人工维护并拥有 **active law corpus** 主权的文件精确为：

```
.github/instructions/agent-rules/基础法案.md
.github/instructions/agent-rules/功能法案.md
```

只有这两份文件中的 `law-record` fence 会被 active law parser 读取并进入 registry。

8.1 冻结模板与 active law 的边界

本说明书是高于 active law corpus 的冻结建设规范。第 66、67 节可以包含精确 32 个 `law-template` fence，用于定义 blank bootstrap 与 existing migration 的固定 meta-law 字节；这些 template：

```
只允许出现在 exact path `docs/governance/法案系统建设说明书-冻结定稿版.md`；
只有 spec hash 与 freeze 完全一致时可读取；
只能由 `law-parser.ts` 的固定 `parseFrozenMetaLawTemplates` 模式读取；
不得直接进入 law registry、Owner report 或当前 law count；
bootstrap renderer 只把 fence label `law-template` 确定性替换为 `law-record`，JSON、标题和人读镜像其它字节不得改变；
渲染后分别进入基础法案和功能法案，随后只以两份 active law 文件为运行时 law source。
```

第 13 节只允许一个 `law-record-example` fence；它是结构示意，不是合法 law block。任何其它 `law-template` 或 `law-record-example` 立即失败。

允许承接或投影法案、但不得创造新义务的对象只有：

```
freeze hash 保护的固定 AGENTS.md；
SYSTEM-BLUEPRINT 与 domain blueprint machine block；
spec descriptor；
Manifest；
generated projection；
task-context 原字节投影；
当前候选 component/final receipts 与 reports。
```

8.2 shadow-law detector

`shadow-law-detector.ts` 扫描全部非忽略、非 runtime 的 Markdown、instruction、YAML、JSON、TS/JS 注释和文本配置。规则固定：

1. 两份 active law 文件之外发现实际 `law-record` fence，返回 `SHADOW_LAW_BLOCK_FOUND`。
2. `law-template` 不是第 66/67 节精确 32 个 frozen template，或 `law-record-example` 不是第 13 节精确一个示例，返回 `SOURCE_SPEC_TEMPLATE_DRIFT`。
3. 除 exact-hash 本说明书、两份 active law 和 exact-hash `AGENTS.md` 外，Markdown/instruction 普通文本行出现中文规范词 `必须|不得|只能|禁止|务必|应当` 或英文规范词 `MUST|MUST NOT|SHALL|SHALL NOT|REQUIRED|FORBIDDEN`，返回 `UNBOUND_NORMATIVE_TEXT`。
4. blueprint/spec/Manifest 只能通过本说明书规定的机器字段承接 Owner、依赖、surface、proof 和 contract；自由文本不得补充规范边界。
5. 生产代码注释中的规范词同样扫描；代码字符串或 fixture 只有被 parser 识别为数据且生产链不消费为治理规则时才不取得主权。
6. task-context 位于 gitignored runtime，只复制带 hash 的权威原字节，不构成第三法案源。
7. existing migration 后，旧 instructions、README、manual、policy、constitution、guardrails 和 special rules 只能删除规范内容、降为纯描述导航或移入 `docs/history/**`，并保证生产入边为零。

scanner 不做语义猜测，不接受人工“意思相同”豁免。

9. 两份法案的固定 front matter

《基础法案.md》开头精确使用：

```
---
documentType: base-law
schemaVersion: 2
status: active
authorityLevel: 100
containerOwner: law-system
lawIdPrefix: LAW
---
```

《功能法案.md》开头精确使用：

```
---
documentType: functional-law
schemaVersion: 2
status: active
authorityLevel: 90
containerOwner: law-system
lawIdPrefix: LAW
---
```

任何字段缺失、额外字段、值不一致、重复 key、重复 front matter 或第二元数据区均必须 FAIL。

10. 基础法案只容纳 project-governance 根不变量

基础法案的物理结构精确为：

```
# 基础法案

## capabilityDomain: project-governance

### [LAW-PROJECT-GOVERNANCE-001-V1] ...
...
### [LAW-PROJECT-GOVERNANCE-010-V1] ...
```

固定边界：

1. 基础法案当前只允许一个 domain 区块：`project-governance`。
2. 第 66 节十条根法条必须完整出现，lawId、标题、机器 JSON、人读镜像、顺序和字节语义不得自行改写。
3. parser 对基础法案与功能法案使用相同的最近 `## capabilityDomain:` 算法，不存在隐式 domain 或例外前缀。
4. `project-governance` 的逻辑 Owner、Manifest、blueprint、spec 和 evidence bindings 必须完整存在。
5. 新增第二个基础 domain 必须先 source-spec-amendment；普通 law-amendment 无权改变基础法案容器结构。

一条规则只有同时满足下列条件才可能进入基础法案：

```
删除任一具体业务能力后仍成立；
对全部 capabilityDomain 生效；
不会因新增产品、页面、API 或算法改变；
属于权威、唯一真相、Owner、证据、失败关闭、Codex 范围、历史退役或冻结；
能够由 project-governance 的固定机器合同执法。
```

基础法案不得包含产品流程、业务字段、页面、API、表、provider、业务状态机、短期迁移、单次任务范围或函数级实现。

11. 功能法案容纳全部非基础法条

功能法案按以下层级精确组织：

```
# 功能法案

## capabilityDomain: audit-system

### [LAW-AUDIT-SYSTEM-001-V1] <标题>
...
```

```
## capabilityDomain: law-system

### [LAW-LAW-SYSTEM-001-V1] <标题>
...
```

固定规则：

1. 初始治理 corpus 的功能法案精确包含 `audit-system` 和 `law-system` 两个 meta domain；`project-governance` 只在基础法案。
2. 业务 domain 与 meta domain 一起按 capabilityDomain 字典序排列。
3. 一个区块只容纳同名 domain 的 lawId；lawId domain segment 必须与 heading 一致。
4. 区块内按 lawKey 编号与版本升序排列。
5. 功能法案允许很长，不得按文件长度拆成第二 active 法案。
6. 物理文件完整性 Owner 是 `law-system`；每个 block 的逻辑 Owner 是 metadata.capabilityDomain。
7. 修改一个 domain 只能改其区块和法定闭包，不得格式化或整理其它区块。

第四部分 | lawId 与法条原子语法

12. lawId 固定格式、lawKey 与历史谱系

lawId 正则固定为：

```
^LAW-[A-Z0-9]+(?:-[A-Z0-9]+)*-[0-9]{3}-V[1-9][0-9]*$
```

映射规则：

```
capabilityDomain = lower-kebab-case
lawId domain segment = capabilityDomain 转大写
lawKey = 删除末尾 -V<major> 的 lawId
```

示例：

```
capabilityDomain: account-person-workspace
lawKey: LAW-ACCOUNT-PERSON-WORKSPACE-001
lawId: LAW-ACCOUNT-PERSON-WORKSPACE-001-V1
```

编号与版本规则：

1. 每个 domain 从 `001` 单调递增。
2. V1 的 `supersedesLawId` 必须为 `null`。
3. Vn（n>1）必须精确 supersede 同 lawKey 的 V(n-1)，不得跳号。
4. requirement、forbidden、appliesTo、Owner、evidenceModes、failure 条件或规范语义任一变化，major 必须加一。
5. 新 major 进入当前工作树时，旧 block、binding、check、spec 当前引用必须在同一 amendment 中物理删除。
6. 当前工作树只允许 active 最新版本；历史只在 Git。
7. 无关语义必须申请新编号，不得复用旧 lawKey。

`law-history-check.ts` 的唯一算法：

```
1. 只读取当前 HEAD 可达的全部祖先 commit，不读取其它本地分支、远端 ref 或网络；
2. 枚举两份法案路径在这些 commit 中出现过的唯一 blob；
3. 用当前版本 parser 兼容读取可识别历史 block；无法解析的历史 blob只记录为历史格式，不取得当前主权；
4. 对每个完整 lawId 计算规范指纹：capabilityDomain + tier + evidenceModes + summary + appliesTo + excludes +
 forbidden + dependsOnLawIds；
5. 同一完整 lawId 出现两个不同规范指纹 -> LAW_HISTORY_ID_REWRITTEN；
6. Vn 缺少 V(n-1) 历史或 supersedes 不匹配 -> LAW_HISTORY_LINEAGE_BROKEN；
7. 当前新 V1 的 lawKey 已在历史出现 -> LAW_HISTORY_KEY_REUSED；
8. 检查只读，不自动改号、不访问网络。
```

13. 每条 active 法条的精确 Markdown 结构

下面使用 `law-record-example` 只为避免本说明书自身被 active parser 误识别。active 两份法案中的实际 fence label 必须是 `law-record`，其它结构和字节顺序精确相同：

```
### [LAW-<DOMAIN>-001-V1] <中文标题>
```

{ ...canonical law JSON... }

```
#### 要

- <与 summary 字节相同的句子>

#### 不要

- <与 forbidden[0] 字节相同的句子>

#### 证据
```

```
- <与 evidenceModes[0] 字节相同的枚举值>

#### 违法后果

- <由 tier 对应的固定句子>
```

`law-parser.ts.parseFrozenMetaLawTemplates` 必须把上述内层 `law-record` 视为被外层示例 fence 包裹的数据，不得把它计入 active block；active `law-parser.ts` 只接受两份法案中未嵌套、未缩进、行内容精确等于三个反引号加 `law-record` 的 fence。

法条 section 不得增加背景故事、当前实现盘点、开发步骤、TODO、方案比较、未来考虑、兼容说明、临时豁免或未进入 metadata 的隐藏要求。实现落位进入 blueprint；字段与 machine contract 进入 spec；任务范围进入 runtime Task Envelope；原因追溯进入非权威历史记录。

14. law-record JSON 精确字段

每个 block 只允许：

```
{
  "schemaVersion": 2,
  "lawId": "LAW-<DOMAIN>-001-V1",
  "supersedesLawId": null,
  "status": "active",
  "tier": "P0",
  "capabilityDomain": "<lower-kebab-case>",
  "enforcementMode": "static",
  "evidenceModes": ["static.ast"],
  "summary": "必须……",
  "appliesTo": ["src/<domain>/**"],
  "excludes": [],
  "forbidden": ["不得……"],
  "dependsOnLawIds": [],
  "routingTags": ["<lower-kebab-case>"]
}
```

字段约束：

14.1 schemaVersion、status、supersedesLawId

```
schemaVersion 固定整数 2；
status 固定 active；
V1 supersedesLawId=null；
Vn 精确指向同 lawKey 的 V(n-1)。
```

14.2 tier

只允许 `P0`、`P1`、`P2`；全部都是失败关闭，不存在 warning law。

14.3 enforcementMode 与 evidenceModes

`enforcementMode` 只允许 `static`、`exam`、`dynamic`、`composite`。

```
只有 static.* -> static
只有 exam.* -> exam
只有 dynamic.* -> dynamic
存在两类或三类 -> composite
```

`evidenceModes` 只允许：

```
static.filesystem
static.text
static.ast
static.types
static.contract
static.dependency
static.config
static.json
```

static.yaml
static.sql
static.migration
exam.golden
exam.property
exam.state-machine
dynamic.browser
dynamic.service
dynamic.database
dynamic.storage
dynamic.provider
dynamic.authorization
dynamic.recovery
dynamic.performance

数组非空、无重复、按上表排序。

14.4 summary

以“必须”或“只能”开头；
只描述一个可独立裁决的不变量；
12 至 240 个 Unicode code point；
不得出现建议、尽量、可能、一般、视情况、以后、后续、暂时、可选、原则上；
不得写开发步骤、文件清单、阶段或任务范围。

14.5 appliesTo 与 excludes

仓库相对 POSIX glob；有效 UTF-8 NFC；不得绝对路径、..、反斜杠、负向 glob、brace、extglob、字符组或问号；
只允许精确文件、目录 /**、目录 /**/*.<literal-extension>；
数组字典序、无重复、无包含冗余；
appliesTo 非空；
只有 project-governance、law-system、audit-system 元法条允许根级 **/*；
excludes 只能排除 appliesTo 内明确子面，不得排除生产主链逃避执法。

14.6 forbidden

1 至 16 项；每项以“不得”或“禁止”开头；一项一个违法条件；不得包含阶段豁免。

14.7 dependsOnLawIds

只能引用当前 active lawId；不得自引；依赖图无环；跨域依赖必须同时存在于双方 blueprint machine block；不得分散 Owner。

14.8 routingTags

1 至 16 个 lower-kebab-case；只用于 exact route 导航；不得作为 PASS 证据；不得使用相关、通用、其它、misc 等模糊词。

15. 人读镜像必须与 JSON 精确一致

`law-parser.ts` 必须验证：

标题 lawId == metadata.lawId；
所在 capabilityDomain heading == metadata.capabilityDomain；
lawId domain segment == capabilityDomain 大写；
“要”唯一 bullet == summary；
“不要”bullets == forbidden，数量、顺序、字节一致；
“证据”bullets == evidenceModes，数量、顺序、字节一致；
“违法后果” == tier 固定句子；
section 中无其它规范性 bullet；
supersedesLawId 与 major 版本关系成立。

任一不一致必须 FAIL，不得选择相信其中一边。

16. tier 固定违法后果文本与 parser 算法

人读镜像固定使用：

P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。
P1：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P1 finding；对应质量入口必须非零退出，并阻断合并和发布。
P2：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P2 finding；对应质量入口必须非零退出，并阻断合并。

16.1 law-parser 的精确解析算法

1. 读取原始字节；拒绝 BOM、非 UTF-8、非 NFC 路径和 CRLF；
2. 验证 front matter 从首字节开始，以精确 ---\n 打开和关闭；
3. 唯一 yaml parser 解析，禁止重复 key；
4. 逐行维护 Markdown fence 状态；
5. 只有精确行 ```law-record 打开法条 block；只有精确行 ``` 关闭；
6. 禁止未关闭、嵌套、缩进 fence 和 block 内注释；
7. block 必须是单一 JSON object，禁止未知字段；
8. 找到最近 level-3 law heading，一个 heading 精确一个 block；
9. 找到最近 level-2 capabilityDomain heading；基础法案和功能法案算法相同；
10. 验证第 15 节镜像、domain、major 与 supersedes；
11. 记录原始 lineStart/lineEnd、metadata hash、section hash；
12. EOF 后拒绝孤立 heading、孤立“要/不要/证据/违法后果”；
13. 任一解析错误只产生结构冲突，不生成部分 registry。

16.2 其它 parser

`manifest-parser.ts`、`blueprint-parser.ts`、`spec-parser.ts` 必须拒绝重复 key、未知字段、空字符串、非法路径、未替换占位符和默认补值。

`shadow-law-detector.ts` 必须在 compiler 生成前执行；`law-history-check.ts` 必须在 projection check 与 quality check 中执行。

第五部分 | capabilityDomain、Owner 与 Manifest

17. capabilityDomain 命名与创建

capabilityDomain 正则固定为：

```
^[a-z0-9]+(?:-[a-z0-9]+)*$
```

每个 capabilityDomain 必须代表一个能够独立回答下列问题的完整责任域：

```
它拥有哪一种事实或能力；
它创建什么；
它读取什么；
它修改什么；
它裁决什么；
它明确不得接管什么。
```

不得创建以下 domain：

```
common、shared、utils、misc、core2、platform2、general、helpers、temporary、legacy、compat。
```

新 capabilityDomain 只有在用户明确授权新增能力域，并且现有 domain 无法合法承接时才允许创建。Codex 不得为了整理代码、追求优雅或减少文件行数自行新增 domain。

18. 物理容器 Owner 与逻辑 law Owner 分离

固定关系：

```
基础法案.md / 功能法案.md 的文件完整性 Owner = law-system
每个 law-record block 的逻辑 Owner = metadata.capabilityDomain
```

因此：

```
一个物理文件可以容纳多个逻辑 Owner；
一个 lawId 只能有一个逻辑 Owner；
Owner 不得通过拥有法案文件路径接管其它 lawId；
Audit 必须按 lawId block 定位主权，不得按 Markdown 文件整体绑定。
```

19. Manifest 固定路径与固定文件名

每个 active capabilityDomain 精确存在：

```
scripts/audit/domains/<capabilityDomain>/manifest.json
scripts/audit/domains/<capabilityDomain>/inspector.ts
scripts/audit/domains/<capabilityDomain>/predicates.ts
```

目录名、manifest.capabilityDomain、inspector literal capabilityDomain、report capabilityDomain 必须完全相同。

不得设置 alias、prefix fallback、umbrella owner 或共享 report。

20. Manifest 精确 schema：一个 lawId 一个原子 binding

Manifest 顶层只允许：

```
{
  "schemaVersion": 2,
  "capabilityDomain": "<lower-kebab-case>",
  "status": "active",
  "allowedLawIds": ["LAW-..."],
```

```
  "blueprintRefs": ["docs/blueprints/domains/.../...-BLUEPRINT.md"],
  "specRefs": ["docs/specs/.../...spec.json"],
  "lawBindings": {
    "LAW-...": {
      "blueprintRef": "docs/blueprints/domains/.../...-BLUEPRINT.md#anchor:law.LAW-...",
      "specRefs": ["docs/specs/.../...spec.json"],
      "staticCheckId": "<domain>.<check-id>",
      "examBindings": [
        {
          "examId": "<domain>.<exam-id>",
          "evidenceModes": ["exam.property"],
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": []
    }
  },
  "ownedPaths": ["src/<domain>/**"],
  "observedPaths": [],
  "evidenceModes": ["static.ast", "exam.property"]
}
```

`proofBindings` 元素精确为：

```
{
  "proofId": "<domain>.<proof-id>",
  "evidenceModes": ["dynamic.service"],
  "frequency": "EVERY_RELEVANT_CHANGE"
}
```

约束固定：

1. schemaVersion=2，status=active。
2. `allowedLawIds` 精确等于该 domain 当前全部 active lawId。
3. `lawBindings` keys 与 `allowedLawIds` 精确相等。
4. 每个 law binding 只有一个稳定 blueprint anchor；anchor 必须存在。
5. `specRefs` 顶层精确等于全部 law binding specRefs 的去重并集。
6. law 含 static.* 时 `staticCheckId` 必须为唯一字符串；不含 static.* 时必须为 `null`。
7. law 的 exam.* 必须被 `examBindings.evidenceModes` 精确覆盖；无 exam.* 时数组必须为空。
8. law 的 dynamic.* 必须被 `proofBindings.evidenceModes` 精确覆盖；无 dynamic.* 时数组必须为空。
9. binding 不得声明法条未要求的 evidenceMode。
10. checkId、examId、proofId 全项目唯一，格式 `<domain>.<lower-kebab-case>`。
11. frequency 只允许 `EVERY_RELEVANT_CHANGE`、`RELEASE_CANDIDATE`。
12. `blueprintRefs` 精确等于全部 binding 使用的 blueprint 文件集合。
13. `evidenceModes` 精确等于 domain 法条 evidenceModes 去重并集。
14. `ownedPaths` 是 primary 写主权；`observedPaths` 只读，不赋予写权。
15. Manifest 不得包含 owner、entry、reportPath、PASS 条件、豁免或新法案义务。
16. 不得再使用分离的 `lawBlueprintBindings`、`lawCheckBindings`、lawIds/checkIds 平行数组作为主权映射。

20.1 ownedPaths / observedPaths 唯一 pattern

只允许精确文件或目录 `/**`。禁止根级宽泛 Owner、glob 元字符、尾随斜杠、`.`、`..`、符号链接越界、大小写 alias 和“最具体者优先”。任何命中重叠直接失败。

21. 候选文件角色与 candidate-aware ownership

每个 candidate path 必须按以下顺序取得一个且只有一个 `fileRole`：

```
1. AGENTS.md、两份 active law、冻结说明书、freeze manifest -> GOVERNANCE_AUTHORITY；
2. docs/blueprints/**、docs/specs/** -> BLUEPRINT_OR_SPEC；
3. scripts/audit/generated/** -> GENERATED_PROJECTION；
```

4. scripts/audit/reports/** -> AUDIT_REPORT；
5. .governance/runtime/** -> RUNTIME_IGNORED；
6. tests/** -> TEST_ONLY；fixtures/** -> FIXTURE_ONLY；examples/** -> EXAMPLE_ONLY；docs/history/** -> HISTORY_ONLY；
7. 其余 PRESENT/DELETED path -> PRODUCTION。

机器规则：

1. 角色判定由 `ownership-check.ts` 基于状态感知 candidate records 执行，不属于纯 compiler。
2. compiler 只验证 Manifest claims 无重叠和 law bindings 完整，不得返回候选级 `UNOWNED_FILE`。
3. 非生产角色只要被生产 import、package script、运行配置、发布包、代码生成、迁移或业务状态写入消费，立即升级为 PRODUCTION。
4. PRESENT 的 PRODUCTION 文件必须精确命中一个 Manifest `ownedPaths`。
5. DELETED record 不读取工作树字节；删除权限由 baseline Owner、pathClaim、surface delta 和 retirement closure 验证。
6. rename 规范化为旧 path DELETE + 新 path PRESENT；不得保留 alias 或双入口。
7. symlink 只 hash link target text，禁止越出仓库根；submodule、unmerged index、特殊设备 file mode 直接 blocked。
8. path 必须是有效 UTF-8、Unicode NFC、仓库相对 POSIX；大小写折叠冲突或同一路径多编码表示直接 blocked。

固定结果：

PRODUCTION PRESENT 零 Owner -> UNOWNED_PRODUCTION_FILE；
PRODUCTION PRESENT 多 Owner -> MULTIPLE_PRIMARY_OWNERS；
非生产路径被生产消费但未升级 -> ROLE_ESCALATION_REQUIRED；
DELETED 无合法 retirement claim -> UNAUTHORIZED_PRODUCTION_DELETE；
新增一级目录无人类架构/domain 授权 -> NEW_TOP_LEVEL_PATH_UNAUTHORIZED。

Codex 不得先创建路径、入口或配置再补 Owner。

第六部分 | 施工图、规格与契约的精确边界

22. 系统施工图与 domain 施工图的固定身份

22.1 SYSTEM-BLUEPRINT

路径：

```
docs/blueprints/SYSTEM-BLUEPRINT.md
```

front matter 精确为：

```
---
documentType: system-blueprint
schemaVersion: 2
status: active
blueprintId: system
owner: project-governance
---
```

正文第一个 fence 必须是唯一 `system-blueprint-record` JSON block，顶层只允许：

```
{
  "schemaVersion": 2,
  "blueprintId": "system",
  "status": "active",
  "owner": "project-governance",
  "domainInventory": [],
  "dependencyEdges": [],
  "crossDomainContracts": [],
  "crossRepoContracts": [],
  "surfaceExtractorBindings": [
    {
      "extractorId": "governance.package-json",
      "pathPattern": "package.json",
      "surfaceKinds": ["package-dependency", "package-script"]
    }
  ]
}
```

元素 schema 在附录 C 固定。系统施工图必须机器列出全部 active domain、primary dependency DAG、跨域合同 Owner、跨仓合同 pin 和 surface extractor；不得靠正文叙述补全。

22.2 capabilityDomain blueprint

路径：

```
docs/blueprints/domains/<capabilityDomain>/<UPPER-DOMAIN>-BLUEPRINT.md
```

front matter 精确为：

```
---
documentType: capability-blueprint
schemaVersion: 2
status: active
blueprintId: domain:<lower-kebab-case>
capabilityDomain: <lower-kebab-case>
owner: <lower-kebab-case>
---
```

正文第一个 fence 必须是唯一 `domain-blueprint-record` JSON block，包含：

```
{
  "schemaVersion": 2,
  "blueprintId": "domain:<domain>",
  "status": "active",
  "capabilityDomain": "<domain>",
```

```
  "owner": "<domain>",
  "lawIds": [],
  "specRefs": [],
  "anchors": [],
  "providedContracts": [],
  "consumedContracts": [],
  "dependencyEdges": [],
  "prohibitedDependencyEdges": [],
  "surfaceExtractorBindings": []
}
```

`anchors` 元素：

```
{
  "anchorId": "law.LAW-<DOMAIN>-001-V1",
  "kind": "law",
  "lawIds": ["LAW-<DOMAIN>-001-V1"]
}
```

稳定 blueprintRef 格式唯一为：

```
<blueprint-path>#anchor:<anchorId>
```

禁止依赖 Markdown 自动标题 anchor、行号或自然语言章节名。

23. domain blueprint 的人读章节

机器 block 之后精确包含：

```
0. 文件身份与承接 lawId
1. 要建设的终态对象
2. 不建设的对象
3. 唯一 Owner 与相邻 Owner
4. 目录与文件落位
5. 输入合同
6. 输出合同
7. 数据与持久化
8. 运行时状态机
9. 调用方向与反向依赖边界
10. 错误、阻断与恢复
11. 安全、隐私与权限
12. 静态证据
13. Exam 证据
14. Dynamic Proof 证据
15. 完成标准
```

人读章节只能解释机器 block 和 active law，不得出现未绑定规范词，不得新增 surface、proof frequency、contract、Owner 或例外。不存在 Exam/Dynamic binding 时，章节写描述性事实“当前 active law 无对应 binding”，不得静默省略。

24. 规格描述符的唯一 schema

规格文件路径精确为：

```
docs/specs/<capabilityDomain>/<spec-id>.spec.json
```

`spec-parser.ts` 只接受一个以 `specKind` 为判别字段的封闭 discriminated union。任何 spec 只能命中以下三个分支之一，不能把不同分支的字段混在同一对象中。

普通 domain spec 分支顶层只允许：

```
{
  "schemaVersion": 1,
  "specId": "<capabilityDomain>.<lower-kebab-case>",
  "status": "active",
  "capabilityDomain": "<capabilityDomain>",
  "owner": "<capabilityDomain>",
  "specKind": "typed-contract",
  "version": 1,
  "lawIds": [],
```

```
  "blueprintRefs": [],
  "authorityArtifacts": [],
  "providedContracts": [],
  "consumedContracts": [],
  "surfaces": [],
  "states": [],
  "errorCodes": []
}
```

普通分支的 `specKind` 只允许：

```
typed-contract
api
event
storage
state-machine
config
security
proof
```

provider 跨仓合同分支顶层只允许：

```
{
  "schemaVersion": 1,
  "specId": "<capabilityDomain>.<lower-kebab-case>",
  "status": "active",
  "capabilityDomain": "<capabilityDomain>",
  "owner": "<capabilityDomain>",
  "specKind": "cross-repo-contract",
  "version": 1,
  "lawIds": [],
  "blueprintRefs": [],
  "authorityArtifacts": [],
  "contractId": "<provider-domain>.<contract>",
  "providerRepo": "owner/name",
  "providerDomain": "<capabilityDomain>",
  "authorityArtifact": {
    "path": "repo/relative/path",
    "selector": "<stable-selector>",
    "language": "json",
    "sha256": "<64 lowercase hex>"
  },
  "consumerRepos": [],
  "surfaces": [],
  "states": [],
  "errorCodes": []
}
```

consumer pin 分支顶层只允许：

```
{
  "schemaVersion": 1,
  "specId": "<capabilityDomain>.<lower-kebab-case>",
  "status": "active",
  "capabilityDomain": "<capabilityDomain>",
  "owner": "<capabilityDomain>",
  "specKind": "contract-pin",
  "version": 1,
  "lawIds": [],
  "blueprintRefs": [],
  "authorityArtifacts": [],
  "contractId": "<provider-domain>.<contract>",
  "providerRepo": "owner/name",
  "providerDomain": "<provider-domain>",
  "providerSpecRef": "docs/specs/<provider-domain>/<spec-id>.spec.json",
  "providerCommitSha": "<40 lowercase hex>",
  "contractSha256": "<64 lowercase hex>",
  "cachePath": null,
  "surfaces": [],
  "states": [],
  "errorCodes": []
}
```

规则固定：

1. 三个分支共享的 identity 字段为 `schemaVersion/specId/status/capabilityDomain/owner/specKind/version/lawIds/blueprintRefs/authorityArtifacts/surfaces/states/errorCodes`；字段缺失、额外字段或跨分支字段混用均为 `SPEC_SCHEMA_INVALID`。
2. spec 只描述精确机器合同，不写原则性口号。

3. `authorityArtifacts` 每项包含 path、selector、language；只有一个 Owner。
4. 普通分支的 `providedContracts.contractId` 全项目唯一；消费端只能通过 `consumedContracts` 引用，不得复制主权 schema。
5. `cross-repo-contract` 的 `providerDomain` 必须等于 `capabilityDomain` 和 `owner`，`contractId` 全局唯一；它不得出现 `providedContracts`、`consumedContracts` 或 `contractSha256`。
6. `contract-pin` 的 `capabilityDomain/owner` 是消费端 Owner，`providerDomain` 是唯一 provider；它不得出现 `providedContracts`、`consumedContracts`、`authorityArtifact` 或 `consumerRepos`。
7. `contractSha256` 精确等于 `providerSpecRef` 对应 provider descriptor 的 canonical raw bytes SHA-256；provider descriptor 不得包含自身文件 hash，避免自引用 hash。
8. `cachePath` 只允许 `null` 或消费仓内 exact-byte generated projection 路径；cache 不取得 contract 主权，也不得成为 fallback。
9. surface 记录包含 kind、identity、ownerDomain、lawIds、extractorId。
10. state 与 error code 必须是 literal 列表；不得由 Markdown 推断。
11. version 单调递增；破坏性改变必须 major 增加并在同一任务更新全部消费者，禁止 dual-read。
12. law 的全部 specRefs 必须来自 Manifest 原子 binding；route 不做猜测。
13. compiler 输入必须包含全部被绑定 spec，并按唯一 discriminated union 验证 owner、lawId、blueprintRef、contract、pin 与 surface identity。

25. 法案、施工图、规格与跨域合同的唯一分工

```
法案：必须、禁止、Owner、证据、后果。
系统施工图：domain inventory、dependency DAG、跨域与跨仓 topology。
domain 施工图：该完整建筑的落位、状态、调用、恢复与稳定 anchors。
spec：字段、schema、API、事件、状态、错误、surface 与 contract identity。
Task Envelope：本次路径、surface、closure 与验证，不拥有长期主权。
```

每个 active lawId 精确绑定一个 domain blueprint anchor，绑定零个或多个 spec；需要字段、contract、surface、状态或 proof 时 specRefs 不得为空。

跨域 contract 规则：

```
一个 contractId 只有一个 provider domain 与一个 provider spec；
消费者只引用 contractId + provider spec version；
依赖方向同时存在于 SYSTEM-BLUEPRINT 和双方 domain blueprint；
禁止复制 v2 contract、compat schema、optional fallback、反向 import 或双 Owner。
```

跨仓 contract 只使用第 24 节 `cross-repo-contract | contract-pin` 两个分支和附录 D 的完整对象；SYSTEM-BLUEPRINT 只保存 topology，不复制 provider schema。provider spec canonical bytes 是唯一合同内容，consumer pin 只固定 provider repo、commit、spec path 和 hash，禁止第二份顶层 schema、手工 cache 或运行时择优。

第七部分 | 唯一 law registry 与 canonical 生成

26. law registry 是唯一索引

唯一法案索引文件固定为：

```
scripts/audit/generated/law-registry.generated.json
```

不得另建：

```
law-index.json
law-map.json
codex-law-route.json
manual-law-list.md
README law table
第二 generated registry
```

law registry 只有导航和 source projection 权，不拥有规范主权、Owner 主权或 PASS 权。

五个 generated artifact 的身份不得混淆：

artifact	必须做	绝对禁区
`law-registry.generated.json`	唯一 law 导航、source location 与原子 binding projection	不裁决 PASS，不重新定义 law/Owner
`authority-census.generated.json`	统计并列出权威输入库存及 hash	不作为第二 law index，不表达完成度
`enforcement-bindings.generated.json`	投影 law 到 blueprint/spec/check/exam/proof/report sink	不创造 binding，不改变 required evidence
`audit-plan.generated.json`	按 Owner 组织原子 lawExecutions	不维护平行 lawIds/checkIds，不决定 scope
`inspector-registry.generated.ts`	提供固定 literal import 与 map	不动态发现 Owner，不 fallback，不承载法条正文

任何消费者需要发现 active law corpus 时只能读取 `law-registry.generated.json`；其它四件只能按上表执行各自单一职责。文件名含 `registry` 不等于取得 law 索引主权。

27. law registry 顶层与 domain record 精确 schema

顶层只允许：

```
{
  "schemaVersion": 2,
  "compilerVersion": "2",
  "sourceSpec": {
    "path": "docs/governance/法案系统建设说明书-冻结定稿版.md",
    "sha256": "..."
  },
  "lawCorpus": {
    "baseLawPath": ".github/instructions/agent-rules/基础法案.md",
    "baseLawSha256": "...",
    "functionalLawPath": ".github/instructions/agent-rules/功能法案.md",
    "functionalLawSha256": "...",
    "corpusSha256": "..."
  },
  "laws": [],
  "domains": []
}
```

`domains[]` 每项只允许：

```
{
  "capabilityDomain": "...",
  "manifestPath": "scripts/audit/domains/.../manifest.json",
  "lawIds": [],
  "blueprintRefs": [],
  "specRefs": [],
```

```
  "ownedPaths": [],
  "observedPaths": []
}
```

全部数组按合同排序；禁止时间、机器名、用户名、绝对路径、随机 UUID。

28. registry law record 与五个 projection 的完整 schema

28.1 law record

```
{
  "lawId": "LAW-...",
  "supersedesLawId": null,
  "tier": "P0",
  "capabilityDomain": "...",
  "enforcementMode": "composite",
  "evidenceModes": [],
  "summary": "...",
  "appliesTo": [],
  "excludes": [],
  "forbidden": [],
  "dependsOnLawIds": [],
  "routingTags": [],
  "source": {
    "documentType": "functional-law",
    "path": ".github/instructions/agent-rules/功能法案.md",
    "heading": "...",
    "lineStart": 1,
    "lineEnd": 10,
    "metadataSha256": "...",
    "sectionSha256": "..."
  },
  "binding": {
    "manifestPath": "...",
    "blueprintRef": "...#anchor:...",
    "specRefs": [],
    "staticCheckId": null,
    "examBindings": [],
    "proofBindings": []
  }
}
```

`laws` 按 lawId 字典序。

28.2 authority-census.generated.json

```
{
  "schemaVersion": 2,
  "counts": {
    "law": 0,
    "domain": 0,
    "manifest": 0,
    "blueprint": 0,
    "spec": 0,
    "staticCheck": 0,
    "exam": 0,
    "proof": 0
  },
  "lawIds": [],
  "domains": [],
  "sourceFiles": []
}
```

`domains[]` 元素：

```
{
  "capabilityDomain": "...",
  "lawCount": 0,
  "manifestPath": "...",
  "blueprintRefs": [],
  "specRefs": []
}
```

`sourceFiles[]` 元素：

```
{
  "path": "...",
  "role": "LAW_SOURCE",
  "sha256": "..."
}
```

`role` 只允许 `SOURCE_SPEC`、`ENTRY`、`LAW_SOURCE`、`MANIFEST`、`BLUEPRINT`、`SPEC`、`INSPECTOR_DECLARATION`、`EXAM_DECLARATION`、`PROOF_DECLARATION`。该文件只做库存，禁止 verdict/ready/complete/pass。

28.3 enforcement-bindings.generated.json

```
{
  "schemaVersion": 2,
  "bindings": [
    {
      "lawId": "LAW-...",
      "capabilityDomain": "...",
      "manifestPath": "...",
      "blueprintRef": "...#anchor:...",
      "specRefs": [],
      "staticCheckId": null,
      "examBindings": [],
      "proofBindings": [],
      "inspectorPath": "scripts/audit/domains/.../inspector.ts",
      "reportPath": "scripts/audit/reports/...report.json"
    }
  ]
}
```

每个 active lawId 精确一项。

`reportPath` 只表示 `invocationMode=TASK` 的 gitignored current-cache sink。`CI_CANDIDATE` 不读取也不写该路径；它的 report sink 由第 56.1 节从仓库外 ephemeral execution root 和 `capabilityDomain` 确定性派生，不能写入 projection 形成第二路径真相。

28.4 audit-plan.generated.json

```
{
  "schemaVersion": 2,
  "candidateSchemaVersion": 2,
  "owners": [
    {
      "capabilityDomain": "...",
      "inspectorImportId": "inspector_0001",
      "lawExecutions": [
        {
          "lawId": "LAW-...",
          "staticCheckId": null,
          "examIds": [],
          "proofIds": [],
          "requiredEvidenceModes": []
        }
      ],
      "ownedPaths": [],
      "observedPaths": []
    }
  ]
}
```

禁止分离 `lawIds` 与 `checkIds` 平行排序；绑定必须原子保存。

28.5 inspector-registry.generated.ts

只允许 literal static import 和 frozen map；import 顺序等于 audit-plan owner 顺序。禁止 dynamic import、glob import、fallback inspector 和 default owner。

29. hash 的精确算法

文件 hash、metadataSha256、sectionSha256、corpusSha256 与 sourceSpec hash 分别按第 7、12、15、30 节定义的原始字节或 canonical JSON 计算；另外固定：

```
blueprintRecordSha256 = canonical domain/system blueprint record bytes ;
specSha256 = spec 原始 canonical JSON bytes ;
manifestSha256 = manifest 原始 canonical JSON bytes ;
bindingSha256 = canonical atomic law binding bytes。
```

30. canonical JSON

对象 key 按 Unicode code point 升序递归排序；数组按本合同指定顺序；缩进 2；单 LF 结尾；禁止尾空格、注释、NaN、Infinity、undefined 和时间字段。

31. 唯一纯 compiler 的输入与输出

唯一 compiler：

```
scripts/audit/core/artifact-compiler.ts
```

它只接收调用方已经读入内存的 exact bytes/parsed records：

```
冻结说明书与 freeze manifest ;
固定 AGENTS ;
基础法案与功能法案 ;
全部 active Manifests ;
全部被绑定 system/domain blueprints ;
全部被绑定 specs ;
全部 inspector literal check declarations ;
全部 exam literal declarations ;
全部 proof literal declarations。
```

compiler 不接收 Git、候选生产文件清单、历史 reports、网络、数据库、浏览器、provider、墙钟或随机源，因此不负责 shadow scan、law history、candidate ownership 或 runtime PASS。

唯一输出精确为五个 generated artifacts；纯函数不得写盘、创建目录、自动修复输入或输出 verdict。

32. source-integrity preflight 与 compiler 固定顺序

32.1 candidate-aware source-integrity preflight

`entry-gate.ts`、`audit:build`、`audit:projection:check` 和 `quality:check` 必须复用同一组只读函数，顺序为：

```
freeze/AGENTS/toolchain check
-> source-spec template check
-> shadow-law scan
-> law-history check
-> candidate path encoding/case check
```

preflight 只允许返回：

```
FREEZE_HASH_MISMATCH
ENTRY_HASH_MISMATCH
CANONICAL_PACKAGE_MANAGER_AMBIGUOUS
SOURCE_SPEC_TEMPLATE_DRIFT
SHADOW_LAW_BLOCK_FOUND
UNBOUND_NORMATIVE_TEXT
LAW_HISTORY_ID_REWRITTEN
LAW_HISTORY_LINEAGE_BROKEN
LAW_HISTORY_KEY_REUSED
PATH_ENCODING_INVALID
PATH_CASE_COLLISION
```

这些函数不得写 projection、Owner report 或业务文件。

32.2 pure compiler conflict codes

compiler 只允许返回：

```
LAW_SOURCE_COUNT_INVALID
LAW_FRONTMATTER_INVALID
LAW_BLOCK_MALFORMED
LAW_UNKNOWN_FIELD
LAW_ID_DUPLICATE
LAW_DOMAIN_MISMATCH
LAW_MIRROR_MISMATCH
LAW_SUPERSEDES_INVALID
LAW_DEPENDENCY_MISSING
LAW_DEPENDENCY_CYCLE
MANIFEST_MISSING
MANIFEST_DUPLICATE
MANIFEST_SCHEMA_INVALID
LAW_UNBOUND
LAW_DOUBLE_BOUND
OWNED_PATH_OVERLAP
BLUEPRINT_MISSING
BLUEPRINT_SCHEMA_INVALID
BLUEPRINT_IDENTITY_MISMATCH
BLUEPRINT_ANCHOR_MISSING
SPEC_MISSING
SPEC_SCHEMA_INVALID
SPEC_ID_DUPLICATE
SPEC_OWNER_MISMATCH
CONTRACT_OWNER_CONFLICT
STATIC_CHECK_ID_MISSING
STATIC_CHECK_ID_DUPLICATE
EXAM_ID_MISSING
EXAM_ID_DUPLICATE
PROOF_ID_MISSING
PROOF_ID_DUPLICATE
EVIDENCE_BINDING_MISMATCH
INSPECTOR_IDENTITY_MISMATCH
INSPECTOR_CONTRACT_INVALID
PLACEHOLDER_REMAINS
```

`GENERATED_FILESET_INVALID` 不属于 pure compiler conflict。pure compiler 只返回内存中的固定五件 `artifactName -> canonical bytes` map；调用 wrapper 在接收返回值后先验证五个固定 key，集合不精确时由 wrapper 返回 `GENERATED_FILESET_INVALID`，不得把该 code 塞进 compiler conflicts 或 Owner verdict。

compiler 顺序固定：

```
两份 active law parse/mirror
-> lawId/domain/supersedes/dependency DAG
-> Manifests 与 atomic law bindings
-> ownedPath claim overlap
-> system/domain blueprint schema 与 anchors
-> specs/contracts/surfaces
-> inspector/exam/proof literal declarations
-> evidence coverage
-> authority census
-> enforcement bindings
-> audit plan
-> inspector registry
-> law registry
-> canonical five artifact bytes。
```

任一冲突返回完整排序 conflicts 且不返回可写 artifact bytes；不得新增 warning 旁路。

33. audit:build 的唯一写入协议

`audit:build`：

```
运行第 32.1 节 source-integrity preflight；
在内存调用唯一 compiler；
若有冲突，stderr 输出按 code/path/lawId 排序的冲突并非零退出；
若无冲突，把五个 artifact 写入同文件系统临时目录；
逐文件 fsync；
验证临时目录文件集合精确为五个；
验证临时字节可重新解析；
以受控替换方式更新 generated 目录；
替换失败必须恢复旧五文件；
```

不得留下部分新、部分旧的 generated 集合。

临时目录缺失、多出或错名任一文件时，`audit:build` 唯一返回 `GENERATED_FILESET_INVALID`；该 code 由 build wrapper 拥有，不得由 pure compiler、Owner inspector 或 final aggregator 返回。

`audit:build` 不得写 Owner report。

34. projection check

`audit:projection:check` 必须：

运行第 32.1 节 source-integrity preflight；
读取现有五个 generated 文件；
以内存调用同一 compiler；
比较文件集合与 exact bytes；
比较前后工作树 hash，证明检查无写入；
缺失、多余、手改、陈旧、无法重算均非零退出。

当前 generated 目录文件集合不是精确五件时，projection check 同样返回 `GENERATED_FILESET_INVALID`；集合正确但任一 exact byte 不一致时返回 `GENERATED_PROJECTION_STALE`。两个 code 不得互换。

projection checker 不得自动调用 `audit:build`，不得修复 generated，不能输出第二 verdict artifact。

第八部分 | Codex 任务防漂移系统

35. Codex 落笔前的固定事务

任何任务，包括 read-only 与 release-proof，都必须先建立可复放任务身份；只有 `.governance/runtime/**` 写入可发生在仓库治理入口之前。

固定顺序：

1. request-capture：写入人类请求原字节；
2. governance:entry：验证 freeze、entry、现役治理与 lock 冲突；
3. governance:task:start：建立事务锁、baseline 与初始 envelope；
4. Codex 只读分析；
5. task-proposal：声明完整初始路径、closures、surfaces；
6. governance:scope:propose；
7. governance:route；
8. 验证并全文读取 task-context；
9. ROUTED 后才允许首个仓库写入。

35.1 request-capture

唯一脚本 `request-capture.ts` 从标准输入接收外层编排器已经合成的完整人类授权文本，原字节写入 `user-request.txt`。禁止摘要、翻译、润色、加入系统消息、开发者消息、工具输出或 Codex 计划。

35.2 task lock 与崩溃恢复

`task-lock.json`：

```
{
  "schemaVersion": 2,
  "taskId": "<16 lowercase hex>",
  "initialCandidateId": "<64 lowercase hex>",
  "userRequestSha256": "<64 lowercase hex>",
  "state": "INITIALIZING",
  "closedCandidateId": null,
  "finalEvidenceSetSha256": null
}
```

状态只允许 `INITIALIZING`、`OPEN`、`CLOSED`、`ABANDONED`。

`task-start.ts` 使用同目录 transaction 临时文件、fsync 和原子 rename：先 O_EXCL 建 `INITIALIZING` lock，再生成 baseline/envelope，全部回读一致后改为 OPEN。崩溃后同 userRequestSha256 只能在临时文件 hash 可重算时恢复；否则 `INCOMPLETE_TASK_INITIALIZATION`。不得删除 lock 规避。

CLOSED 必须写入最终 candidateId 与全部 component/final receipt hash 的集合 hash；candidate 改变后该 CLOSED 只代表历史任务，不代表当前候选。

35.3 Task Envelope v2

顶层只允许：

```
{
  "schemaVersion": 2,
  "taskId": "...",
  "taskType": "implementation",
  "bootstrapMode": "none",
  "initialCandidateId": "...",
  "userRequestSha256": "...",
  "explicitTargets": [],
  "proposedWritePaths": [],
  "proposedDeletePaths": [],
  "pathClaims": [],
  "proposedSurfaces": [],
```

```
  "initialClosures": [],
  "scopeExpansions": [],
  "governanceLawAmendmentAuthorized": false,
  "sourceSpecAmendmentAuthorized": false,
  "crossRepoChangeSet": null
}
```

35.4 taskType 与 bootstrapMode 的唯一分类

`taskType` 只允许 `source-spec-amendment`、`law-amendment`、`release-proof`、`read-only`、`implementation`。按以下顺序首次命中即停止：

```
1. 人类原文逐字包含“授权修订《法案系统建设说明书》”并点名范围 -> source-spec-amendment；
2. 人类明确要求新增、修改、升级或退役 active lawId/基础法案/功能法案 -> law-amendment；
3. 人类明确要求生产发布证明，且除 runtime evidence 外不允许仓库写入 -> release-proof；
4. 人类明确使用“只读”并禁止仓库写入 -> read-only；
5. 其它 -> implementation。
```

Codex 不得自行提升到 amendment，也不得把 amendment 降为 implementation。

`bootstrapMode` 只允许 `none`、`blank`、`existing`：

```
blank：仓库没有本说明书治理七件套，且人类明确授权从零建制；
existing：仓库已有生产代码或旧治理，且人类明确授权一次性迁移；
none：其余任务。
```

blank/existing 只建设或迁移治理，不得同时施工新业务能力。`read-only`、`release-proof` 的 repo write/delete paths 必须为空。

35.5 explicitTargets

kind 只允许 `path`、`lawId`、`capabilityDomain`、`artifact`、`command`、`surface`、`repository`。value 与 sourceQuote 必须逐字来自 user-request。

35.6 pathClaims 与完整初始闭包

每个 pathClaim：

```
{
  "path": "src/exact/file.ts",
  "action": "WRITE",
  "phase": "INITIAL",
  "admission": "PRIMARY_IMPLEMENTATION",
  "primaryDomain": "example-domain",
  "lawId": "LAW-EXAMPLE-DOMAIN-001-V1",
  "closureRef": null,
  "evidenceRef": "authority:..."
}
```

`phase` 只允许 INITIAL、EXPANSION。初始 proposal 可以并且必须声明已知的 GENERATED_PROJECTION、OWNER_CLOSURE、CONTRACT_CLOSURE、PROOF_CLOSURE、OLD_PATH_RETIREMENT；这些记录进入 `initialClosures`，不需要伪装成 post-route expansion。

`initialClosures[]` 与 `scopeExpansions[]` 使用同一证据字段：reasonCode、path、lawId、triggerPath、triggerRef、counterfactualFailure、explanation；区别只有前者在首次 route 前，后者在 ROUTED 后发现。

35.7 proposedSurfaces

每项：

```
{
  "action": "ADD",
  "kind": "http-route",
  "identity": "POST /v1/example",
  "primaryDomain": "example-domain",
  "lawId": "LAW-EXAMPLE-DOMAIN-001-V1",
```

```
  "blueprintRef": "docs/blueprints/domains/example-domain/EXAMPLE-DOMAIN-BLUEPRINT.md#anchor:surface.
 http.example",
  "specRef": "docs/specs/example-domain/example-api.spec.json",
  "admission": "PRIMARY_IMPLEMENTATION"
}
```

`action` 只允许 ADD、MODIFY、DELETE；身份改名必须旧 DELETE + 新 ADD。

kind 只允许：

```
http-route
cli-command
package-script
public-export
environment-variable
secret-name
feature-flag
event-type
queue-or-topic
database-table
database-column
storage-key-family
config-key
worker-entry
scheduled-job
package-dependency
runtime-listener
outbound-host
browser-permission
native-host
filesystem-write-root
cross-repo-contract
```

dependency、listener、外连主机、浏览器权限和 Native Host 都是 surface，不能藏在普通路径 diff 中。

35.8 Envelope 写权

只有 task-start 创建，scope-propose 原子更新。Codex 只能写 runtime proposal，不得手改 envelope。任何字节无法由 baseline、request、approved proposal 和法定算法重算，返回 `TASK_ENVELOPE_TAMPERED`。

36. task baseline 与状态感知文件记录

baseline 顶层保留 taskId、initialCandidateId、gitHead、workspaceStateSha256、freezeSha256、lawCorpusSha256、files、preExistingDirtyPaths、generatedArtifacts。

`files[]` 每项：

```
{
  "path": "repo/relative/path",
  "presence": "PRESENT",
  "contentKind": "REGULAR",
  "sha256": "<hash or null>",
  "size": 0,
  "mode": "100644",
  "indexOid": "<git oid or null>",
  "indexState": "CLEAN",
  "worktreeState": "CLEAN"
}
```

presence：PRESENT、DELETED。contentKind：REGULAR、SYMLINK。index/worktree state：CLEAN、MODIFIED、ADDED、DELETED、RENAMED、UNTRACKED。

算法使用 `git status --porcelain=v2 -z`、`git ls-files --stage -z` 与 untracked enumeration；tracked deletion 记录 indexOid/mode，不读取不存在的工作树文件。rename 在候选 hash 中表现为 old DELETE + new PRESENT。

preExistingDirtyPaths 与目标相交仍 blocked；Codex 不得 reset、clean、stash、restore、覆盖或移动用户基线。

37. task-proposal 与初始完整施工分母

首次 proposal 顶层：taskId、proposedWritePaths、proposedDeletePaths、pathClaims、proposedSurfaces、initialClosures、scopeExpansions。其中 scopeExpansions 必须为空，但 initialClosures 可以非空。

固定流程：

只读扫描唯一 Owner、合同、调用边、proof 和旧路径；
列出最小且完整的初始闭包；
把已知 generated、contract、proof、retirement 一次性纳入 initialClosures；
scope:propose 验证并原子写 envelope；
route 成功后才写。

禁止以“可能需要”为由预授权全仓；禁止把应在初始闭包声明的路径故意延后成 expansion。

38. 合法 post-route scope expansion

只有任务已 ROUTED 且只读阶段无法预先确定的真实闭包，才允许增加 EXPANSION phase path；原因码精确为五个：

38.1 GENERATED_PROJECTION

已获准 source/Manifest/blueprint/spec/declaration 变化；
唯一 compiler 在内存证明固定五文件 expected bytes 改变；
path 只属于固定五个 projection；
report/cache/summary 不得伪装 projection。

38.2 OWNER_CLOSURE

新增 path 与已获准 triggerPath 属于同一 primary domain；
存在可复放 import/export/call/data-ownership edge；
不修改会产生精确 compile/runtime-contract/governance failure code；
不得用于整理、抽象、去重、重命名或未来复用。

38.3 CONTRACT_CLOSURE

已获准实现改变唯一 typed contract/schema/migration 的输入、输出、状态、错误、持久化或版本；
contract provider path 是唯一主权源；
全部消费者由 dependency graph 精确枚举；
不得建立 v2 copy、compat schema、optional fallback、dual-read 或 dual-write。

38.4 PROOF_CLOSURE

active law atomic binding 明确要求对应 Static/Exam/Dynamic component；
现有 declaration/实现缺失、对本候选失效或确定性失败；
新增 path 位于固定 proof 目录并绑定唯一 checkId/examId/proofId；
不得建立第二 verdict、runner、fixture truth 或报告写入者。

38.5 OLD_PATH_RETIREMENT

旧 path 与新 path 承担同一 active role；
新 path 已在 envelope；
旧入口、注册、导出、配置、测试、文档与 surface 引用可穷举；
本次把全部引用降为零；
不得保留 alias、forwarder、feature flag、deprecated facade 或 rollback branch。

每项 expansion 精确记录：reasonCode、path、lawId、triggerPath、triggerRef、counterfactualFailure、explanation、approvedEnvelopeSha256。triggerPath 必须已获准，evidence 必须来自当前 candidate，counterfactualFailure 必须是可复现 code。

新增 dependency/network/browser/native-host 等 surface 时，path expansion 与 proposedSurface 必须同时批准。每项 expansion 在首次触碰前执行 scope:propose 和重新 route；旧记录不得删除、改 lawId、改 admission 或改 phase。cleanup/refactor/style/future-proofing/compatibility 均不是合法理由。

39. 唯一路由、amendment transaction 与 existing bootstrap

39.1 普通 route

route 只接收当前 exact-byte registry、envelope、baseline、五个 projection。它必须：

验证 lock/request/baseline/envelope；
基础法案全文进入 context；
现有 path 通过 ownedPaths 找唯一 primary domain；
新 path 通过最近已存在父目录和 blueprint/spec 声明找唯一 domain；
把 proposedSurfaces 的 ownerDomain 加入 primary domains；
无 path 时按 explicit lawId/domain/artifact/command/surface exact route，禁止仅依赖模糊 tag；
加入 primary domain 全部 active lawId；
加入 observer domains；
递归展开正向依赖，删除/law amendment 时加入反向闭包；
固定加入 project-governance、law-system、audit-system 全部 active lawId；
从 Manifest 原子 binding 获取 blueprintRefs、specRefs、static/exam/proof declarations；
生成 context 与 receipt，回读 hash；
ROUTED 前不写仓库。

39.2 law/source-spec amendment staging transaction

唯一脚本 `amendment-transaction.ts`，状态：PREPARED -> STAGING -> STAGED_VALIDATED -> APPLIED -> POST_VERIFIED | ABORTED。

1. 在旧 authority 下完成 request、baseline、pre-route 和完整 amendment plan；
2. 创建 OS 临时目录中的隔离工作树，绑定 initialCandidateId；
3. Codex 只在 staging 修改说明书/法案及其完整闭包；真实工作树保持不变；
4. staging 内生成新 freeze（source-spec 时）、新 manifests/blueprints/specs/declarations 和五个 projection；
5. 使用 staged 新 authority 生成 post-route，并执行 parser、compiler、history、mutants、quality；
6. 计算完整 patch、删除清单和 stagedCandidateId；
7. 再次验证真实工作树仍等于 initialCandidateId；
8. 以受控原子 patch 应用全部文件；任一步失败恢复旧字节；
9. 在真实工作树重算 projection、post-route、scope 和 quality；
10. 成功后关闭 task；失败保留 OPEN 且不得留下 source/projection 混合状态。

source-spec amendment 中 PDF 只有 `enabled=true` 才在 staging 生成；默认 false 不要求 PDF。

39.3 bootstrapMode=existing 一次性受限入口

既有仓在新 entry 尚不存在时，允许唯一 bootstrap 例外：只可创建本说明书固定治理路径、runtime baseline/migration contract，并删除被 migration contract 穷举的旧治理路径；不得改业务功能语义。

`bootstrap-existing.ts` 必须：

验证人类原文含明确 existing 治理迁移授权；
捕获全仓状态感知 baseline；
列出全部旧法案源、instructions、registry、Owner map、compiler、runner、generated、report writer、CI 入口；
为每项给出 MIGRATE/DELETE/HISTORY_ONLY/KEEP_NON_NORMATIVE 决策和 evidenceRef；
列出全部生产根与拟 domain Owner；
禁止新增业务 domain、surface 或功能；
在隔离 staging 中一次性建立新系统并把旧引用降为零；
通过 migration mutants 后原子应用；
例外完成即永久失效，后续只走正常 entry。

40. route receipt v2 完整 schema

成功 receipt 顶层只允许：

```
{
```

```
  "schemaVersion": 2,
  "routeAlgorithmVersion": "2",
  "authorityEpoch": "CURRENT",
  "taskId": "...",
  "initialCandidateId": "...",
  "userRequestSha256": "...",
  "envelopeSha256": "...",
  "registrySha256": "...",
  "primaryDomains": [],
  "observerDomains": [],
  "lawIds": [],
  "dependencyEdges": [],
  "sourceSections": [],
  "blueprintRefs": [],
  "specRefs": [],
  "manifestRefs": [],
  "inspectorRefs": [],
  "examRefs": [],
  "proofRefs": [],
  "surfaceRefs": [],
  "crossRepoContractRefs": [],
  "taskContextSha256": "...",
  "status": "ROUTED",
  "blockCode": null
}
```

`authorityEpoch` 只允许 CURRENT、STAGED_AMENDMENT。sourceSections 绑定 lawId/path/lineStart/lineEnd/sectionSha256；所有 document refs 绑定 path/sha256；dependency edge 绑定 fromLawId/toLawId/direction；surface ref 绑定 action/kind/identity/specRef。数组按合同 key 排序、无重复。

失败 receipt 具有同样 identity/hash 字段，closure 数组全部为空，`taskContextSha256=null`、`status=BLOCKED`、`blockCode` 使用第 41 节枚举；不得生成部分 context。

41. route/block 的完整固定 code

只允许：

```
ACTIVE_TASK_EXISTS
USER_REQUEST_NOT_CAPTURED
TASK_LOCK_MISMATCH
INCOMPLETE_TASK_INITIALIZATION
TASK_ENVELOPE_TAMPERED
BASELINE_TAMPERED
FREEZE_HASH_MISMATCH
ENTRY_HASH_MISMATCH
CANONICAL_PACKAGE_MANAGER_AMBIGUOUS
SOURCE_SPEC_TEMPLATE_DRIFT
LAW_REGISTRY_STALE
GENERATED_PROJECTION_STALE
SHADOW_LAW_BLOCK_FOUND
UNBOUND_NORMATIVE_TEXT
PATH_ENCODING_INVALID
PATH_CASE_COLLISION
NO_PRIMARY_OWNER
MULTIPLE_PRIMARY_OWNERS
NEW_TOP_LEVEL_PATH_NOT_AUTHORIZED
NO_EXACT_TEXT_ROUTE
LAW_AMENDMENT_NOT_AUTHORIZED
SOURCE_SPEC_AMENDMENT_NOT_AUTHORIZED
LAW_SOURCE_SECTION_MISSING
LAW_SECTION_HASH_MISMATCH
LAW_DEPENDENCY_MISSING
LAW_DEPENDENCY_CYCLE
LAW_HISTORY_ID_REWRITTEN
LAW_HISTORY_LINEAGE_BROKEN
LAW_HISTORY_KEY_REUSED
BLUEPRINT_MISSING
BLUEPRINT_SCHEMA_INVALID
BLUEPRINT_ANCHOR_MISSING
SPEC_MISSING
SPEC_SCHEMA_INVALID
MANIFEST_MISSING
INSPECTOR_MISSING
EXAM_ID_MISSING
PROOF_ID_MISSING
EVIDENCE_BINDING_MISMATCH
BLOCKED_PREEXISTING_DIRTY_TARGET
USER_REQUEST_CONFLICTS_WITH_ACTIVE_LAW
```

```
INITIAL_CLOSURE_INVALID
SCOPE_EXPANSION_INVALID
TASK_CONTEXT_HASH_MISMATCH
AMENDMENT_STAGING_REQUIRED
AMENDMENT_STAGED_VALIDATION_FAILED
EXISTING_BOOTSTRAP_NOT_AUTHORIZED
MIGRATION_CONTRACT_INCOMPLETE
CROSS_REPO_CONTRACT_UNPINNED
DOMAIN_CREATION_INCOMPLETE
LAW_RETIREMENT_INCOMPLETE
```

出现任一 code：不得写仓库、不得创建临时 Owner/法案/fallback、不得选择最接近 domain；只输出 taskId、code、path/lawId/evidenceRef，非零退出并保留可安全恢复的 task state。

顶级目录错误码按执行时点唯一分工：首次 route 对尚未写入、但 proposal 已声明的新顶级目录缺少人类架构/domain 授权时，只能返回 `NEW_TOP_LEVEL_PATH_NOT_AUTHORIZED`；最终 scope check 对实际 candidate 中已经出现、却不在获准 envelope 的新顶级目录，只能返回 `NEW_TOP_LEVEL_PATH_UNAUTHORIZED`。二者不得由同一 command 互换返回。

42. Codex 的完整禁止深化清单

除人类原文明确授权且 active law/blueprint/spec 已允许外，Codex 不得：

```
新增功能、产品范围、API、CLI、事件、队列、worker、表、列、环境变量、secret、feature flag；
新增顶级目录、架构层、capabilityDomain、Owner、registry、compiler、runner、quality aggregator；
把明确流程抽象成万能 planner、shared/common/utils 主权或第二 dispatcher；
加入 fallback、compat、legacy、shadow、bridge、dual-read、dual-write；
为未来场景预留接口、升级无关依赖、全仓格式化、重命名无关符号；
顺手修复相邻问题、改变 tier/evidenceModes/Owner/appliesTo、降低 predicate；
使用 fixture/mock/no-op/固定返回冒充生产能力；
保留旧路径以便回滚；回滚只使用 Git；
把治理迁移顺带升级 Node/package manager；
把 dependency、listener、outbound host、browser permission 或 Native Host 藏在普通 path diff；
在消费仓复制跨仓 contract 主权；
在真实工作树暴露 amendment 中间 source/projection 不一致；
把 initial closure 故意延后伪装 expansion；
把 tracked deletion 当读取错误而取消 retirement；
删除/改写 task lock、baseline、envelope、route receipt 或 task-context 逃避检查。
```

43. 完整任务而非阶段任务

获准能力必须在同一 task 闭合唯一实现、合同、surface、持久化、错误恢复、旧路径、projection、当前 CHANGE context 必需的 Static/Exam/Dynamic 和 final aggregate。外部环境使当前必需 Dynamic 无法完成时，结果固定 `NOT_PRODUCTION_PROVEN`，不得声称任务完成。

任务通过 CHANGE context 只证明本次变更闭合；只有第 63 节 RELEASE context 可产生 `SYSTEM_PROVEN`。这不是阶段交付，也不允许把缺失实现留到以后。

44. 最终 scope check 完整算法

输入固定为 baseline、Task Envelope、route receipt、task-context、当前 Git index/worktree、五个 projection、surface extractor registry 和 cross-repo pin。顺序：

```
1. 重算 request/envelope/route/context hash，验证同一 OPEN task；
2. 以 baseline 状态感知 records 计算 PRESENT/DELETED/rename/mode/symlink/staged/unstaged/untracked 实际 delta；
3. 实际 WRITE 必须属于 proposedWritePaths，实际 DELETE 必须属于 proposedDeletePaths；
4. 每个 pathClaim 与 write/delete 集合、action、phase、admission 精确一致；
5. INITIAL closure 在首次 route 前批准；EXPANSION 在首次触碰前批准并重新 route；
6. generated 变化只在 GENERATED_PROJECTION claim 下且 exact bytes 来自唯一 compiler；
7. 说明书/freeze/AGENTS 只在 source-spec amendment staging，active law 只在 law/source-spec amendment staging；
8. preExistingDirty path 的 path/hash/mode/index/worktree state 与 baseline 完全一致；
9. 新顶级目录、新 domain、新 dependency 和所有 surface 均有明确授权、唯一 Owner、lawId、blueprint anchor、spec 与 extractor；
10. 每个新增 PRESENT production file 精确一个 Owner；每个 DELETE 有合法 retirement；
11. 从 baseline 与当前 candidate 提取实际 surfaces，按 action/kind/identity 与 proposedSurfaces 精确相等；identity 改名必须旧
 DELETE + 新 ADD；
12. extractor 无法解析潜在 surface-bearing 文件即失败，不得当作无 delta；
```

13. route 依赖的 law/Manifest/blueprint/spec/declaration 变化使普通 route stale；amendment 只能使用 STAGED_AMENDMENT post-route；
14. cross-repo contractId/version/provider commit/hash/changeSet participant 与 pin 精确一致；
15. proposed path/surface 未实际完成时，不得以任务完成关闭 lock。

scope failure code 只允许：

```
TASK_ID_MISMATCH
RUNTIME_ARTIFACT_TAMPERED
SCOPE_DIFF_OUTSIDE_ENVELOPE
INITIAL_CLOSURE_INVALID
SCOPE_EXPANSION_AFTER_WRITE
GENERATED_BYTES_NOT_FROM_COMPILER
FROZEN_SPEC_MODIFIED_WITHOUT_AUTHORIZATION
LAW_SOURCE_MODIFIED_WITHOUT_AUTHORIZATION
PREEXISTING_DIRTY_PATH_CHANGED
NEW_TOP_LEVEL_PATH_UNAUTHORIZED
NEW_PRODUCTION_FILE_UNOWNED
UNAUTHORIZED_PRODUCTION_DELETE
PATH_CLAIM_MISMATCH
UNAUTHORIZED_SURFACE_DELTA
MISSING_DECLARED_SURFACE_DELTA
SURFACE_BINDING_MISMATCH
SURFACE_DELTA_UNDETERMINED
ROUTE_STALE_AFTER_GOVERNANCE_CHANGE
CROSS_REPO_CONTRACT_UNPINNED
```

任一失败非零退出；“只是格式化、import 排序、自动生成或顺手修复”不得放行。

`NEW_TOP_LEVEL_PATH_UNAUTHORIZED` 只属于本节最终 scope check；route 尚未允许写入时不得使用该 code，必须使用第 41 节的 `NEW_TOP_LEVEL_PATH_NOT_AUTHORIZED`。

45. AGENTS.md 只做 frozen 守门

动作顺序固定：request capture -> entry -> task start -> base law/registry -> proposal -> route -> context -> write -> scope/projection/quality。AGENTS 不复制业务 law，不维护手工路由表，其完整字节受 freeze hash 保护。

46. governance:entry 固定检查

entry 运行时 `user-request.txt` 必须已经存在。它检查：

spec/AGENTS hash；工具链；两份法案；shadow law；五个 projection；registry corpus；manifests/blueprints/specs/declarations；唯一 compiler/build/route/scope/aggregate；runtime 可写且 ignored；OPEN lock 与当前 request hash；path encoding/case collisions。

entry 只输出文本与退出码，不写 PASS artifact。

47. 普通任务对本说明书的读取规则

所有任务先读取固定 AGENTS、基础法案和完整 law registry；
route 后必须全文读取 task-context；
涉及 law-system、audit-system、project-governance、Owner、compiler、generated、quality、migration、cross-repo contract 时全文读取本说明书；
普通业务任务不需要把整份说明书重复塞入上下文，但 freeze hash 必须通过；
read-only 与 release-proof 同样建立 request、baseline、route 和 lock，只是 proposed repo writes 必须为空；
本说明书或 AGENTS hash 不匹配时全部任务停止。

第十部分 | Static Audit 的唯一主链

48. Static Audit 的唯一职责

Static Audit 只以当前候选的静态结构证明 active lawId，包括：

```
文件系统；
文本；
AST；
类型；
schema；
合同；
依赖；
配置；
JSON/YAML/SQL；
数据库迁移声明；
目录和 Owner 绑定。
```

Static Audit 不得：

```
启动业务服务；
访问浏览器、网络、数据库、对象存储或 provider；
读取历史运行结果作为当前证据；
把 tests、fixtures、golden、mutation、acceptance 当作生产法条证据；
输出 release verdict；
消费 Exam 或 Dynamic Proof 的 PASS 结果；
修改业务源码；
自动修复违法。
```

49. CandidateIdentity v2 精确算法

候选枚举固定使用 Git 状态感知三件套，而不是“列出路径后逐个读取”：

```
git status --porcelain=v2 -z
git ls-files --stage -z
受 .gitignore 约束的 untracked enumeration
```

每个候选 record 规范为：

```
{
  "path": "repo/relative/path",
  "presence": "PRESENT",
  "contentKind": "REGULAR",
  "mode": "100644",
  "sha256": "<hash or null>",
  "indexOid": "<oid or null>",
  "indexState": "CLEAN",
  "worktreeState": "CLEAN"
}
```

规则：

1. PRESENT regular hash 原始字节；PRESENT symlink hash 链接目标文本。
2. DELETED 不读取文件，hash 字段 null，保留 indexOid/mode/state。
3. rename 规范化为 old DELETED + new PRESENT；rename hint 不进入 candidateId。
4. file mode、index/worktree state、untracked 都进入 record。
5. 忽略 `.git/**`、`.governance/runtime/**`、reports、artifacts、coverage、依赖与构建缓存；五个 generated 包含。
6. 路径必须有效 UTF-8 NFC；检测大小写折叠冲突。
7. submodule、unmerged conflict、越界 symlink、特殊设备文件直接 blocked。
8. records 按 path + presence 排序，canonical JSON 后参与 hash。

candidateId 精确计算：

```
sha256(
  "candidate-schema-v2\n"
  + compilerVersion + "\n"
  + gitHeadOrUNBORN + "\n"
  + canonicalJson(candidateRecords) + "\n"
)
```

绝对路径、时间、机器名不进入。tracked deletion 是合法候选状态，不得因读取失败 blocked。

50. candidate mutation sentinel

`candidate-mutation-sentinel.ts` 被 `audit:run` 与 `quality:check` 复用，但两者输出权不同。

`audit:run` 只在以下节点重新计算 candidateId：

```
建立 evidence index 前；
index sealed 后；
每个 Owner 执行前；
每个 Owner 执行后；
发布本次 Static component receipt set 前。
```

`quality:check` 在消费 Static receipts 后继续于以下节点重算：

```
产生 EvidenceRequirementDecisions 前；
全部 required Exam/Dynamic components 后；
全部 Owner report 写入本次 invocation 临时 sink 后；
`TASK` 正式替换 report cache 前，或 `CI_CANDIDATE` 写 invocation receipt 前。
```

任一变化：

```
删除本次 execution evidence root 内的 receipts 与临时 reports；
`TASK` 不更新正式 report cache；`CI_CANDIDATE` 不写 invocation receipt；
返回 CANDIDATE_DRIFT；
非零退出。
```

`audit:run` 永远不生成或发布 Owner report；它只能把 Static component receipts 写入本次 execution evidence root。只有 `quality:check` 的 final aggregator 可以进入 report 节点。

51. 唯一 CandidateStaticEvidenceIndex

每次 `audit:run` 只能建立一个运行期 index，固定包含：

```
CandidateIdentity；
file catalog；
canonical glob -> immutable sorted file ids；
text index；
JSON/YAML parsed tree；
TypeScript/JavaScript 单一 AST project；
SQL statement index；
dependency graph；
source hash；
parse blockers；
compiled law query catalog；
sealed=true。
```

规则：

```
同一文件原始字节只读一次；
同一 representation 最多解析一次；
同一 glob 只编译一次；
同一 law query 只编译一次；
index sealed 后禁止增量补读；
Owner 只能获得只读 view；
Owner 不得直接访问 fs、parser、network、process.env、Date、Math.random；
运行结束销毁 index；
不得跨 run cache。
```

52. 生产证据分母

static evidence 的文件分母固定由：

```
全部 active law appliesTo/excludes；
全部 manifest ownedPaths/observedPaths；
对应 blueprint/spec refs；
固定治理源码；
```

求并集得到。

任何 discovery、read、decode、parse 失败均是 blocker，不得解释为“文件不存在所以禁止项未出现”。

以下路径永远不得作为业务 law PASS 证据：

```
tests/**
fixtures/**
examples/**
artifacts/**
coverage/**
docs/history/**
scripts/audit/reports/**
```

治理 meta inspector 可以读取 tests/governance 源码验证质量系统本身是否有 mutant coverage，但测试内容仍不能证明业务实现成立。

53. Inspector 精确合同

每个 `inspector.ts` 只能：

```
从 inspector-contract 导入 defineInspector；
从 predicates.ts 导入本 domain predicates；
以 literal capabilityDomain 注册；
以 literal checkId map 注册；
接收只读 evidence view；
返回 assertion results 和 findings。
```

固定形状：

```
export default defineInspector({
  capabilityDomain: "<domain>",
  checks: {
    "<domain>.<check-id>": predicate
  }
});
```

禁止：

```
动态计算 capabilityDomain；
动态 import inspector；
读取 manifest 决定自己是谁；
直接写 report；
固定返回 PASS；
空 assertions；
catch 后返回通过；
直接读磁盘或启动服务；
调用其它 domain 私有 predicate；
依据文件数量、token、anchor 或 owner 存在直接 PASS。
```

54. Law Predicate 精确结果

每个 checkId 对应一个 lawId，并返回：

```
{
  "lawId": "LAW-...",
  "checkId": "domain.check-id",
  "assertions": [
    {
      "assertionId": "domain.assertion-id",
```

```
    "passed": true,
    "evidenceRefs": ["ast:path#..."],
    "message": "精确说明检查的关系"
   }
  ],
  "findings": []
}
```

规则：

```
assertions 至少 1 项；
每个 assertionId 全项目唯一；
passed=true 必须至少一个 exact evidenceRef，纯 absence law 必须提供 scope denominator ref；
任一 passed=false 必须产生 finding；
任一 finding 使该 Static component receipt 为 COMPONENT_FAIL；
predicate 抛错等同 EXECUTED_FAIL；
不得返回 coverage percentage；
不得把多个 lawId 合成一个 predicate receipt。
```

55. SemanticEvidenceRef 固定格式

只允许：

```
file:<path>#sha256:<hash>
text:<path>#L<start>-L<end>:<hash>
ast:<path>#<node-kind>:<start>-<end>:<hash>
json:<path>#<json-pointer>:<hash>
yaml:<path>#<json-pointer>:<hash>
sql:<path>#statement:<ordinal>:<hash>
dep:<from>-><to>:<kind>
scope:<lawId>:<target-set-hash>:<count>
authority:<path>#L<start>-L<end>:<section-hash>
```

禁止使用自然语言“见代码”“文件存在”“大概在某处”作为 evidenceRef。

56. 证据 requirement decision、component receipt 与 LawQualityReceipt

56.1 verificationContext

只允许：

```
CHANGE：普通写任务、law/source-spec amendment、existing migration；证明本次变更闭合。
RELEASE：release-proof 或 blank bootstrap 终检；证明当前完整 candidate 可取得 SYSTEM_PROVEN。
```

context 由 taskType/bootstrapMode 确定，Codex 不得手工选择较弱 context：`release-proof` 与 `bootstrapMode=blank` 固定 RELEASE；其它写任务固定 CHANGE。`CI_CANDIDATE` invocation 固定使用 CHANGE，永远不得产生 RELEASE 或 `SYSTEM_PROVEN`。

执行身份字段固定为：

```
invocationMode：TASK | CI_CANDIDATE；
TASK：executionId == taskId，taskId 非空，baseSha/headSha 为 null；
CI_CANDIDATE：taskId 为 null，executionId 按第 64 节计算，baseSha/headSha 为精确 40 位 Git SHA；
同一执行链的 decision、component、LawQualityReceipt、Owner report 必须具有完全相同的 invocationMode/executionId/taskId/
 baseSha/headSha/candidateId/context；
任一不一致按 RECEIPT_EXECUTION_IDENTITY_MISMATCH 失败。
```

执行输出根固定为：

```
TASK：.governance/runtime/evidence/<executionId>/**；Owner report 先写该 root 的 temp/reports，再由唯一 report-
 writer 原子替换 scripts/audit/reports/** current cache。
CI_CANDIDATE：mkdtemp 创建的仓库根 realpath 之外 OS/CI ephemeral execution
 root；decision、component、LawQualityReceipt、Owner report、mutant receipt 与 ci-invocation-receipt 全部只能写该
 root。
```

`report-writer.ts` 是两种 invocation 共用的唯一 canonical serializer/writer。它必须接收显式 `invocationMode + executionRoot + sinkPath`，验证 `CI_CANDIDATE` 的 executionRoot 和 sinkPath 均不位于仓库根内，并拒绝 symlink、相对路径逃逸、repo report cache 与 invocationMode 不匹配。除 freeze 中 `projectChecks[].allowedEphemeralWritePatterns` 精确允许的 ignored cache/build 输出外，CI checkout 不得产生任何写入。

`executionDomainSet` 只按以下规则产生，不得由 runner 手写：

```
TASK/CHANGE：route receipt 中 primaryDomains、observerDomains、依赖闭包和三个固定 meta domains 的去重集合；
TASK/RELEASE：law registry 中全部 active domains；
CI_CANDIDATE/CHANGE：law registry 中全部 active domains。
```

每个 executionDomainSet 内 domain 必须产生完整 Owner report；集合外 domain 在该 invocation 不产生 report。只有 TASK/RELEASE 才允许据全部 active domains 声明 `systemProven=true`。

56.2 EvidenceRequirementDecision

aggregator 在执行 component 前，为每个 atomic binding 产生非 verdict decision：

```
{
  "schemaVersion": 1,
  "candidateId": "...",
  "invocationMode": "TASK",
  "executionId": "...",
  "taskId": "...",
  "baseSha": null,
  "headSha": null,
  "verificationContext": "CHANGE",
  "lawId": "LAW-...",
  "bindingId": "domain.binding-id",
  "evidenceModes": [],
  "decision": "REQUIRED",
  "reasonCode": "DELTA_INTERSECTION",
  "triggerRefs": []
}
```

`decision` 只允许 REQUIRED、NOT_REQUIRED；`reasonCode` 只允许 STATIC_ALWAYS、DELTA_INTERSECTION、META_GOVERNANCE_CHANGE、FULL_RELEASE、NO_RELEVANT_DELTA、RELEASE_ONLY_BINDING。

固定算法：

```
Static binding 在 CHANGE/RELEASE 均 REQUIRED；
RELEASE context 下全部 Exam/Dynamic binding REQUIRED；
CHANGE context 下 EVERY_RELEVANT_CHANGE 仅在 actual path/surface/spec/contract delta 与 law appliesTo、Owner
 paths、binding refs 或依赖闭包相交时 REQUIRED；
CHANGE context 下 RELEASE_CANDIDATE 为 NOT_REQUIRED；
source-spec/law amendment 与 existing migration 对三个 meta domain 视为 META_GOVERNANCE_CHANGE；
NOT_REQUIRED 不是 PASS，不产生 component receipt，也不得用于 SYSTEM_PROVEN。
```

56.3 component receipt

每个 REQUIRED binding 精确一个 component receipt：

```
{
  "schemaVersion": 1,
  "candidateId": "...",
  "invocationMode": "TASK",
  "executionId": "...",
  "taskId": "...",
  "baseSha": null,
  "headSha": null,
  "verificationContext": "CHANGE",
  "lawId": "LAW-...",
  "capabilityDomain": "...",
  "component": "static",
  "bindingId": "domain.check-id",
  "evidenceModes": ["static.ast"],
  "outcome": "COMPONENT_PASS",
  "assertions": [],
  "findings": [],
  "targetSetHash": "...",
```

```
  "targetCount": 0,
  "cleanupOutcome": "NONE_REQUIRED"
}
```

component 只允许 static、exam、dynamic；outcome 只允许 COMPONENT_PASS、COMPONENT_FAIL；cleanupOutcome 只允许 NONE_REQUIRED、CLEANUP_PASS、CLEANUP_FAIL。Exam/Dynamic 追加运行参数 hash、真实 locator、case/attempt 与脱敏字段。不存在 SKIPPED_PASS、WARNING_PASS。

56.4 LawQualityReceipt

唯一 aggregator 为每个 active lawId 产生 context-bound receipt：

```
{
  "schemaVersion": 3,
  "candidateId": "...",
  "invocationMode": "TASK",
  "executionId": "...",
  "taskId": "...",
  "baseSha": null,
  "headSha": null,
  "verificationContext": "CHANGE",
  "lawId": "LAW-...",
  "capabilityDomain": "...",
  "declaredEvidenceModes": [],
  "requirementDecisionRefs": [],
  "requiredBindingIds": [],
  "components": [],
  "findingCount": 0,
  "failureCodes": [],
  "outcome": "PASS"
}
```

outcome 只允许 PASS、FAIL，且脱离 `verificationContext` 引用 PASS 属于无效陈述。

每个 REQUIRED binding 必须有且只有一个 candidate/execution/context 一致的 COMPONENT_PASS；每个 required evidenceMode 必须被精确覆盖一次。缺失、重复、COMPONENT_FAIL、finding、candidate mismatch、execution identity mismatch、receipt hash 不符、cleanup fail 均 FAIL，并使用：

```
REQUIRED_EVIDENCE_MISSING
DUPLICATE_EVIDENCE_COVERAGE
COMPONENT_FAILED
CANDIDATE_MISMATCH
RECEIPT_EXECUTION_IDENTITY_MISMATCH
RECEIPT_INTEGRITY_FAILED
PROOF_CLEANUP_FAILED
```

Static/Exam/Dynamic runner 永远不得写 LawQualityReceipt 或最终 PASS。

57. 唯一 context-bound Owner report

每个 `(invocationMode, executionId, candidateId, capabilityDomain)` 必须有且只有一个 Owner report；不得把 TASK current cache 与 CI ephemeral report 当成两个并存 verdict：

```
{
  "schemaVersion": 4,
  "candidateId": "...",
  "invocationMode": "TASK",
  "executionId": "...",
  "taskId": "...",
  "baseSha": null,
  "headSha": null,
  "verificationContext": "CHANGE",
  "capabilityDomain": "...",
  "lawQualityReceipts": [],
  "findingCount": 0,
  "verdict": "PASS",
  "systemProven": false
}
```

对 `executionDomainSet` 中的每个 domain，`lawQualityReceipts` 精确覆盖 manifest.allowedLawIds；任一 law FAIL、finding、缺 receipt 或 candidate/execution/context mismatch -> report FAIL。集合外 domain 不生成空 report 或伪 receipt。`systemProven=true` 只允许 `invocationMode=TASK`、RELEASE context 且全项目条件满足；CI_CANDIDATE 固定为 false。

`verdict` 只允许 `PASS`、`FAIL`；不得出现 `PARTIAL`、`WARNING`、`SKIPPED`、`READY`、`SYSTEM_PROVEN` 或任何自定义成功态。`systemProven` 是 RELEASE 全项目闭合属性，不是第三种 verdict。

只有 `aggregate-quality.ts` 可调用唯一 `report-writer.ts` 原子写 report；component runner 只写本次 execution evidence。`TASK` report 原子发布到 `scripts/audit/reports/<domain>.report.json` current cache；`CI_CANDIDATE` report 只发布到仓库外 `<executionRoot>/reports/<domain>.report.json`。两者 schema、identity 和 hash 算法完全相同，输出 sink 由 invocationMode 唯一决定。报告不作为下一次 evidence 输入。

58. Static 与 final quality 状态机

```
candidate capture
-> source/projection/ownership
-> one static index build/seal
-> all Static predicates
-> requirement decisions
-> required Exam/Dynamic components
-> candidate recheck
-> per-law context aggregate
-> per-owner temp reports
-> candidate recheck
-> invocation-bound atomic publish
-> TASK：更新 current report cache 后 task close 或 SYSTEM_PROVEN package
-> CI_CANDIDATE：仓库外写 invocation receipt 后结束。
```

任一漂移删除本次 execution root 的 receipts 和 temp reports；TASK 不更新 current report cache，CI_CANDIDATE 不产生 invocation receipt。

59. 三类证据不得互相冒充

```
Static Audit 证明静态结构；
Deterministic Exam 证明可重复确定性行为；
Dynamic Proof 证明真实外部运行。
```

不得用 AST 证明真实点击、fixture 证明 provider、截图证明全部算法、Dynamic 重定义 law、Exam 让 Static PASS，或建立第四证据 owner。

60. Exam registry、receipt 与 scope

存在 exam binding 时才建立 Exam 目录。literal declaration 精确声明 examId、lawId、evidenceModes、frequency、inputSetSha256、seed、timeoutMs；一个 examId 只绑定一个 lawId。

Exam component receipt 追加 inputSetSha256、seed、caseCount、failedCaseIds。失败非零退出，不写最终 report，不访问不需要的外部系统。

61. Dynamic Proof registry、凭证、安全与清理

proof declaration 精确声明 proofId、lawId、evidenceModes、frequency、credentialRefs、sideEffectClass、cleanupPolicy、maxAttempts、timeoutMs、retentionClass。

```
sideEffectClass：NONE | REVERSIBLE | IRREVERSIBLE_EXPLICIT_USER_ONLY；
cleanupPolicy：ALWAYS | NONE_REQUIRED；
retentionClass：REDACTED_LOCATORS_ONLY；
```

IRREVERSIBLE 必须有用户原文逐字授权。凭证只从 process environment、OS keychain 或受控 runtime secret provider 读取，不写 repo/receipt/截图/日志。动作使用 executionId/candidateId 派生 idempotency key；CI_CANDIDATE 不得执行 IRREVERSIBLE 动作。cleanup fail 即 component fail。禁止无限 retry/scroll/poll/search，禁止 fixture/mock/预制成功。

62. frequency 的唯一机器来源

frequency 只存在于 Manifest atomic exam/proof binding，并由 compiler 投影；只允许 EVERY_RELEVANT_CHANGE、RELEASE_CANDIDATE。blueprint 人读文本和 runner 不得建立第二频率。

63. 唯一质量总入口与两种证明边界

逻辑脚本名固定 `quality:check`。context 由第 56.1 节自动确定。顺序：

```
request/entry/task identity
-> scope check（写任务）
-> source integrity/history/ownership
-> projection check
-> freeze.projectChecks（按顺序、候选前后无非忽略写入）
-> Static components
-> requirement decisions
-> required governance mutants in isolated worktrees
-> required Exam components
-> required Dynamic components
-> candidate sentinel
-> LawQualityReceipts
-> Owner reports
-> TASK/CHANGE：task close
-> TASK/RELEASE：SYSTEM_PROVEN package
-> CI_CANDIDATE：只通过退出码与仓库外 ephemeral invocation receipt 结束，不写 candidate、source、projection、runtime 或 repo
 report cache，不关 task。
```

CHANGE context 全部 PASS 只允许关闭本次 task，表述为“本次变更已证明”；required mutant set 只覆盖本次改变的 law/inspector/governance closure。不得表述为整个系统发布就绪。RELEASE context 必须执行完整 mutant catalog。

`SYSTEM_PROVEN` 只在 RELEASE context 且以下全部成立时产生：

```
全部 active law 的全部 declared binding 都被 REQUIRED 且 PASS；
全部 Owner report PASS 且 systemProven=true；
全部 required mutants 以 expected failure code PASS；
旧治理/旧入口/兼容引用为零；
candidate 未漂移；
跨仓 change-set 全部 participant RELEASE receipt PASS。
```

quality 只通过一个 aggregator 和一个 report writer，不创建第二 verdict、READY、coverage verdict 或 release verdict。

64. CI 固定边界

CI 不是第二质量系统。仓库启用 CI 时只能有一个代码质量主工作流调用同一 `quality:check`，并满足：

```
普通 PR 校验使用只读 CI_CANDIDATE invocation；
CI_CANDIDATE 必须是 clean checkout，base/head commit 明确，actual delta 由 Git 计算；禁止任何
 candidate、source、projection、governance runtime、Owner report cache 写入、task close 和 SYSTEM_PROVEN；唯一例外是
 freeze.projectChecks 明确列出的 gitignored `allowedEphemeralWritePatterns`，且运行前后 candidateId 与 clean
 tracked/untracked state 必须一致；
CI_CANDIDATE 的 executionId = sha256("ci-candidate-v1\n" + repository + "\n" + baseSha + "\n" + headSha).
 slice(0,16)；
它运行 source integrity、projection、projectChecks、ownership、全量 Static、与 delta 相交的 EVERY_RELEVANT_CHANGE Exam/
 Proof 和 required mutants；
发布只能由人类授权的 release-proof task 或受保护的 manual release workflow 进入 RELEASE context；
任何自动 push/PR 事件不得自行提升为 RELEASE；
供应链 workflow 只可构建、发布或 attestation，不得写 LawQualityReceipt、Owner report 或第二 verdict；
CI 不得自动 audit:build 修复 stale projection。
```

同一 aggregator 必须在第 56.1 节仓库外 CI ephemeral execution root 产生一个非规范性 `ci-invocation-receipt.json`，精确结构为：

```
{
  "schemaVersion": 1,
  "invocationMode": "CI_CANDIDATE",
  "executionId": "<16 lowercase hex>",
  "repository": "owner/name",
  "baseSha": "<40 lowercase hex>",
  "headSha": "<40 lowercase hex>",
  "candidateId": "<64 lowercase hex>",
  "lawQualityReceiptHashes": [],
  "ownerReportHashes": [],
  "mutantReceiptSetHash": "<64 lowercase hex>"
}
```

该 receipt 只绑定本次 CI 执行刚刚在同一 execution root 产生的逐 law/Owner receipts，不得引用历史或另一 execution；不包含 PASS、READY、coverage、release 或 systemProven 字段，不提交仓库，不取得第二 verdict 主权。最终成功或失败只认同一 `quality:check` 进程退出码。它不是 Task Envelope，也不覆盖交互式 scope 证明。跨仓协调器只收集两仓独立 TASK/RELEASE receipts，不建立第二 quality runner。

第十二部分 | 空白仓库一次性建制

65. bootstrapMode=blank 的一次性完整建制

空白仓首次 bootstrap 是唯一入口例外，只允许建设治理七件套和本节固定 meta corpus，不允许业务功能。

固定动作：

精确 Node/package manager pin、strict tsconfig 与 blank typecheck；
本说明书、freeze（PDF disabled）、固定 AGENTS；
两份法案；
project-governance 十条 base laws；
law-system 八条 functional laws；
audit-system 十四条 functional laws；
SYSTEM-BLUEPRINT 与三个 meta blueprints；
附录 B 三个 exact manifests；
附录 C meta spec catalog；
parsers、shadow/
history、compiler、projection、task、route、scope、candidate、ownership、surface、amendment、aggregate、mutants；
五个 projection；
按需 meta Exams；
runtime ignore；
完整 tests/mutants；
canonical package scripts；
audit:build -> projection -> RELEASE context quality（内部执行完整 governance mutant catalog）；
全部通过后产生 candidate-bound SYSTEM_PROVEN。

当 entry-gate 可运行后，剩余 bootstrap 必须先通过它。不得留下 placeholder、TODO、示例 inspector、固定 PASS 或未绑定 law。

66. 基础法案固定十条根法条

以下十条 block 是空白 bootstrap 与 existing migration 的唯一 project-governance 基础 corpus。必须按此顺序和 canonical JSON 写入基础法案。

[LAW-PROJECT-GOVERNANCE-001-V1] 权威顺序唯一


```
{
  "appliesTo": [
    ".github/instructions/agent-rules/**",
    "AGENTS.md",
    "docs/governance/**"
  ],
  "capabilityDomain": "project-governance",
  "dependsOnLawIds": [],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.text",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得让任务提示词、代码、测试、generated artifact、历史报告或讨论记录覆盖上级规范主权。",
    "不得把人类未明确授权的法案或说明书修订解释为普通实施范围。"
  ],
  "lawId": "LAW-PROJECT-GOVERNANCE-001-V1",
  "routingTags": [
    "authority-order",
    "project-governance"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须把人类任务授权仅用于确定本次范围，并按照冻结说明书、基础法案、功能法案、施工图、规格、任务协议、实现和证据的顺序裁决规范冲突。",
  "supersedesLawId": null,
  "tier": "P0"
}
```

要

- 必须把人类任务授权仅用于确定本次范围，并按照冻结说明书、基础法案、功能法案、施工图、规格、任务协议、实现和证据的顺序裁决规范冲突。

不要

- 不得让任务提示词、代码、测试、generated artifact、历史报告或讨论记录覆盖上级规范主权。
- 不得把人类未明确授权的法案或说明书修订解释为普通实施范围。

证据

- static.text
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-PROJECT-GOVERNANCE-002-V1] 每项能力唯一真相


```
{
  "appliesTo": [
    "**/*"
  ],
  "capabilityDomain": "project-governance",
  "dependsOnLawIds": [
    "LAW-PROJECT-GOVERNANCE-001-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.dependency",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得建立 compatibility、fallback、shadow、bridge、dual-write 或平行主链。",
    "不得让 projection、缓存、历史文件或消费端副本取得源头主权。"
  ],
  "lawId": "LAW-PROJECT-GOVERNANCE-002-V1",
  "routingTags": [
    "project-governance",
    "single-source-of-truth"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使每项生产事实、能力、入口、合同、registry、compiler、runner 和 report writer 只有一个 active 真相源。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使每项生产事实、能力、入口、合同、registry、compiler、runner 和 report writer 只有一个 active 真相源。

不要

- 不得建立 compatibility、fallback、shadow、bridge、dual-write 或平行主链。
- 不得让 projection、缓存、历史文件或消费端副本取得源头主权。

证据

- static.dependency
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-PROJECT-GOVERNANCE-003-V1] 每项能力唯一 Owner


```
{
  "appliesTo": [
    "**/*"
  ],
  "capabilityDomain": "project-governance",
  "dependsOnLawIds": [
    "LAW-PROJECT-GOVERNANCE-001-V1",
    "LAW-PROJECT-GOVERNANCE-002-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.filesystem",
    "static.dependency",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得使用 shared、common、umbrella、prefix fallback 或多 Owner 共同主权。",
    "不得以 observedPaths、消费关系或文件容器所有权冒充逻辑主权。"
  ],
  "lawId": "LAW-PROJECT-GOVERNANCE-003-V1",
  "routingTags": [
    "owner",
    "project-governance"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使每个 capabilityDomain、lawId、生产文件、合同和外部 surface 具有精确一个 primary Owner。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使每个 capabilityDomain、lawId、生产文件、合同和外部 surface 具有精确一个 primary Owner。

不要

- 不得使用 shared、common、umbrella、prefix fallback 或多 Owner 共同主权。
- 不得以 observedPaths、消费关系或文件容器所有权冒充逻辑主权。

证据

- static.filesystem
- static.dependency
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-PROJECT-GOVERNANCE-004-V1] 终态无阶段


```
{
  "appliesTo": [
    ".github/instructions/agent-rules/**",
    "docs/blueprints/**",
    "docs/specs/**"
  ],
  "capabilityDomain": "project-governance",
  "dependsOnLawIds": [
    "LAW-PROJECT-GOVERNANCE-001-V1"
```

```
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
   "static.text",
   "exam.property"
  ],
  "excludes": [],
  "forbidden": [
   "不得把阶段、最小版本、未来接入、默认关闭、人工晋级、待完善或临时壳写成合法目标。",
   "不得用任务拆分改变已经授权能力的完整交付分母。"
  ],
  "lawId": "LAW-PROJECT-GOVERNANCE-004-V1",
  "routingTags": [
   "project-governance",
   "terminal-state"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 active 法案、施工图和规格直接描述完整生产终态。",
  "supersedesLawId": null,
  "tier": "P0"
 }
```


要

- 必须使 active 法案、施工图和规格直接描述完整生产终态。

不要

- 不得把阶段、最小版本、未来接入、默认关闭、人工晋级、待完善或临时壳写成合法目标。
- 不得用任务拆分改变已经授权能力的完整交付分母。

证据

- static.text
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-PROJECT-GOVERNANCE-005-V1] 证据失败关闭


```
 {
  "appliesTo": [
   "scripts/audit/**",
   "scripts/dynamic-proof/**",
   "scripts/exam/**"
  ],
  "capabilityDomain": "project-governance",
  "dependsOnLawIds": [
   "LAW-PROJECT-GOVERNANCE-001-V1",
   "LAW-PROJECT-GOVERNANCE-002-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
   "static.contract",
   "exam.property"
  ],
  "excludes": [],
  "forbidden": [
   "不得用 warning、partial、coverage、人工声明、错误证据类型或单一组件 PASS 放行整条法案。",
   "不得让 Static、Exam 或 Dynamic Proof 互相冒充。"
  ],
  "lawId": "LAW-PROJECT-GOVERNANCE-005-V1",
  "routingTags": [
   "evidence-closure",
   "project-governance"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使每个 active lawId 绑定全部必需证据组件，并在缺失、失败、过期、候选漂移或证据类型错误时整体失败。",
  "supersedesLawId": null,
```

"tier": "P0"
}

要

- 必须使每个 active lawId 绑定全部必需证据组件，并在缺失、失败、过期、候选漂移或证据类型错误时整体失败。

不要

- 不得用 warning、partial、coverage、人工声明、错误证据类型或单一组件 PASS 放行整条法案。
- 不得让 Static、Exam 或 Dynamic Proof 互相冒充。

证据

- static.contract
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-PROJECT-GOVERNANCE-006-V1] Codex 范围不可漂移

{
"appliesTo": [
".governance/runtime/**",
"scripts/audit/core/**"
],
"capabilityDomain": "project-governance",
"dependsOnLawIds": [
"LAW-PROJECT-GOVERNANCE-001-V1",
"LAW-PROJECT-GOVERNANCE-003-V1"
],
"enforcementMode": "composite",
"evidenceModes": [
"static.contract",
"exam.property"
],
"excludes": [],
"forbidden": [
"不得顺手深化、扩域、重构、预留未来能力、修改未授权路径或先写后补范围。",
"不得删除、改写或重建 task lock、baseline、route receipt 或 task context 以逃避检查。"
],
"lawId": "LAW-PROJECT-GOVERNANCE-006-V1",
"routingTags": [
"codex-scope",
"project-governance"
],
"schemaVersion": 2,
"status": "active",
"summary": "必须使任何写任务在首个仓库写入前完成请求捕获、baseline、Task Envelope、完整初始闭包和路由，并在结束时通过 scope check。",
"supersedesLawId": null,
"tier": "P0"
}

要

- 必须使任何写任务在首个仓库写入前完成请求捕获、baseline、Task Envelope、完整初始闭包和路由，并在结束时通过 scope check。

不要

- 不得顺手深化、扩域、重构、预留未来能力、修改未授权路径或先写后补范围。
- 不得删除、改写或重建 task lock、baseline、route receipt 或 task context 以逃避检查。

证据

- static.contract
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-PROJECT-GOVERNANCE-007-V1] 历史无当前主权


```
{
  "appliesTo": [
    "**/*"
  ],
  "capabilityDomain": "project-governance",
  "dependsOnLawIds": [
    "LAW-PROJECT-GOVERNANCE-001-V1",
    "LAW-PROJECT-GOVERNANCE-002-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.text",
    "static.dependency",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得保留 retired law、旧入口、旧 Owner mapping、旧 runner、旧 generated 或历史方案作为兼容依赖。",
    "不得复用已经退役的 lawKey 表示无关语义。"
  ],
  "lawId": "LAW-PROJECT-GOVERNANCE-007-V1",
  "routingTags": [
    "history-retirement",
    "project-governance"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使当前工作树只保留 active 权威、唯一现役实现和被标记为非权威的历史材料。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使当前工作树只保留 active 权威、唯一现役实现和被标记为非权威的历史材料。

不要

- 不得保留 retired law、旧入口、旧 Owner mapping、旧 runner、旧 generated 或历史方案作为兼容依赖。
- 不得复用已经退役的 lawKey 表示无关语义。

证据

- static.text
- static.dependency
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-PROJECT-GOVERNANCE-008-V1] 源头说明书冻结


```
{
  "appliesTo": [
    "AGENTS.md",
    "docs/governance/**"
```

```
  ],
  "capabilityDomain": "project-governance",
  "dependsOnLawIds": [
   "LAW-PROJECT-GOVERNANCE-001-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
   "static.config",
   "static.json",
   "exam.property"
  ],
  "excludes": [],
  "forbidden": [
   "不得由 Codex 自动修订说明书、入口文件、freeze hash 或工具链 pin。",
   "不得把可选 PDF 派生物视为权威源或普通任务必需交付。"
  ],
  "lawId": "LAW-PROJECT-GOVERNANCE-008-V1",
  "routingTags": [
   "freeze",
   "project-governance"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须以外部 freeze manifest 的 SHA-256 保护本说明书和固定 AGENTS 入口，并把文本冻结与候选系统实证分成两个独立状态。",
  "supersedesLawId": null,
  "tier": "P0"
 }
```


要

- 必须以外部 freeze manifest 的 SHA-256 保护本说明书和固定 AGENTS 入口，并把文本冻结与候选系统实证分成两个独立状态。

不要

- 不得由 Codex 自动修订说明书、入口文件、freeze hash 或工具链 pin。
- 不得把可选 PDF 派生物视为权威源或普通任务必需交付。

证据

- static.config
- static.json
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-PROJECT-GOVERNANCE-009-V1] 无主路径阻断


```
 {
  "appliesTo": [
   "**/*"
  ],
  "capabilityDomain": "project-governance",
  "dependsOnLawIds": [
   "LAW-PROJECT-GOVERNANCE-002-V1",
   "LAW-PROJECT-GOVERNANCE-003-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
   "static.filesystem",
   "static.dependency",
   "exam.property"
  ],
  "excludes": [],
  "forbidden": [
   "不得先创建无 Owner 文件、目录、入口、依赖、配置或 surface 再补治理。",
   "不得把 tracked deletion 的读取失败当作阻止合法物理删除的理由。"
  ],
  "lawId": "LAW-PROJECT-GOVERNANCE-009-V1",
  "routingTags": [
```

```
    "ownership-check",
    "project-governance"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使全部候选路径具有确定角色，并使全部生产文件、合同和 surface 被精确一个 primary Owner 命中。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使全部候选路径具有确定角色，并使全部生产文件、合同和 surface 被精确一个 primary Owner 命中。

不要

- 不得先创建无 Owner 文件、目录、入口、依赖、配置或 surface 再补治理。
- 不得把 tracked deletion 的读取失败当作阻止合法物理删除的理由。

证据

- static.filesystem
- static.dependency
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-PROJECT-GOVERNANCE-010-V1] 任务完整闭合


```
{
  "appliesTo": [
    "**/*"
  ],
  "capabilityDomain": "project-governance",
  "dependsOnLawIds": [
    "LAW-PROJECT-GOVERNANCE-002-V1",
    "LAW-PROJECT-GOVERNANCE-003-V1",
    "LAW-PROJECT-GOVERNANCE-005-V1",
    "LAW-PROJECT-GOVERNANCE-006-V1",
    "LAW-PROJECT-GOVERNANCE-007-V1",
    "LAW-PROJECT-GOVERNANCE-009-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.contract",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得用先搭壳、以后补、TODO、no-op、mock success、临时 bridge 或缺证据状态宣称完成。",
    "不得在跨仓变更任一参与仓未通过时宣布整体闭合。"
  ],
  "lawId": "LAW-PROJECT-GOVERNANCE-010-V1",
  "routingTags": [
    "project-governance",
    "task-completion"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使获准能力在同一任务完成唯一实现、合同、持久化、错误恢复、全部法定证据、generated projection 和旧路径退休。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使获准能力在同一任务完成唯一实现、合同、持久化、错误恢复、全部法定证据、generated projection 和旧路径退休。

不要

- 不得用先搭壳、以后补、TODO、no-op、mock success、临时 bridge 或缺证据状态宣称完成。
- 不得在跨仓变更任一参与仓未通过时宣布整体闭合。

证据

- static.contract
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

67. law-system 与 audit-system 固定完整法条

功能法案初始 meta corpus 精确为下列八条 law-system 与十四条 audit-system；不是“至少”，不得由 Codex 增删、合并、拆分或自行补元数据。

67.1 law-system

[LAW-LAW-SYSTEM-001-V1] 两份 active 法案与零 shadow law


```
{
  "appliesTo": [
    ".github/instructions/agent-rules/**",
    "AGENTS.md",
    "docs/**/*.md"
  ],
  "capabilityDomain": "law-system",
  "dependsOnLawIds": [
    "LAW-PROJECT-GOVERNANCE-001-V1",
    "LAW-PROJECT-GOVERNANCE-002-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.text",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得在 instructions、README、AGENTS、施工图、规格、注释或任务上下文中建立第三法案源。",
    "不得仅扫描 law-record fence 而放过普通规范性文字形成 shadow law。"
  ],
  "lawId": "LAW-LAW-SYSTEM-001-V1",
  "routingTags": [
    "law-source",
    "shadow-law"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 active 法案容器精确为基础法案和功能法案两份，并使其它文件中的未绑定规范性句子为零。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使 active 法案容器精确为基础法案和功能法案两份，并使其它文件中的未绑定规范性句子为零。

不要

- 不得在 instructions、README、AGENTS、施工图、规格、注释或任务上下文中建立第三法案源。
- 不得仅扫描 law-record fence 而放过普通规范性文字形成 shadow law。

证据

- static.text
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-LAW-SYSTEM-002-V1] 法条原子语法与镜像一致


```
{
  "appliesTo": [
    ".github/instructions/agent-rules/**",
    "scripts/audit/core/law-parser.ts"
  ],
  "capabilityDomain": "law-system",
  "dependsOnLawIds": [
    "LAW-LAW-SYSTEM-001-V1",
    "LAW-PROJECT-GOVERNANCE-005-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.text",
    "static.json",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得接受未知字段、默认补值、宽松 Markdown 修复、隐藏要求或镜像分叉。",
    "不得把一个不可独立裁决的复合口号作为单一 lawId。"
  ],
  "lawId": "LAW-LAW-SYSTEM-002-V1",
  "routingTags": [
    "law-parser",
    "law-schema"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使每个 law-record 采用唯一 schema、原子 requirement、非空 forbidden 和与机器字段逐字一致的人读镜像。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使每个 law-record 采用唯一 schema、原子 requirement、非空 forbidden 和与机器字段逐字一致的人读镜像。

不要

- 不得接受未知字段、默认补值、宽松 Markdown 修复、隐藏要求或镜像分叉。
- 不得把一个不可独立裁决的复合口号作为单一 lawId。

证据

- static.text
- static.json
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-LAW-SYSTEM-003-V1] lawId、domain 与历史谱系唯一


```
{
 "appliesTo": [
  ".github/instructions/agent-rules/**",
  "scripts/audit/core/law-history-check.ts"
 ],
 "capabilityDomain": "law-system",
 "dependsOnLawIds": [
  "LAW-LAW-SYSTEM-002-V1",
  "LAW-PROJECT-GOVERNANCE-007-V1"
 ],
 "enforcementMode": "composite",
 "evidenceModes": [
  "static.text",
  "static.json",
  "exam.property"
 ],
 "excludes": [],
 "forbidden": [
  "不得复用编号承载无关语义、跳过前一 major 版本或让同一完整 lawId 在历史中出现不同规范指纹。",
  "不得让 Codex 自行选择历史比较范围或忽略当前 HEAD 祖先中的冲突。"
 ],
 "lawId": "LAW-LAW-SYSTEM-003-V1",
 "routingTags": [
  "law-history",
  "law-id"
 ],
 "schemaVersion": 2,
 "status": "active",
 "summary": "必须使 lawId 前缀、capabilityDomain、编号、版本和 supersedesLawId 形成可由当前 Git 祖先历史复放的唯一谱系。",
 "supersedesLawId": null,
 "tier": "P0"
}
```


要

- 必须使 lawId 前缀、capabilityDomain、编号、版本和 supersedesLawId 形成可由当前 Git 祖先历史复放的唯一谱系。

不要

- 不得复用编号承载无关语义、跳过前一 major 版本或让同一完整 lawId 在历史中出现不同规范指纹。
- 不得让 Codex 自行选择历史比较范围或忽略当前 HEAD 祖先中的冲突。

证据

- static.text
- static.json
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-LAW-SYSTEM-004-V1] lawId 原子绑定完整


```
{
 "appliesTo": [
  "scripts/audit/core/manifest-parser.ts",
  "scripts/audit/domains/**/manifest.json"
 ],
 "capabilityDomain": "law-system",
 "dependsOnLawIds": [
  "LAW-LAW-SYSTEM-002-V1",
  "LAW-PROJECT-GOVERNANCE-003-V1",
  "LAW-PROJECT-GOVERNANCE-005-V1"
 ],
 "enforcementMode": "composite",
 "evidenceModes": [
  "static.contract",
  "static.json",
```

```
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得使用彼此分离且可能错位的 lawIds/checkIds 数组或多张人工映射表。",
    "不得让任一 evidenceMode 没有对应 binding，或让 binding 声明法条未要求的证据。"
  ],
  "lawId": "LAW-LAW-SYSTEM-004-V1",
  "routingTags": [
    "law-binding",
    "manifest"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使每个 active lawId 在唯一 Manifest 原子记录中绑定 blueprint anchor、全部 spec、Static check、Exam 和
Dynamic Proof。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使每个 active lawId 在唯一 Manifest 原子记录中绑定 blueprint anchor、全部 spec、Static check、Exam 和 Dynamic Proof。

不要

- 不得使用彼此分离且可能错位的 lawIds/checkIds 数组或多张人工映射表。
- 不得让任一 evidenceMode 没有对应 binding，或让 binding 声明法条未要求的证据。

证据

- static.contract
- static.json
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-LAW-SYSTEM-005-V1] 施工图与规格机器合同唯一


```
{
  "appliesTo": [
    "docs/blueprints/**",
    "docs/specs/**",
    "scripts/audit/core/blueprint-parser.ts",
    "scripts/audit/core/spec-parser.ts"
  ],
  "capabilityDomain": "law-system",
  "dependsOnLawIds": [
    "LAW-LAW-SYSTEM-004-V1",
    "LAW-PROJECT-GOVERNANCE-002-V1",
    "LAW-PROJECT-GOVERNANCE-003-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.contract",
    "static.json",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得依赖自由 Markdown 推断 contract、proof frequency、surface、调用方向或 spec closure。",
    "不得在消费端复制跨域合同或让规格改变法案 tier、Owner 与禁止面。"
  ],
  "lawId": "LAW-LAW-SYSTEM-005-V1",
  "routingTags": [
    "blueprint",
    "spec"
  ],
  "schemaVersion": 2,
```

```
  "status": "active",
  "summary": "必须使系统施工图、domain 施工图、稳定 anchor 和规格描述符具有可解析 schema、唯一 Owner、版本和 lawId 绑定。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使系统施工图、domain 施工图、稳定 anchor 和规格描述符具有可解析 schema、唯一 Owner、版本和 lawId 绑定。

不要

- 不得依赖自由 Markdown 推断 contract、proof frequency、surface、调用方向或 spec closure。
- 不得在消费端复制跨域合同或让规格改变法案 tier、Owner 与禁止面。

证据

- static.contract
- static.json
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-LAW-SYSTEM-006-V1] 唯一 compiler 与五个 projection


```
{
  "appliesTo": [
    "scripts/audit/core/artifact-compiler.ts",
    "scripts/audit/generated/**"
  ],
  "capabilityDomain": "law-system",
  "dependsOnLawIds": [
    "LAW-LAW-SYSTEM-004-V1",
    "LAW-LAW-SYSTEM-005-V1",
    "LAW-PROJECT-GOVERNANCE-002-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.ast",
    "static.json",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得建立第二 compiler、第六 generated artifact、人工 registry 或让 compiler 读取候选生产文件、历史报告和外部系统。",
    "不得让 generated artifact 输出 PASS、READY 或规范性新义务。"
  ],
  "lawId": "LAW-LAW-SYSTEM-006-V1",
  "routingTags": [
    "compiler",
    "generated-projection"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使唯一纯 compiler 从冻结说明书、两份法案、Manifests、施工图、规格和证据声明生成精确五个 deterministic projection。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使唯一纯 compiler 从冻结说明书、两份法案、Manifests、施工图、规格和证据声明生成精确五个 deterministic projection。

不要

- 不得建立第二 compiler、第六 generated artifact、人工 registry 或让 compiler 读取候选生产文件、历史报告和外部系统。
- 不得让 generated artifact 输出 PASS、READY 或规范性新义务。

证据

- static.ast
- static.json
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-LAW-SYSTEM-007-V1] projection exact-byte 与只读校验


```
{
  "appliesTo": [
    "scripts/audit/core/build-audit-artifacts.ts",
    "scripts/audit/core/projection-check.ts",
    "scripts/audit/generated/**"
  ],
  "capabilityDomain": "law-system",
  "dependsOnLawIds": [
    "LAW-LAW-SYSTEM-006-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.ast",
    "static.json",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得由检查入口自动修复 projection、留下部分新旧文件或接受 key 顺序和 hash 漂移。",
    "不得把 report、cache、summary 或时间字段伪装为 projection。"
  ],
  "lawId": "LAW-LAW-SYSTEM-007-V1",
  "routingTags": [
    "atomic-write",
    "projection-check"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 audit:build 原子写入五个 projection，并使 projection check 复用同一 compiler 进行 exact-byte 只读比较。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使 audit:build 原子写入五个 projection，并使 projection check 复用同一 compiler 进行 exact-byte 只读比较。

不要

- 不得由检查入口自动修复 projection、留下部分新旧文件或接受 key 顺序和 hash 漂移。
- 不得把 report、cache、summary 或时间字段伪装为 projection。

证据

- static.ast
- static.json
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-LAW-SYSTEM-008-V1] 法案与说明书修订原子事务


```
{
  "appliesTo": [
    ".github/instructions/agent-rules/**",
    "docs/governance/**",
    "scripts/audit/core/amendment-transaction.ts"
  ],
  "capabilityDomain": "law-system",
  "dependsOnLawIds": [
    "LAW-LAW-SYSTEM-006-V1",
    "LAW-LAW-SYSTEM-007-V1",
    "LAW-PROJECT-GOVERNANCE-006-V1",
    "LAW-PROJECT-GOVERNANCE-008-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.contract",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得在真实工作树暴露新 source 配旧 projection、旧 freeze 配新说明书或继续写入时使用失效 route。",
    "不得让 amendment 同时施工未授权业务功能或保留新旧治理兼容桥。"
  ],
  "lawId": "LAW-LAW-SYSTEM-008-V1",
  "routingTags": [
    "amendment-transaction",
    "law-amendment"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 law-amendment 和 source-spec-amendment 在隔离 staging 中完成新
source、binding、projection、freeze 和 post-route 验证后一次性应用。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使 law-amendment 和 source-spec-amendment 在隔离 staging 中完成新 source、binding、projection、freeze 和 post-route 验证后一次性应用。

不要

- 不得在真实工作树暴露新 source 配旧 projection、旧 freeze 配新说明书或继续写入时使用失效 route。
- 不得让 amendment 同时施工未授权业务功能或保留新旧治理兼容桥。

证据

- static.contract
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

67.2 audit-system

[LAW-AUDIT-SYSTEM-001-V1] Audit 只消费唯一执行投影


```
{
```

```
  "appliesTo": [
   "scripts/audit/core/run-generated-plan.ts",
   "scripts/audit/generated/**"
  ],
  "capabilityDomain": "audit-system",
  "dependsOnLawIds": [
   "LAW-LAW-SYSTEM-006-V1",
   "LAW-LAW-SYSTEM-007-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
   "static.ast",
   "exam.property"
  ],
  "excludes": [],
  "forbidden": [
   "不得在 Audit runner 内重建 law parser、Owner map、routing map 或第二执行计划。",
   "不得接受陈旧、手改或无法 exact-byte 重算的 projection。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-001-V1",
  "routingTags": [
   "audit-plan",
   "audit-system"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 Static Audit 只消费唯一 compiler 生成的 enforcement bindings、audit plan 和 inspector registry。",
  "supersedesLawId": null,
  "tier": "P0"
 }
```


要

- 必须使 Static Audit 只消费唯一 compiler 生成的 enforcement bindings、audit plan 和 inspector registry。

不要

- 不得在 Audit runner 内重建 law parser、Owner map、routing map 或第二执行计划。
- 不得接受陈旧、手改或无法 exact-byte 重算的 projection。

证据

- static.ast
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-002-V1] 每条法案证据 binding 完整


```
 {
  "appliesTo": [
   "scripts/audit/domains/**/manifest.json",
   "scripts/audit/generated/enforcement-bindings.generated.json"
  ],
  "capabilityDomain": "audit-system",
  "dependsOnLawIds": [
   "LAW-LAW-SYSTEM-004-V1",
   "LAW-PROJECT-GOVERNANCE-005-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
   "static.contract",
   "static.json",
   "exam.property"
  ],
  "excludes": [],
  "forbidden": [
   "不得遗漏必需组件、增加法条未要求组件或用一个模糊 checkId 覆盖多个 lawId。",
   "不得让相同 checkId、examId 或 proofId 绑定不兼容的证据语义。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-002-V1",
```

```
  "routingTags": [
    "audit-system",
    "evidence-binding"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使每个 active lawId 的 required evidence modes 与 Static、Exam、Dynamic bindings 精确相等。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使每个 active lawId 的 required evidence modes 与 Static、Exam、Dynamic bindings 精确相等。

不要

- 不得遗漏必需组件、增加法条未要求组件或用一个模糊 checkId 覆盖多个 lawId。
- 不得让相同 checkId、examId 或 proofId 绑定不兼容的证据语义。

证据

- static.contract
- static.json
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-003-V1] 一个候选一个 immutable evidence index


```
{
  "appliesTo": [
    "scripts/audit/core/run-generated-plan.ts",
    "scripts/audit/shared/evidence-index.ts"
  ],
  "capabilityDomain": "audit-system",
  "dependsOnLawIds": [
    "LAW-AUDIT-SYSTEM-001-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.ast",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得让 Owner 直接读取磁盘、增量补读、访问网络或构建第二 AST、JSON、YAML、SQL 和 dependency index。",
    "不得在 candidate 漂移后继续消费旧 index。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-003-V1",
  "routingTags": [
    "audit-system",
    "evidence-index"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使每次 Static Audit 对同一 candidateId 只建立一个 sealed、只读、不可跨 run 复用的 evidence index。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使每次 Static Audit 对同一 candidateId 只建立一个 sealed、只读、不可跨 run 复用的 evidence index。

不要

- 不得让 Owner 直接读取磁盘、增量补读、访问网络或构建第二 AST、JSON、YAML、SQL 和 dependency index。
- 不得在 candidate 漂移后继续消费旧 index。

证据

- static.ast
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-004-V1] 每个 lawId 独立 Static predicate 与 receipt


```
{
  "appliesTo": [
    "scripts/audit/domains/**/inspector.ts",
    "scripts/audit/domains/**/predicates.ts"
  ],
  "capabilityDomain": "audit-system",
  "dependsOnLawIds": [
    "LAW-AUDIT-SYSTEM-002-V1",
    "LAW-AUDIT-SYSTEM-003-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.ast",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得让一个 predicate 返回多个 lawId、返回空 assertions 或把 Owner 汇总冒充逐法条执行。",
    "不得用文件存在、token 命中、数量或固定值直接通过。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-004-V1",
  "routingTags": [
    "predicate",
    "static-receipt"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使每个含 static evidence 的 lawId 由唯一 checkId 执行独立 predicate，并产生绑定当前 candidateId 的 Static
component receipt。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使每个含 static evidence 的 lawId 由唯一 checkId 执行独立 predicate，并产生绑定当前 candidateId 的 Static component receipt。

不要

- 不得让一个 predicate 返回多个 lawId、返回空 assertions 或把 Owner 汇总冒充逐法条执行。
- 不得用文件存在、token 命中、数量或固定值直接通过。

证据

- static.ast
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-005-V1] 任一 finding 强制失败


```
{
 "appliesTo": [
  "scripts/audit/core/report-writer.ts",
  "scripts/audit/shared/**"
 ],
 "capabilityDomain": "audit-system",
 "dependsOnLawIds": [
  "LAW-AUDIT-SYSTEM-004-V1",
  "LAW-PROJECT-GOVERNANCE-005-V1"
 ],
 "enforcementMode": "composite",
 "evidenceModes": [
  "static.contract",
  "exam.property"
 ],
 "excludes": [],
 "forbidden": [
  "不得以 warning、partial、manual accepted、catch 后继续或 coverage 百分比放行。",
  "不得让 finding 与 PASS 同时出现在同一 receipt 或最终报告。"
 ],
 "lawId": "LAW-AUDIT-SYSTEM-005-V1",
 "routingTags": [
  "fail-closed",
  "finding"
 ],
 "schemaVersion": 2,
 "status": "active",
 "summary": "必须使任一 assertion 失败、finding、parse blocker、target 分母不完整或 predicate 异常产生组件失败。",
 "supersedesLawId": null,
 "tier": "P0"
}
```


要

- 必须使任一 assertion 失败、finding、parse blocker、target 分母不完整或 predicate 异常产生组件失败。

不要

- 不得以 warning、partial、manual accepted、catch 后继续或 coverage 百分比放行。
- 不得让 finding 与 PASS 同时出现在同一 receipt 或最终报告。

证据

- static.contract
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-006-V1] Static 不执行或消费 Exam 与 Dynamic


```
{
 "appliesTo": [
  "scripts/audit/**"
 ],
 "capabilityDomain": "audit-system",
 "dependsOnLawIds": [
  "LAW-AUDIT-SYSTEM-004-V1",
  "LAW-PROJECT-GOVERNANCE-005-V1"
 ],
 "enforcementMode": "composite",
 "evidenceModes": [
  "static.ast",
  "exam.property"
 ],
 "excludes": [],
 "forbidden": [
  "不得由 Static 启动业务服务、浏览器、数据库、存储、provider 或消费其历史 PASS。",
  "不得用 Static receipt 表示整条 composite law 已通过。"
```

```
  ],
  "lawId": "LAW-AUDIT-SYSTEM-006-V1",
  "routingTags": [
    "evidence-separation",
    "static-audit"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 Static Audit 仅证明静态结构，并把 Exam 和 Dynamic Proof 留给各自 runner。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使 Static Audit 仅证明静态结构，并把 Exam 和 Dynamic Proof 留给各自 runner。

不要

- 不得由 Static 启动业务服务、浏览器、数据库、存储、provider 或消费其历史 PASS。
- 不得用 Static receipt 表示整条 composite law 已通过。

证据

- static.ast
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-007-V1] 测试与 fixture 不得作为业务静态证据


```
{
  "appliesTo": [
    "scripts/audit/core/ownership-check.ts",
    "scripts/audit/shared/evidence-index.ts"
  ],
  "capabilityDomain": "audit-system",
  "dependsOnLawIds": [
    "LAW-AUDIT-SYSTEM-003-V1",
    "LAW-PROJECT-GOVERNANCE-009-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.dependency",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得用 tests、fixtures、examples、golden、mutation、artifacts、coverage 或历史报告证明业务实现成立。",
    "不得把被生产链消费的 fixture 继续标记为非生产。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-007-V1",
  "routingTags": [
    "evidence-denominator",
    "test-boundary"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使业务 law 的静态证据只来自生产分母、权威文件和绑定 contract。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使业务 law 的静态证据只来自生产分母、权威文件和绑定 contract。

不要

- 不得用 tests、fixtures、examples、golden、mutation、artifacts、coverage 或历史报告证明业务实现成立。
- 不得把被生产链消费的 fixture 继续标记为非生产。

证据

- static.dependency
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-008-V1] Audit 对陈旧 projection 失败关闭


```
{
  "appliesTo": [
    "scripts/audit/core/projection-check.ts",
    "scripts/audit/core/run-generated-plan.ts"
  ],
  "capabilityDomain": "audit-system",
  "dependsOnLawIds": [
    "LAW-AUDIT-SYSTEM-001-V1",
    "LAW-LAW-SYSTEM-007-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.ast",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得自动调用 audit:build、忽略多余 generated 文件或在 stale projection 上继续执行。",
    "不得把旧 report 的 candidateId 当作当前 projection 证明。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-008-V1",
  "routingTags": [
    "audit-system",
    "projection-freshness"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 audit:run 在执行任何 Owner 前通过只读 projection check 并验证全部 projection hash。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使 audit:run 在执行任何 Owner 前通过只读 projection check 并验证全部 projection hash。

不要

- 不得自动调用 audit:build、忽略多余 generated 文件或在 stale projection 上继续执行。
- 不得把旧 report 的 candidateId 当作当前 projection 证明。

证据

- static.ast
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-009-V1] candidate drift 使全部证据失效


```
{
 "appliesTo": [
  "scripts/audit/core/candidate-identity.ts",
  "scripts/audit/core/candidate-mutation-sentinel.ts"
 ],
 "capabilityDomain": "audit-system",
 "dependsOnLawIds": [
  "LAW-AUDIT-SYSTEM-003-V1",
  "LAW-PROJECT-GOVERNANCE-006-V1"
 ],
 "enforcementMode": "composite",
 "evidenceModes": [
  "static.ast",
  "exam.property"
 ],
 "excludes": [],
 "forbidden": [
  "不得发布部分旧候选、部分新候选的证据集合。",
  "不得通过排除未授权生产文件或忽略 tracked deletion 隐藏 candidate drift。"
 ],
 "lawId": "LAW-AUDIT-SYSTEM-009-V1",
 "routingTags": [
  "audit-system",
  "candidate-drift"
 ],
 "schemaVersion": 2,
 "status": "active",
 "summary": "必须在质量链关键节点重算 candidateId，并在任一漂移时丢弃本次组件 receipts 与最终 reports。",
 "supersedesLawId": null,
 "tier": "P0"
}
```


要

- 必须在质量链关键节点重算 candidateId，并在任一漂移时丢弃本次组件 receipts 与最终 reports。

不要

- 不得发布部分旧候选、部分新候选的证据集合。
- 不得通过排除未授权生产文件或忽略 tracked deletion 隐藏 candidate drift。

证据

- static.ast
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-010-V1] 禁止固定 PASS 与空证明


```
{
 "appliesTo": [
  "scripts/audit/domains/**",
  "scripts/audit/shared/**"
 ],
 "capabilityDomain": "audit-system",
 "dependsOnLawIds": [
  "LAW-AUDIT-SYSTEM-004-V1",
  "LAW-AUDIT-SYSTEM-005-V1"
 ],
 "enforcementMode": "composite",
 "evidenceModes": [
  "static.ast",
  "exam.property"
 ],
 "excludes": [],
 "forbidden": [
  "不得固定返回 PASS、吞异常、使用 generic token、文件存在、Owner 存在或 target 数量作为充分证明。",
```

```
    "不得让纯 absence 法条缺少 scope denominator ref。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-010-V1",
  "routingTags": [
    "anti-fake",
    "predicate"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使每个成功组件 receipt 至少包含一个可复放 assertion 和精确 evidenceRef。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使每个成功组件 receipt 至少包含一个可复放 assertion 和精确 evidenceRef。

不要

- 不得固定返回 PASS、吞异常、使用 generic token、文件存在、Owner 存在或 target 数量作为充分证明。
- 不得让纯 absence 法条缺少 scope denominator ref。

证据

- static.ast
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-011-V1] Owner 身份五方同名


```
{
  "appliesTo": [
    "scripts/audit/domains/**",
    "scripts/audit/reports/**"
  ],
  "capabilityDomain": "audit-system",
  "dependsOnLawIds": [
    "LAW-LAW-SYSTEM-004-V1",
    "LAW-PROJECT-GOVERNANCE-003-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.filesystem",
    "static.json",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得设置 alias、default owner、prefix fallback、共享 report 或运行时改名。",
    "不得让文件容器 Owner 接管其它 lawId 的逻辑 Owner。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-011-V1",
  "routingTags": [
    "identity",
    "owner-report"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 capabilityDomain、目录、Manifest、inspector literal identity 和最终 report identity 完全一致。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使 capabilityDomain、目录、Manifest、inspector literal identity 和最终 report identity 完全一致。

不要

- 不得设置 alias、default owner、prefix fallback、共享 report 或运行时改名。
- 不得让文件容器 Owner 接管其它 lawId 的逻辑 Owner。

证据

- static.filesystem
- static.json
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-012-V1] 历史方案和旧报告对 Audit 入边为零


```
{
  "appliesTo": [
    "scripts/audit/**",
    "scripts/dynamic-proof/**",
    "scripts/exam/**"
  ],
  "capabilityDomain": "audit-system",
  "dependsOnLawIds": [
    "LAW-PROJECT-GOVERNANCE-007-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.dependency",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得读取历史 PASS、旧 report 或迁移前 law map 作为当前候选证据。",
    "不得保留 compatibility reader 使旧质量链继续可达。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-012-V1",
  "routingTags": [
    "audit-system",
    "history-isolation"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 Static、Exam、Dynamic 和最终 aggregator 对历史报告、旧治理说明、旧 registry 与旧 runner 的执行入边为零。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使 Static、Exam、Dynamic 和最终 aggregator 对历史报告、旧治理说明、旧 registry 与旧 runner 的执行入边为零。

不要

- 不得读取历史 PASS、旧 report 或迁移前 law map 作为当前候选证据。
- 不得保留 compatibility reader 使旧质量链继续可达。

证据

- static.dependency
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-013-V1] 复合证据缺一即整条法案失败


```
{
  "appliesTo": [
    "scripts/audit/core/aggregate-quality.ts",
    "scripts/audit/core/run-quality-check.ts"
  ],
  "capabilityDomain": "audit-system",
  "dependsOnLawIds": [
    "LAW-AUDIT-SYSTEM-002-V1",
    "LAW-AUDIT-SYSTEM-005-V1",
    "LAW-AUDIT-SYSTEM-006-V1",
    "LAW-AUDIT-SYSTEM-009-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.contract",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得让 Static PASS、Exam PASS 或 Dynamic PASS 单独冒充 composite law PASS。",
    "不得生成 SKIPPED_AS_PASS、NOT_APPLICABLE_AS_PASS 或依赖人工补签的最终状态。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-013-V1",
  "routingTags": [
    "composite-evidence",
    "quality-aggregate"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使 final aggregator 按 required evidence modes 验证全部组件 receipt，并在任一缺失、失败、过期或候选不一致时把
LawQualityReceipt.outcome 固定为 FAIL。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使 final aggregator 按 required evidence modes 验证全部组件 receipt，并在任一缺失、失败、过期或候选不一致时把 LawQualityReceipt.outcome 固定为 FAIL。

不要

- 不得让 Static PASS、Exam PASS 或 Dynamic PASS 单独冒充 composite law PASS。
- 不得生成 SKIPPED_AS_PASS、NOT_APPLICABLE_AS_PASS 或依赖人工补签的最终状态。

证据

- static.contract
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

[LAW-AUDIT-SYSTEM-014-V1] 唯一最终汇总与报告写入


```
{
  "appliesTo": [
    "scripts/audit/core/aggregate-quality.ts",
    "scripts/audit/core/report-writer.ts",
    "scripts/audit/core/run-quality-check.ts",
    "scripts/audit/reports/**"
  ],
  "capabilityDomain": "audit-system",
```

```
  "dependsOnLawIds": [
    "LAW-AUDIT-SYSTEM-013-V1",
    "LAW-PROJECT-GOVERNANCE-010-V1"
  ],
  "enforcementMode": "composite",
  "evidenceModes": [
    "static.ast",
    "exam.property"
  ],
  "excludes": [],
  "forbidden": [
    "不得由 Static runner、Exam runner、Dynamic runner、其它 CI 步骤或第二 writer 写最终 Owner report 或第二 verdict
  artifact。",
    "不得让 CI_CANDIDATE 写仓库内 report cache，或让 TASK 与 CI 共用一个可覆盖输出路径。",
    "不得在任一参与仓、参与 domain 或跨仓合同未通过时关闭任务。"
  ],
  "lawId": "LAW-AUDIT-SYSTEM-014-V1",
  "routingTags": [
    "final-report",
    "quality-aggregate"
  ],
  "schemaVersion": 2,
  "status": "active",
  "summary": "必须使唯一 quality aggregator 和 report-writer 在 candidate 未漂移且全部 required components 完成后，按
  invocationMode 向唯一合法 sink 原子写入每个 domain 唯一 Owner report。",
  "supersedesLawId": null,
  "tier": "P0"
}
```


要

- 必须使唯一 quality aggregator 和 report-writer 在 candidate 未漂移且全部 required components 完成后，按 invocationMode 向唯一合法 sink 原子写入每个 domain 唯一 Owner report。

不要

- 不得由 Static runner、Exam runner、Dynamic runner、其它 CI 步骤或第二 writer 写最终 Owner report 或第二 verdict artifact。
- 不得让 CI_CANDIDATE 写仓库内 report cache，或让 TASK 与 CI 共用一个可覆盖输出路径。
- 不得在任一参与仓、参与 domain 或跨仓合同未通过时关闭任务。

证据

- static.ast
- exam.property

违法后果

- P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。

68. package scripts 精确逻辑名

`package.json.scripts` 必须存在：


```
{
  "typecheck": "tsc -p tsconfig.json --noEmit",
  "governance:request:capture": "tsx scripts/audit/core/request-capture.ts",
  "governance:entry": "tsx scripts/audit/core/entry-gate.ts",
  "governance:task:start": "tsx scripts/audit/core/task-start.ts",
  "governance:task:abandon": "tsx scripts/audit/core/abandon-task.ts",
  "governance:scope:propose": "tsx scripts/audit/core/propose-task-scope.ts",
  "governance:route": "tsx scripts/audit/core/route-task.ts",
  "governance:scope:check": "tsx scripts/audit/core/scope-check.ts",
  "governance:ownership:check": "tsx scripts/audit/core/ownership-check.ts",
  "governance:law-history:check": "tsx scripts/audit/core/law-history-check.ts",
  "governance:amendment": "tsx scripts/audit/core/amendment-transaction.ts",
  "governance:bootstrap:existing": "tsx scripts/audit/core/bootstrap-existing.ts",
  "governance:mutants:check": "tsx scripts/audit/core/run-governance-mutants.ts",
  "audit:build": "tsx scripts/audit/core/build-audit-artifacts.ts",
```

```
    "audit:projection:check": "tsx scripts/audit/core/projection-check.ts",
    "audit:run": "tsx scripts/audit/core/run-generated-plan.ts",
    "quality:check": "tsx scripts/audit/core/run-quality-check.ts"
  }
```

blank 必须使用上面的 `typecheck`。existing 可以保留 freeze.projectChecks 绑定的现役 script 名，但同一语义不得保留 alias；存在 exam/dynamic binding 时再增加 `exam:check`、`dynamic-proof:check`。

调用命令由 freeze 的 canonical package manager 决定，逻辑 script 名不变。

第十三部分 | 既有仓库迁移与新增能力

69. bootstrapMode=existing 的一次性迁移合同

69.1 migration-contract.json

runtime migration contract 顶层：

```
{
  "schemaVersion": 1,
  "taskId": "...",
  "initialCandidateId": "...",
  "repository": "owner/name",
  "canonicalPackageManager": "pnpm",
  "projectChecks": [],
  "legacyAuthorities": [],
  "legacyExecutors": [],
  "legacyGenerated": [],
  "productionRoots": [],
  "domainOwnershipPlan": [],
  "deletePaths": [],
  "keepNonNormativePaths": [],
  "oldReferenceQueries": [],
  "expectedZeroReferencePaths": []
}
```

每个旧 path 必须有唯一 disposition：MIGRATE、DELETE、HISTORY_ONLY、KEEP_NON_NORMATIVE；不得遗漏、不得 KEEP_NORMATIVE。

迁移必须在同一 staging transaction 完成：

```
迁入两份法案与 exact meta corpus；
把现有有效业务法条按 block 迁入功能法案；
建立 system/domain blueprints、specs、Manifests 和 atomic evidence bindings；
建立唯一 compiler/route/quality；
删除旧法案源、旧 instructions 主权、旧 index、Owner map、runner、generated、CI quality 入口和 compatibility reader；
保留文档只能降为无规范词的导航或 HISTORY_ONLY，生产入边为零；
保留当前 canonical Node/package manager，除非另有明确 toolchain amendment；
全量 mutants、旧引用零、quality PASS 后原子应用；
新旧治理不得并行任何一个 commit 或工作树终态。
```

69.2 两个仓库的固定主权边界

`xuanxue-web` 与 `ADVISION-X/AGENT` 均为独立既有仓：

```
每仓独立 request、task lock、baseline、candidateId、law registry、Manifests、quality gate 和 reports；
不得共享一个 runtime lock 或用一仓 PASS 覆盖另一仓；
每仓分别 bootstrapMode=existing；
治理迁移不得同时施工网站业务或 AGENT 新能力。
```

69.3 跨仓 contract

跨仓 contract 只有一个 provider repository/domain/spec。provider 和 consumer 都必须使用第 24 节同一个 discriminated union；消费者只保留完整 `contract-pin` spec 与可选 exact-byte cache projection。

provider `cross-repo-contract` spec 必须包含完整公共 identity、contractId、version、providerRepo、providerDomain、authorityArtifact 和 consumerRepos。provider spec 不保存自身 hash；其 canonical raw bytes hash 由 consumer pin 和跨仓证明包外部固定。

consumer `contract-pin` spec 必须包含完整公共 identity、contractId、version、providerRepo、providerDomain、providerSpecRef、providerCommitSha、contractSha256 和 cachePath/null；cache 如存在只读、exact-byte、不得编辑。

跨仓 change-set：

```
{
 "schemaVersion": 1,
 "changeSetId": "<64 hex>",
 "contractId": "...",
 "contractVersion": 1,
 "participants": [
  {
   "repository": "owner/name",
   "role": "PROVIDER",
   "initialCandidateId": "...",
   "finalCandidateId": "...",
   "qualityReceiptSha256": "...",
   "outcome": "PASS"
  }
 ]
}
```

changeSetId 由 contractId/version、参与仓名和各 initialCandidateId canonical hash 得出。任一 participant FAIL/MISSING、pin 不一致或 provider contract 未先成立，整体不得宣布闭合。外层 receipt 不取得任一仓规范主权。

70. 新增 capabilityDomain 的固定门槛

新增 domain 必须在同一次 law-amendment staging transaction 中形成完整终态闭包：

```
人类原文明确授权新增 capabilityDomain；
现有 domain 无法承接的边界由现有 law/blueprint/Owner 证据证明；
功能法案写入该 domain 完整 active lawId 集；
SYSTEM-BLUEPRINT.domainInventory 增加精确记录；
建立唯一 domain blueprint machine block、stable anchors 和全部 spec descriptors；
建立唯一 Manifest，并为每个 lawId 写一个 atomic lawBindings record；
建立唯一 inspector、Static predicates、法定 Exams 和 Dynamic Proofs；
ownedPaths、observedPaths、contracts、surfaces 与相邻 Owner 无重叠；
输入、输出、持久化、状态、错误、恢复、安全和清理合同完整；
旧实现、临时目录、兼容入口和第二链路为零；
重建五个 projection，执行 history、ownership、mutants 与 quality。
```

禁止为了整理代码、减少文件行数、建立通用层或预留未来能力新增 domain。缺一项即 `DOMAIN_CREATION_INCOMPLETE`。

71. 新增 lawId 的固定闭环

新增 lawId 只能通过 law-amendment staging transaction，在原子应用前同时完成：

```
写入唯一 law-record metadata 与逐字镜像；
确认 lawId/domain/编号/history lineage 合法；
更新 Manifest.allowedLawIds；
在 Manifest.lawBindings 中增加同一 lawId 的唯一原子 binding；
绑定 blueprintPath + anchorId、全部 specRefs、static checkId、required examIds 和 proofIds；
更新 domain blueprint 的 lawIds、anchor 和证据声明；
建立或更新 spec descriptors 与 typed contracts；
实现 Static predicate，以及 law 要求的 Exam/Dynamic Proof；
为法条和每个 evidence component 建立固定 mutants；
在 staging 中重建五个 projection并通过 projection/history/ownership/quality；
原子应用后旧路径、旧 binding、第二入口和 shadow rule 为零。
```

不得先写 law 再补 binding、实现或证据；不得使用分离的 `lawBlueprintBindings`、`lawCheckBindings` 或平行数组。

72. lawId 修订和退役

72.1 语义修订

以下任一规范维度变化均创建下一 major version，并把 `supersedesLawId` 精确指向同一 lawKey 的前一 active major：

summary；forbidden；appliesTo/excludes；capabilityDomain/Owner；evidenceModes；tier/失败条件；依赖义务；surface/contract 主权。

同一完整 lawId 的规范指纹不得在 Git 祖先历史中变化。仅标题或不取得规范主权的排版澄清可以保留 lawId，但 law-record JSON 和逐字镜像不得改变。

修订必须在同一 staging transaction 中删除旧 current block、更新全部正向/反向依赖、Manifest atomic binding、blueprint/spec、predicates/proofs、mutants 和五个 projection；不得保留 alias、兼容法条或双版本 active。

72.2 退役

退役必须同时：

确认全部反向依赖已删除或合法迁移；
删除 active block、Manifest binding、blueprint/spec 当前引用、predicate/exam/proof 和实现入口；
把由该 law 独占的生产路径和 surfaces 删除或完整迁入现有 active Owner；
执行旧引用零、history、mutants 和 quality；
只由 Git 保留历史，不建立 tombstone、deprecated block、redirect 或 compatibility route。

任何 current 引用残留即 `LAW_RETIREMENT_INCOMPLETE`。

第十四部分 | 必须阻断的 Codex 漂移模式

73. 架构漂移

必须阻断：

新增第二入口；
新增第二 server、gateway、dispatcher、bus、registry、queue、worker；
把明确流程抽象成万能 planner；
把业务 Owner 下沉到 shared/common/utils；
把 manifest 变成第二法案；
把 generated registry 变成人工编辑源；
把 runner 变成 owner mapper；
把 inspector 变成业务执行器。

74. 范围漂移

必须阻断：

未在 Task Envelope 的修改；
为了格式统一修改整份功能法案；
为了类型优雅重构相邻 domain；
为了未来支持增加接口；
为了少报错捕获并吞掉异常；
为了 CI 通过减少 target denominator；
发现旧代码后未经授权全面清理；
新增无关测试、文档或配置。

75. 证据漂移

必须阻断：

文件存在即 PASS；
token 命中即 PASS；
owner 存在即 PASS；
数量达到即 PASS；
fixture 成功即生产成功；
历史报告即当前证据；
静态 schema 即真实 provider 可用；
截图即完整动态证明；
测试自己断言自己正确。

76. 兼容漂移

必须阻断：

legacy reader；
old/new 双写；
fallback to old implementation；
shadow route；
compat alias；
temporary bridge；
feature flag 保留旧链；
“先不删除，以防回滚”。

需要回滚时使用 Git 回滚，不在 active 工作树保留两套真相。

77. 文档漂移

必须阻断：

Codex 把自己的理解写进基础法案；
在法条增加背景和方案建议；
在施工图写阶段计划；
在规格写原则性口号；
在 AGENTS 维护手工路由大表；
在 README 复制 lawId 清单；
在历史报告声明当前 READY。

第十五部分 | Mutation 与独立自检

78. 唯一 mutation runner 与固定 catalog

唯一命令逻辑名：

```
governance:mutants:check
```

唯一 runner `run-governance-mutants.ts` ：

```
读取 tests/governance/mutants/manifest.json ；
为每个 mutant 创建 OS 临时隔离 worktree/copy ；
验证起始 candidateId ；
只应用一个 mutant ；
运行 manifest 指定唯一 command ；
要求非零退出且 stderr 含 exact expectedFailureCode ；
记录 runtime mutant receipt ；
销毁隔离目录 ；
重算原工作树 candidateId 与关键 source hash ；
任何未阻断、错误 code、超时、恢复失败或原工作树变化均整体 FAIL。
```

mutant manifest 每项：mutantId、category、targetPath、mutationKind、command、expectedFailureCode、timeoutMs。不得用自然语言“应该失败” 。

固定 catalog 至少覆盖：

78.1 法案与 shadow law

删除/复制/错 domain law ；弱化 summary ；空 forbidden ；未知字段 ；镜像分叉 ；依赖缺失/成环 ；第三 active law-record ；source spec 少/多/改一个 law-template ；普通 instructions 写未绑定规范句 ；错误 supersedes ；复用历史 lawKey。

78.2 Manifest/blueprint/spec/binding

缺/双 manifest ；law 未绑定/双绑定 ；atomic binding 丢 spec/static/exam/proof ；check/exam/proof 重复 ；anchor 缺失 ；spec owner 冲突 ；跨域 contract 双 Owner ；ownedPaths overlap。

78.3 projection/compiler

手改/删除/新增第六 projection ；key order/hash/time字段漂移 ；projection checker 自动修复 ；第二 compiler ；audit-plan 平行 arrays 错配。

78.4 证据与 composite

固定 PASS ；空 assertions ；catch 后通过 ；fixture/business evidence ；一个 predicate 多 law ；Static component PASS 冒充整条 law ；REQUIRED Exam/Dynamic 缺失 ；CHANGE PASS 冒充 RELEASE/SYSTEM_PROVEN ；candidate/context mismatch ；cleanup fail ；第二 report writer。

78.5 Candidate/scope/surface

运行中改源码 ；遗漏 untracked ；tracked delete 读取失败 ；rename 未拆 DELETE+ADD ；覆盖 pre-existing dirty ；先写后扩 scope ；initial closure 伪装 expansion ；新增 dependency/listener/outbound host/browser permission/native host 未声明 ；surface action 缺失 ；extractor 未绑定。

78.6 amendment/existing/cross-repo

新 source 配旧 projection 强行 route ；真实工作树中间态 ；source-spec 默认强制 PDF ；未授权 existing bootstrap ；migration contract 漏旧 normative file ；保留旧 runner bridge ；消费仓复制 contract ；pin hash 错 ；一仓 PASS 冒充跨仓 PASS。

78.7 工具链与任务锁

request 未 capture 先 entry；INITIALIZING 崩溃伪恢复；删除 OPEN lock；Codex 自行 abandon；latest/^ 治理依赖；同仓双 package manager 治理入口；篡改 task-context/envelope/baseline。

任何 mutant 未按 exact code 被阻断，规范只能保持 `SPEC_FROZEN`，系统不得宣称 `SYSTEM_PROVEN`。

79. 独立自检维度

定稿或大规模迁移完成后，必须由与施工过程隔离的审查执行以下五类检查：

语义完整性：每条规则是否同时有要、不要、Owner、证据、后果；
结构唯一性：是否存在第二法案、第二索引、第二 Owner map、第二 compiler、第二 runner；
路由完整性：任务是否得到完整 domain law 闭包；
证据真实性：是否用正确证据证明正确对象；
反作弊能力：mutants 是否全部失败关闭。

审查不得以“总体不错”“覆盖率很高”代替逐项 finding。

附录 A | AGENTS.md 固定正文

bootstrap 生成的 AGENTS 完整规范性正文精确为以下内容；package manager 命令由 freeze 生成时替换成当前仓唯一调用形式，替换后整文件 hash 写入 freeze。项目描述只能追加不含规范词的纯身份说明。

```
# Codex Governance Entry

本仓库使用冻结的法案系统。Codex 不是治理规则的共同设计者。

任务动作顺序：

1. 把外层编排器提供的完整人类授权原字节送入 `governance:request:capture`。
2. 运行 `governance:entry`。
3. 建立或恢复同一 task 的 lock、baseline 和 Task Envelope。
4. 全文读取基础法案与 law registry。
5. 只读建立 task-proposal，声明精确 paths、initial closures、pathClaims 和 surfaces。
6. 运行 scope propose 与 route。
7. 验证并全文读取 task-context；route 不是 ROUTED 或 hash 不匹配时停止。
8. 写入只限 envelope；新增范围只用五个 reasonCode，并在首次触碰前重新 route。
9. 禁止第二法案、索引、Owner、compiler、runner、入口、fallback、compat、shadow、bridge 和 dual-write。
10. 说明书/入口/freeze 只在明确 source-spec-amendment staging 中修改；法案只在明确 law-amendment staging 中修改。
11. 完成后执行 scope、ownership、history、projection 与 quality；只有全部 required evidence 和 final aggregate 成功才关闭 task。
12. 任一证据缺失、跨仓 participant 未通过或 candidate 漂移时，不得声称完成或生产级。
```

`AGENTS.md` 中上述规范性内容由本说明书直接授权并受 freeze hash 保护，因此不是 shadow law。

第十六部分 | 最终验收与冻结完成定义

80. 法案正文验收

active 容器精确两份；
基础法案精确一个 project-governance 区块和十条固定 law；
功能法案精确包含固定 law-system 八条与 audit-system 十四条，再加经授权业务 laws；
全部 lawId 使用 LAW- 前缀且 domain/heading 一致；
全部 supersedes/history lineage 成立；
镜像与 metadata exact-byte；
forbidden 非空；
阶段语义为零；
两份 active law 之外的实际 law-record 与未绑定规范文字为零；source spec law-template 精确 32 个；
当前 retired block 为零。

81. blueprint/spec/index/compiler 验收

SYSTEM-BLUEPRINT machine block 完整；
每个 domain blueprint、stable anchor、spec descriptor 可解析；
第 24 节普通、cross-repo-contract、contract-pin 三个 spec 分支互斥且各自字段完整；
law->blueprint anchor->spec->static/exam/proof binding 完整；
跨域 contract 单 Owner；
跨仓 provider spec 与 consumer pin 使用同一 discriminated union，provider spec 不自写自身 hash；
唯一 compiler；
五个 projection exact-byte；
所有嵌套 schema 完整；
audit-plan 使用 atomic lawExecutions；
compiler 不承担 candidate ownership；
projection checker 只读；
GENERATED_FILESET_INVALID 只由 build/projection wrapper 负责，pure compiler 不返回该 code；
手改任一 projection 必败。

82. Owner、candidate 与 surface 验收

每个 domain 一个 Manifest/inspector；
每个 law、生产 PRESENT 文件、contract、surface 一个 primary Owner；
tracked deletion、rename、mode、symlink、staged/unstaged/untracked 有确定 candidate 表示；
无主/重叠为零；
pre-existing dirty 未变化；
initial closure 与 expansion phase 可证明；
dependency/listener/outbound host/browser permission/native host 均可提取；
未绑定 extractor 必败。

83. Codex、amendment 与 existing migration 验收

真实执行：单 domain、跨 domain、新增、删除、rename、法案修订、说明书修订、未知路径、dirty tree、task-start 崩溃、existing migration。

必须证明：

request capture 先于 entry；
未 ROUTED 不可写；
INITIALIZING 可安全恢复或 blocked；
完整 initial closures 合法；
非法 expansion blocked；
law/source-spec amendment 在 staging 无死锁；
真实工作树无 source/projection 混合中间态；
existing bootstrap 只迁移治理且旧引用为零；
普通任务不能改法案/说明书；
PDF disabled 时不构成交付依赖。

84. 质量主链与复合证据验收

一个 candidate 一个 static index；
每个 required binding 一个 component receipt；
每个 law 一个带 verificationContext 的 LawQualityReceipt；
每个 invocation/execution/candidate/domain 一个 Owner report；
audit:run 只写 Static component receipts，永远不写 Owner report；
Static/Exam/Dynamic runner 不写 LawQualityReceipt 或最终 PASS；
缺任一 REQUIRED Exam/Dynamic 必使对应 context 的 LawQualityReceipt FAIL；
任一 finding/parse blocker/cleanup fail/candidate drift 必败；
只有 final aggregator 通过唯一 report-writer 写 reports；TASK 只写 runtime evidence 与 repo current-cache，CI_CANDIDATE 只写仓库外 ephemeral execution root；
CI_CANDIDATE 除 freeze.projectChecks 精确列出的 ignored ephemeral cache 外不产生 checkout 写入；
CHANGE PASS 只关闭任务，RELEASE PASS 才能 SYSTEM_PROVEN；
quality:check 唯一；
mutant runner exact failure code 全通过；
不存在第二 verdict artifact。

85. 双仓、冻结状态与最终完成定义

两仓必须分别完成 existing migration 和独立 quality；跨仓 contract 只有一个 provider Owner，consumer pin exact hash；任一 participant 未通过不得闭合。

状态定义：

FINAL_SOURCE_VALIDATED：说明书源文本已经通过本文件定义的结构、镜像、schema、交叉引用、枚举和冲突自检，但外部 freeze 与治理实现尚未因此自动成立。
SPEC_FROZEN：本 Markdown、固定 AGENTS 和工具链由 freeze hash 保护。
SYSTEM_PROVEN：当前 candidate 在 RELEASE context 下，全部 active laws 的全部 declared evidence、Owners、mutants、旧路径零和跨仓 RELEASE 参与者均通过。

只有以下全部成立，才允许宣布系统终态完成：

正文少而唯一；
法条细而独立；
要与不要完整；
Owner 精确唯一；
blueprint/spec/binding 全机器可读；
索引全而无主权；
Codex 有 request、baseline、初始闭包、route、scope；
修订与既有迁移无事务死锁；
Static/Exam/Dynamic 各守边界；
复合证据缺一必败；
所有 mutants 失败关闭；
旧法案、旧索引、旧 runner、旧 generated、兼容桥与第二链路为零；
两仓独立通过且跨仓合同 pin 一致；
freeze 生效；
普通 Codex 无法修改源头说明书。

最终运行公式：

人类请求原文
-> capture
-> entry + task transaction
-> baseline + complete initial closure
-> route + task context
-> 唯一实现施工
-> path/surface/ownership scope
-> projection
-> Static components
-> EvidenceRequirementDecisions
-> required governance mutants
-> required Exam components
-> required Dynamic components
-> final per-law aggregate
-> per-owner reports
-> participant receipts（跨仓时）
-> PASS | FAIL

本说明书的最终原则：

> **Codex 不是治理规则的共同设计者。Codex 只能读取已经冻结且机器闭合的边界，在被授权的精确范围内完成唯一实现，并用当前候选的全部真实证据证明。凡未被明示的扩张、深化、兼容、预留、影子规范和第二路径，一律禁止。**

附录 B | 空白 bootstrap 的三个 meta Manifest

以下 JSON 是 blank bootstrap 的 exact canonical source。existing migration 只能按当前仓 canonical package manager/lockfile 调整 project-governance 的工具链 path claim；lawBindings、lawIds、check/exam IDs 不得自行改变。

B.1 project-governance


```
{
 "allowedLawIds": [
  "LAW-PROJECT-GOVERNANCE-001-V1",
  "LAW-PROJECT-GOVERNANCE-002-V1",
  "LAW-PROJECT-GOVERNANCE-003-V1",
  "LAW-PROJECT-GOVERNANCE-004-V1",
  "LAW-PROJECT-GOVERNANCE-005-V1",
  "LAW-PROJECT-GOVERNANCE-006-V1",
  "LAW-PROJECT-GOVERNANCE-007-V1",
  "LAW-PROJECT-GOVERNANCE-008-V1",
  "LAW-PROJECT-GOVERNANCE-009-V1",
  "LAW-PROJECT-GOVERNANCE-010-V1"
 ],
 "blueprintRefs": [
  "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md"
 ],
 "capabilityDomain": "project-governance",
 "evidenceModes": [
  "static.filesystem",
  "static.text",
  "static.contract",
  "static.dependency",
  "static.config",
  "static.json",
  "exam.property"
 ],
 "lawBindings": {
  "LAW-PROJECT-GOVERNANCE-001-V1": {
   "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-001-V1",
   "examBindings": [
    {
     "evidenceModes": [
      "exam.property"
     ],
     "examId": "project-governance.authority-order-mutant",
     "frequency": "EVERY_RELEVANT_CHANGE"
    }
   ],
   "proofBindings": [],
   "specRefs": [
    "docs/specs/project-governance/authority-order.spec.json"
   ],
   "staticCheckId": "project-governance.authority-order"
  },
  "LAW-PROJECT-GOVERNANCE-002-V1": {
   "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-002-V1",
   "examBindings": [
    {
     "evidenceModes": [
      "exam.property"
     ],
     "examId": "project-governance.single-truth-mutant",
     "frequency": "EVERY_RELEVANT_CHANGE"
    }
   ],
   "proofBindings": [],
   "specRefs": [
    "docs/specs/project-governance/single-truth.spec.json"
   ],
   "staticCheckId": "project-governance.single-truth"
  },
  "LAW-PROJECT-GOVERNANCE-003-V1": {
   "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-003-V1",
   "examBindings": [
    {
```

```
        "evidenceModes": [
          "exam.property"
        ],
        "examId": "project-governance.single-owner-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/project-governance/single-owner.spec.json"
    ],
    "staticCheckId": "project-governance.single-owner"
  },
  "LAW-PROJECT-GOVERNANCE-004-V1": {
    "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-004-V1",
    "examBindings": [
      {
        "evidenceModes": [
          "exam.property"
        ],
        "examId": "project-governance.terminal-state-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/project-governance/terminal-state.spec.json"
    ],
    "staticCheckId": "project-governance.terminal-state"
  },
  "LAW-PROJECT-GOVERNANCE-005-V1": {
    "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-005-V1",
    "examBindings": [
      {
        "evidenceModes": [
          "exam.property"
        ],
        "examId": "project-governance.evidence-fail-closed-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/project-governance/evidence-fail-closed.spec.json"
    ],
    "staticCheckId": "project-governance.evidence-fail-closed"
  },
  "LAW-PROJECT-GOVERNANCE-006-V1": {
    "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-006-V1",
    "examBindings": [
      {
        "evidenceModes": [
          "exam.property"
        ],
        "examId": "project-governance.codex-scope-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/project-governance/codex-scope.spec.json"
    ],
    "staticCheckId": "project-governance.codex-scope"
  },
  "LAW-PROJECT-GOVERNANCE-007-V1": {
    "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-007-V1",
    "examBindings": [
      {
        "evidenceModes": [
          "exam.property"
        ],
        "examId": "project-governance.history-retirement-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/project-governance/history-retirement.spec.json"
    ],
    "staticCheckId": "project-governance.history-retirement"
  },
```

```
    "LAW-PROJECT-GOVERNANCE-008-V1": {
      "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-008-V1",
      "examBindings": [
        {
          "evidenceModes": [
            "exam.property"
          ],
          "examId": "project-governance.source-freeze-mutant",
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": [],
      "specRefs": [
        "docs/specs/project-governance/source-freeze.spec.json"
      ],
      "staticCheckId": "project-governance.source-freeze"
    },
    "LAW-PROJECT-GOVERNANCE-009-V1": {
      "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-009-V1",
      "examBindings": [
        {
          "evidenceModes": [
            "exam.property"
          ],
          "examId": "project-governance.ownership-block-mutant",
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": [],
      "specRefs": [
        "docs/specs/project-governance/ownership-block.spec.json"
      ],
      "staticCheckId": "project-governance.ownership-block"
    },
    "LAW-PROJECT-GOVERNANCE-010-V1": {
      "blueprintRef": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md#anchor:
law.LAW-PROJECT-GOVERNANCE-010-V1",
      "examBindings": [
        {
          "evidenceModes": [
            "exam.property"
          ],
          "examId": "project-governance.task-completion-mutant",
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": [],
      "specRefs": [
        "docs/specs/project-governance/task-completion.spec.json"
      ],
      "staticCheckId": "project-governance.task-completion"
    }
  },
  "observedPaths": [
    ".github/instructions/agent-rules/**",
    "docs/blueprints/**",
    "docs/specs/**",
    "scripts/audit/generated/**",
    "tests/governance/**"
  ],
  "ownedPaths": [
    ".gitignore",
    ".nvmrc",
    "AGENTS.md",
    "docs/governance/**",
    "package.json",
    "pnpm-lock.yaml",
    "scripts/audit/core/abandon-task.ts",
    "scripts/audit/core/bootstrap-existing.ts",
    "scripts/audit/core/entry-gate.ts",
    "scripts/audit/core/ownership-check.ts",
    "scripts/audit/core/propose-task-scope.ts",
    "scripts/audit/core/request-capture.ts",
    "scripts/audit/core/route-task.ts",
    "scripts/audit/core/scope-check.ts",
    "scripts/audit/core/surface-delta.ts",
    "scripts/audit/core/task-start.ts",
    "scripts/audit/domains/project-governance/**",
    "scripts/audit/surface-extractors/**",
    "scripts/exam/exams/project-governance/**",
    "tsconfig.json"
  ],
  "schemaVersion": 2,
```

```
  "specRefs": [
   "docs/specs/project-governance/authority-order.spec.json",
   "docs/specs/project-governance/codex-scope.spec.json",
   "docs/specs/project-governance/evidence-fail-closed.spec.json",
   "docs/specs/project-governance/history-retirement.spec.json",
   "docs/specs/project-governance/ownership-block.spec.json",
   "docs/specs/project-governance/single-owner.spec.json",
   "docs/specs/project-governance/single-truth.spec.json",
   "docs/specs/project-governance/source-freeze.spec.json",
   "docs/specs/project-governance/task-completion.spec.json",
   "docs/specs/project-governance/terminal-state.spec.json"
  ],
  "status": "active"
}
```


B.2 law-system


```
{
  "allowedLawIds": [
    "LAW-LAW-SYSTEM-001-V1",
    "LAW-LAW-SYSTEM-002-V1",
    "LAW-LAW-SYSTEM-003-V1",
    "LAW-LAW-SYSTEM-004-V1",
    "LAW-LAW-SYSTEM-005-V1",
    "LAW-LAW-SYSTEM-006-V1",
    "LAW-LAW-SYSTEM-007-V1",
    "LAW-LAW-SYSTEM-008-V1"
  ],
  "blueprintRefs": [
    "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md"
  ],
  "capabilityDomain": "law-system",
  "evidenceModes": [
    "static.text",
    "static.ast",
    "static.contract",
    "static.json",
    "exam.property"
  ],
  "lawBindings": {
    "LAW-LAW-SYSTEM-001-V1": {
      "blueprintRef": "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md#anchor:law.LAW-LAW-
SYSTEM-001-V1",
      "examBindings": [
        {
          "evidenceModes": [
            "exam.property"
          ],
          "examId": "law-system.two-law-sources-mutant",
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": [],
      "specRefs": [
        "docs/specs/law-system/two-law-sources.spec.json"
      ],
      "staticCheckId": "law-system.two-law-sources"
    },
    "LAW-LAW-SYSTEM-002-V1": {
      "blueprintRef": "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md#anchor:law.LAW-LAW-
SYSTEM-002-V1",
      "examBindings": [
        {
          "evidenceModes": [
            "exam.property"
          ],
          "examId": "law-system.law-record-schema-mutant",
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": [],
      "specRefs": [
        "docs/specs/law-system/law-record-schema.spec.json"
      ],
      "staticCheckId": "law-system.law-record-schema"
    },
    "LAW-LAW-SYSTEM-003-V1": {
      "blueprintRef": "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md#anchor:law.LAW-LAW-
SYSTEM-003-V1",
      "examBindings": [
        {
          "evidenceModes": [
```

```
          "exam.property"
        ],
        "examId": "law-system.law-history-lineage-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/law-system/law-history-lineage.spec.json"
    ],
    "staticCheckId": "law-system.law-history-lineage"
  },
  "LAW-LAW-SYSTEM-004-V1": {
    "blueprintRef": "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md#anchor:law.LAW-LAW-
SYSTEM-004-V1",
    "examBindings": [
      {
        "evidenceModes": [
          "exam.property"
        ],
        "examId": "law-system.atomic-law-binding-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/law-system/atomic-law-binding.spec.json"
    ],
    "staticCheckId": "law-system.atomic-law-binding"
  },
  "LAW-LAW-SYSTEM-005-V1": {
    "blueprintRef": "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md#anchor:law.LAW-LAW-
SYSTEM-005-V1",
    "examBindings": [
      {
        "evidenceModes": [
          "exam.property"
        ],
        "examId": "law-system.blueprint-spec-contract-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/law-system/blueprint-spec-contract.spec.json"
    ],
    "staticCheckId": "law-system.blueprint-spec-contract"
  },
  "LAW-LAW-SYSTEM-006-V1": {
    "blueprintRef": "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md#anchor:law.LAW-LAW-
SYSTEM-006-V1",
    "examBindings": [
      {
        "evidenceModes": [
          "exam.property"
        ],
        "examId": "law-system.single-compiler-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/law-system/single-compiler.spec.json"
    ],
    "staticCheckId": "law-system.single-compiler"
  },
  "LAW-LAW-SYSTEM-007-V1": {
    "blueprintRef": "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md#anchor:law.LAW-LAW-
SYSTEM-007-V1",
    "examBindings": [
      {
        "evidenceModes": [
          "exam.property"
        ],
        "examId": "law-system.projection-exact-byte-mutant",
        "frequency": "EVERY_RELEVANT_CHANGE"
      }
    ],
    "proofBindings": [],
    "specRefs": [
      "docs/specs/law-system/projection-exact-byte.spec.json"
    ],
    "staticCheckId": "law-system.projection-exact-byte"
  },
  "LAW-LAW-SYSTEM-008-V1": {
```

```
    "blueprintRef": "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md#anchor:law.LAW-LAW-
SYSTEM-008-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "law-system.amendment-transaction-mutant",
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/law-system/amendment-transaction.spec.json"
    ],
    "staticCheckId": "law-system.amendment-transaction"
   }
  },
  "observedPaths": [
   "AGENTS.md",
   "scripts/audit/domains/**/manifest.json",
   "scripts/audit/reports/**",
   "tests/governance/**"
  ],
  "ownedPaths": [
   ".github/instructions/agent-rules/**",
   "docs/blueprints/**",
   "docs/specs/**",
   "scripts/audit/README.md",
   "scripts/audit/core/amendment-transaction.ts",
   "scripts/audit/core/artifact-compiler.ts",
   "scripts/audit/core/blueprint-parser.ts",
   "scripts/audit/core/build-audit-artifacts.ts",
   "scripts/audit/core/canonical-json.ts",
   "scripts/audit/core/hash.ts",
   "scripts/audit/core/law-history-check.ts",
   "scripts/audit/core/law-parser.ts",
   "scripts/audit/core/manifest-parser.ts",
   "scripts/audit/core/projection-check.ts",
   "scripts/audit/core/shadow-law-detector.ts",
   "scripts/audit/core/spec-parser.ts",
   "scripts/audit/domains/law-system/**",
   "scripts/audit/generated/**",
   "scripts/exam/exams/law-system/**"
  ],
  "schemaVersion": 2,
  "specRefs": [
   "docs/specs/law-system/amendment-transaction.spec.json",
   "docs/specs/law-system/atomic-law-binding.spec.json",
   "docs/specs/law-system/blueprint-spec-contract.spec.json",
   "docs/specs/law-system/law-history-lineage.spec.json",
   "docs/specs/law-system/law-record-schema.spec.json",
   "docs/specs/law-system/projection-exact-byte.spec.json",
   "docs/specs/law-system/single-compiler.spec.json",
   "docs/specs/law-system/two-law-sources.spec.json"
  ],
  "status": "active"
 }
```


B.3 audit-system

初始 32 条 meta law 没有任何 `dynamic.*` evidenceMode，因此本 blank Manifest 不预占 `scripts/dynamic-proof/**`。只有后续经授权的 law-amendment 首次建立真实 dynamic binding 时，才在同一原子闭包中创建共享 registry/runner、proof path、Owner claim、package script 和 mutants；路径主权声明不得先于首个 binding 充当未来预留。


```
 {
  "allowedLawIds": [
   "LAW-AUDIT-SYSTEM-001-V1",
   "LAW-AUDIT-SYSTEM-002-V1",
   "LAW-AUDIT-SYSTEM-003-V1",
   "LAW-AUDIT-SYSTEM-004-V1",
   "LAW-AUDIT-SYSTEM-005-V1",
   "LAW-AUDIT-SYSTEM-006-V1",
   "LAW-AUDIT-SYSTEM-007-V1",
   "LAW-AUDIT-SYSTEM-008-V1",
   "LAW-AUDIT-SYSTEM-009-V1",
   "LAW-AUDIT-SYSTEM-010-V1",
   "LAW-AUDIT-SYSTEM-011-V1",
```

```
    "LAW-AUDIT-SYSTEM-012-V1",
    "LAW-AUDIT-SYSTEM-013-V1",
    "LAW-AUDIT-SYSTEM-014-V1"
  ],
  "blueprintRefs": [
    "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md"
  ],
  "capabilityDomain": "audit-system",
  "evidenceModes": [
    "static.filesystem",
    "static.ast",
    "static.contract",
    "static.dependency",
    "static.json",
    "exam.property"
  ],
  "lawBindings": {
    "LAW-AUDIT-SYSTEM-001-V1": {
      "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-001-V1",
      "examBindings": [
        {
          "evidenceModes": [
            "exam.property"
          ],
          "examId": "audit-system.consume-single-plan-mutant",
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": [],
      "specRefs": [
        "docs/specs/audit-system/consume-single-plan.spec.json"
      ],
      "staticCheckId": "audit-system.consume-single-plan"
    },
    "LAW-AUDIT-SYSTEM-002-V1": {
      "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-002-V1",
      "examBindings": [
        {
          "evidenceModes": [
            "exam.property"
          ],
          "examId": "audit-system.evidence-binding-complete-mutant",
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": [],
      "specRefs": [
        "docs/specs/audit-system/evidence-binding-complete.spec.json"
      ],
      "staticCheckId": "audit-system.evidence-binding-complete"
    },
    "LAW-AUDIT-SYSTEM-003-V1": {
      "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-003-V1",
      "examBindings": [
        {
          "evidenceModes": [
            "exam.property"
          ],
          "examId": "audit-system.single-evidence-index-mutant",
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": [],
      "specRefs": [
        "docs/specs/audit-system/single-evidence-index.spec.json"
      ],
      "staticCheckId": "audit-system.single-evidence-index"
    },
    "LAW-AUDIT-SYSTEM-004-V1": {
      "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-004-V1",
      "examBindings": [
        {
          "evidenceModes": [
            "exam.property"
          ],
          "examId": "audit-system.one-law-one-predicate-mutant",
          "frequency": "EVERY_RELEVANT_CHANGE"
        }
      ],
      "proofBindings": [],
      "specRefs": [
```

```
      "docs/specs/audit-system/one-law-one-predicate.spec.json"
    ],
    "staticCheckId": "audit-system.one-law-one-predicate"
   },
   "LAW-AUDIT-SYSTEM-005-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-005-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "audit-system.finding-fail-mutant",
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/audit-system/finding-fail.spec.json"
    ],
    "staticCheckId": "audit-system.finding-fail"
   },
   "LAW-AUDIT-SYSTEM-006-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-006-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "audit-system.evidence-separation-mutant",
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/audit-system/evidence-separation.spec.json"
    ],
    "staticCheckId": "audit-system.evidence-separation"
   },
   "LAW-AUDIT-SYSTEM-007-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-007-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "audit-system.production-evidence-only-mutant",
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/audit-system/production-evidence-only.spec.json"
    ],
    "staticCheckId": "audit-system.production-evidence-only"
   },
   "LAW-AUDIT-SYSTEM-008-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-008-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "audit-system.projection-freshness-mutant",
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/audit-system/projection-freshness.spec.json"
    ],
    "staticCheckId": "audit-system.projection-freshness"
   },
   "LAW-AUDIT-SYSTEM-009-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-009-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "audit-system.candidate-drift-mutant",
```

```
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/audit-system/candidate-drift.spec.json"
    ],
    "staticCheckId": "audit-system.candidate-drift"
   },
   "LAW-AUDIT-SYSTEM-010-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-010-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "audit-system.anti-fake-pass-mutant",
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/audit-system/anti-fake-pass.spec.json"
    ],
    "staticCheckId": "audit-system.anti-fake-pass"
   },
   "LAW-AUDIT-SYSTEM-011-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-011-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "audit-system.identity-five-way-mutant",
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/audit-system/identity-five-way.spec.json"
    ],
    "staticCheckId": "audit-system.identity-five-way"
   },
   "LAW-AUDIT-SYSTEM-012-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-012-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "audit-system.history-zero-input-mutant",
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/audit-system/history-zero-input.spec.json"
    ],
    "staticCheckId": "audit-system.history-zero-input"
   },
   "LAW-AUDIT-SYSTEM-013-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-013-V1",
    "examBindings": [
     {
      "evidenceModes": [
       "exam.property"
      ],
      "examId": "audit-system.composite-evidence-mutant",
      "frequency": "EVERY_RELEVANT_CHANGE"
     }
    ],
    "proofBindings": [],
    "specRefs": [
     "docs/specs/audit-system/composite-evidence.spec.json"
    ],
    "staticCheckId": "audit-system.composite-evidence"
   },
   "LAW-AUDIT-SYSTEM-014-V1": {
    "blueprintRef": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md#anchor:law.LAW-
AUDIT-SYSTEM-014-V1",
    "examBindings": [
```

```
      {
       "evidenceModes": [
        "exam.property"
       ],
       "examId": "audit-system.single-final-report-mutant",
       "frequency": "EVERY_RELEVANT_CHANGE"
      }
     ],
     "proofBindings": [],
     "specRefs": [
      "docs/specs/audit-system/single-final-report.spec.json"
     ],
     "staticCheckId": "audit-system.single-final-report"
    }
   },
   "observedPaths": [
    "scripts/audit/domains/**",
    "scripts/audit/generated/**",
    "tests/governance/**"
   ],
   "ownedPaths": [
    "scripts/audit/core/aggregate-quality.ts",
    "scripts/audit/core/candidate-identity.ts",
    "scripts/audit/core/candidate-mutation-sentinel.ts",
    "scripts/audit/core/report-writer.ts",
    "scripts/audit/core/run-generated-plan.ts",
    "scripts/audit/core/run-governance-mutants.ts",
    "scripts/audit/core/run-quality-check.ts",
    "scripts/audit/domains/audit-system/**",
    "scripts/audit/reports/**",
    "scripts/audit/shared/**",
    "scripts/exam/exam-registry.ts",
    "scripts/exam/exams/audit-system/**",
    "scripts/exam/run-exam.ts"
   ],
   "schemaVersion": 2,
   "specRefs": [
    "docs/specs/audit-system/anti-fake-pass.spec.json",
    "docs/specs/audit-system/candidate-drift.spec.json",
    "docs/specs/audit-system/composite-evidence.spec.json",
    "docs/specs/audit-system/consume-single-plan.spec.json",
    "docs/specs/audit-system/evidence-binding-complete.spec.json",
    "docs/specs/audit-system/evidence-separation.spec.json",
    "docs/specs/audit-system/finding-fail.spec.json",
    "docs/specs/audit-system/history-zero-input.spec.json",
    "docs/specs/audit-system/identity-five-way.spec.json",
    "docs/specs/audit-system/one-law-one-predicate.spec.json",
    "docs/specs/audit-system/production-evidence-only.spec.json",
    "docs/specs/audit-system/projection-freshness.spec.json",
    "docs/specs/audit-system/single-evidence-index.spec.json",
    "docs/specs/audit-system/single-final-report.spec.json"
   ],
   "status": "active"
  }
```

附录 C｜机器施工图、规格与 surface extractor 固定合同

C.1 blank SYSTEM-BLUEPRINT record


```
{
  "blueprintId": "system",
  "crossDomainContracts": [
    {
      "consumerDomains": [
        "audit-system"
      ],
      "contractId": "law-system.enforcement-projection",
      "providerDomain": "law-system",
      "providerSpecRef": "docs/specs/law-system/single-compiler.spec.json"
    },
    {
      "consumerDomains": [
        "law-system"
      ],
      "contractId": "project-governance.authority-order",
      "providerDomain": "project-governance",
      "providerSpecRef": "docs/specs/project-governance/authority-order.spec.json"
    },
    {
      "consumerDomains": [
        "audit-system"
      ],
      "contractId": "project-governance.candidate-task",
      "providerDomain": "project-governance",
      "providerSpecRef": "docs/specs/project-governance/codex-scope.spec.json"
    }
  ],
  "crossRepoContracts": [],
  "dependencyEdges": [
    {
      "contractId": "law-system.enforcement-projection",
      "fromDomain": "audit-system",
      "toDomain": "law-system"
    },
    {
      "contractId": "project-governance.candidate-task",
      "fromDomain": "audit-system",
      "toDomain": "project-governance"
    },
    {
      "contractId": "project-governance.authority-order",
      "fromDomain": "law-system",
      "toDomain": "project-governance"
    }
  ],
  "domainInventory": [
    {
      "blueprintPath": "docs/blueprints/domains/audit-system/AUDIT-SYSTEM-BLUEPRINT.md",
      "capabilityDomain": "audit-system",
      "manifestPath": "scripts/audit/domains/audit-system/manifest.json"
    },
    {
      "blueprintPath": "docs/blueprints/domains/law-system/LAW-SYSTEM-BLUEPRINT.md",
      "capabilityDomain": "law-system",
      "manifestPath": "scripts/audit/domains/law-system/manifest.json"
    },
    {
      "blueprintPath": "docs/blueprints/domains/project-governance/PROJECT-GOVERNANCE-BLUEPRINT.md",
      "capabilityDomain": "project-governance",
      "manifestPath": "scripts/audit/domains/project-governance/manifest.json"
    }
  ],
  "owner": "project-governance",
  "schemaVersion": 2,
  "status": "active",
  "surfaceExtractorBindings": [
    {
      "extractorId": "governance.package-json",
      "pathPattern": "package.json",
      "surfaceKinds": [
```

```
      "package-dependency",
      "package-script"
     ]
    }
   ]
}
```

C.2 system record 元素 schema

```
{
 "domainInventory": {"capabilityDomain":"...","manifestPath":"...","blueprintPath":"..."},
 "dependencyEdges": {"fromDomain":"consumer","toDomain":"provider","contractId":"provider.contract"},
 "crossDomainContracts": {"contractId":"...","providerDomain":"...","providerSpecRef":"...",
"consumerDomains":[]},
 "crossRepoContracts": {"contractId":"...","contractVersion":1,"providerRepo":"owner/name",
"providerSpecRef":"...","consumerRepos":[]},
 "surfaceExtractorBindings": {"pathPattern":"...","extractorId":"...","surfaceKinds":[]}
}
```

`crossRepoContracts.contractVersion` 必须等于对应 `providerSpecRef` 的 `cross-repo-contract.version`；`providerRepo + providerSpecRef + contractId + contractVersion` 是 topology identity。consumer 的 `contract-pin` 继续负责 commit 与 exact hash，SYSTEM-BLUEPRINT 不复制 pin 或 provider schema。

C.3 三个 meta domain blueprint 的确定性 renderer

blank bootstrap 不允许 Codex 手写三份 machine record。唯一 renderer 按附录 B Manifest 与 C.1 system record 生成：

```
schemaVersion=2 ; status=active ; blueprintId=`domain:<domain>` ; capabilityDomain=owner=<domain> ;
lawIds=Manifest.allowedLawIds ; specRefs=Manifest.specRefs ;
anchors=每个 lawId 一个 `{anchorId:"law.<lawId>",kind:"law",lawIds:[lawId]}` ;
dependencyEdges=C.1 中 fromDomain 等于本 domain 的边 ;
providedContracts/consumedContracts=本节合同目录 ;
prohibitedDependencyEdges=本节反向边目录 ;
surfaceExtractorBindings=本节 extractor 目录 ;
所有数组按 canonical key 排序。
```

合同目录精确为：

contractId	provider domain	provider spec	consumer domain/spec
`project-governance.authority-order`	`project-governance`	`project-governance.authority-order`	`law-system/law-system.single-compiler`
`project-governance.candidate-task`	`project-governance`	`project-governance.codex-scope`	`audit-system/audit-system.candidate-drift`
`law-system.enforcement-projection`	`law-system`	`law-system.single-compiler`	`audit-system/audit-system.consume-single-plan`

prohibited dependency edge 精确为：

```
project-governance -> law-system
project-governance -> audit-system
law-system -> audit-system
```

这些边表示 lower-authority/provider 反向依赖 consumer，必须为零；audit-system 可以按 C.1 消费 law-system/project-governance，law-system 可以消费 project-governance。

surface extractor 目录精确为：

```
[
 {
  "extractorId": "governance.package-json",
  "pathPattern": "package.json",
  "surfaceKinds": ["package-dependency", "package-script"]
 }
]
```

C.1 `surfaceExtractorBindings` 与 project-governance domain record 必须使用该精确记录；其它两个 meta domain 为空。

C.4 meta spec descriptor renderer

每个附录 B law binding 精确生成一份第 24 节 descriptor：

specId=<domain>.<slug>；status=active；owner=capabilityDomain=<domain>；version=1；
lawIds=[lawId]；blueprintRefs=[binding.blueprintRef]；specKind 取 C.5 表；
authorityArtifacts 按 path 排序包含：active law section、Manifest JSON pointer、domain blueprint anchor、inspector check declaration、存在时 exam/proof declaration；
providedContracts/consumedContracts 取 C.3 合同目录；
surfaces 取 C.4.1 package-script surface 目录；
states/errorCodes 取本说明书中由该 spec authority selector 指向的固定枚举；无枚举时必须为空数组，不得自行发明。

`authorityArtifacts` selector 格式精确为：

law:<lawId>
json-pointer:/lawBindings/<RFC6901-escaped-lawId>
anchor:<anchorId>
check:<staticCheckId>
exam:<examId>
proof:<proofId>

C.4.1 package-script surface 目录

scriptName	ownerDomain	lawId	specId
`typecheck`	`project-governance`	`LAW-PROJECT-GOVERNANCE-010-V1`	`project-governance.task-completion`
`governance:request:capture`	`project-governance`	`LAW-PROJECT-GOVERNANCE-006-V1`	`project-governance.codex-scope`
`governance:entry`	`project-governance`	`LAW-PROJECT-GOVERNANCE-006-V1`	`project-governance.codex-scope`
`governance:task:start`	`project-governance`	`LAW-PROJECT-GOVERNANCE-006-V1`	`project-governance.codex-scope`
`governance:task:abandon`	`project-governance`	`LAW-PROJECT-GOVERNANCE-006-V1`	`project-governance.codex-scope`
`governance:scope:propose`	`project-governance`	`LAW-PROJECT-GOVERNANCE-006-V1`	`project-governance.codex-scope`
`governance:route`	`project-governance`	`LAW-PROJECT-GOVERNANCE-006-V1`	`project-governance.codex-scope`
`governance:scope:check`	`project-governance`	`LAW-PROJECT-GOVERNANCE-006-V1`	`project-governance.codex-scope`
`governance:ownership:check`	`project-governance`	`LAW-PROJECT-GOVERNANCE-009-V1`	`project-governance.ownership-block`
`governance:amendment`	`law-system`	`LAW-LAW-SYSTEM-008-V1`	`law-system.amendment-transaction`
`governance:bootstrap:existing`	`project-governance`	`LAW-PROJECT-GOVERNANCE-007-V1`	`project-governance.history-retirement`
`governance:mutants:check`	`audit-system`	`LAW-AUDIT-SYSTEM-010-V1`	`audit-system.anti-fake-pass`
`governance:law-history:check`	`law-system`	`LAW-LAW-SYSTEM-003-V1`	`law-system.law-history-lineage`
`audit:build`	`law-system`	`LAW-LAW-SYSTEM-006-V1`	`law-system.single-compiler`
`audit:projection:check`	`law-system`	`LAW-LAW-SYSTEM-007-V1`	`law-system.projection-exact-byte`
`audit:run`	`audit-system`	`LAW-AUDIT-SYSTEM-001-V1`	`audit-system.consume-single-plan`
`quality:check`	`audit-system`	`LAW-AUDIT-SYSTEM-014-V1`	`audit-system.single-final-report`

存在 exam/proof binding 时，`exam:check` 与 `dynamic-proof:check` 分别归 `audit-system.evidence-separation`；其它脚本不得出现。

C.5 meta spec catalog

每个表项按第 24 节 exact descriptor schema 建立；version=1，owner/capabilityDomain 取 specId 前缀，lawIds 只含表中 lawId，blueprintRef 取 Manifest binding，authorityArtifacts 指向实际 parser/runner/contract 源文件，不得留空。

spec path	specId	specKind	lawId
`docs/specs/audit-system/anti-fake-pass.spec.json`	`audit-system.anti-fake-pass`	`proof`	`LAW-AUDIT-SYSTEM-010-V1`
`docs/specs/audit-system/candidate-drift.spec.json`	`audit-system.candidate-drift`	`proof`	`LAW-AUDIT-SYSTEM-009-V1`
`docs/specs/audit-system/composite-evidence.spec.json`	`audit-system.composite-evidence`	`proof`	`LAW-AUDIT-SYSTEM-013-V1`
`docs/specs/audit-system/consume-single-plan.spec.json`	`audit-system.consume-single-plan`	`proof`	`LAW-AUDIT-SYSTEM-001-V1`
`docs/specs/audit-system/evidence-binding-complete.spec.json`	`audit-system.evidence-binding-complete`	`proof`	`LAW-AUDIT-SYSTEM-002-V1`
`docs/specs/audit-system/evidence-separation.spec.json`	`audit-system.evidence-separation`	`proof`	`LAW-AUDIT-SYSTEM-006-V1`
`docs/specs/audit-system/finding-fail.spec.json`	`audit-system.finding-fail`	`proof`	`LAW-AUDIT-SYSTEM-005-V1`
`docs/specs/audit-system/history-zero-input.spec.json`	`audit-system.history-zero-input`	`proof`	`LAW-AUDIT-SYSTEM-012-V1`
`docs/specs/audit-system/identity-five-way.spec.json`	`audit-system.identity-five-way`	`proof`	`LAW-AUDIT-SYSTEM-011-V1`
`docs/specs/audit-system/one-law-one-predicate.spec.json`	`audit-system.one-law-one-predicate`	`proof`	`LAW-AUDIT-SYSTEM-004-V1`
`docs/specs/audit-system/production-evidence-only.spec.json`	`audit-system.production-evidence-only`	`proof`	`LAW-AUDIT-SYSTEM-007-V1`
`docs/specs/audit-system/projection-freshness.spec.json`	`audit-system.projection-freshness`	`proof`	`LAW-AUDIT-SYSTEM-008-V1`
`docs/specs/audit-system/single-evidence-index.spec.json`	`audit-system.single-evidence-index`	`proof`	`LAW-AUDIT-SYSTEM-003-V1`
`docs/specs/audit-system/single-final-report.spec.json`	`audit-system.single-final-report`	`proof`	`LAW-AUDIT-SYSTEM-014-V1`
`docs/specs/law-system/amendment-transaction.spec.json`	`law-system.amendment-transaction`	`proof`	`LAW-LAW-SYSTEM-008-V1`
`docs/specs/law-system/atomic-law-binding.spec.json`	`law-system.atomic-law-binding`	`proof`	`LAW-LAW-SYSTEM-004-V1`
`docs/specs/law-system/blueprint-spec-contract.spec.json`	`law-system.blueprint-spec-contract`	`proof`	`LAW-LAW-SYSTEM-005-V1`
`docs/specs/law-system/law-history-lineage.spec.json`	`law-system.law-history-lineage`	`proof`	`LAW-LAW-SYSTEM-003-V1`
`docs/specs/law-system/law-record-schema.spec.json`	`law-system.law-record-schema`	`proof`	`LAW-LAW-SYSTEM-002-V1`
`docs/specs/law-system/projection-exact-byte.spec.json`	`law-system.projection-exact-byte`	`proof`	`LAW-LAW-SYSTEM-007-V1`
`docs/specs/law-system/single-compiler.spec.json`	`law-system.single-compiler`	`proof`	`LAW-LAW-SYSTEM-006-V1`
`docs/specs/law-system/two-law-sources.spec.json`	`law-system.two-law-sources`	`proof`	`LAW-LAW-SYSTEM-001-V1`

spec path	specId	specKind	lawId
`docs/specs/project-governance/authority-order.spec.json`	`project-governance.authority-order`	`proof`	`LAW-PROJECT-GOVERNANCE-001-V1`
`docs/specs/project-governance/codex-scope.spec.json`	`project-governance.codex-scope`	`proof`	`LAW-PROJECT-GOVERNANCE-006-V1`
`docs/specs/project-governance/evidence-fail-closed.spec.json`	`project-governance.evidence-fail-closed`	`proof`	`LAW-PROJECT-GOVERNANCE-005-V1`
`docs/specs/project-governance/history-retirement.spec.json`	`project-governance.history-retirement`	`proof`	`LAW-PROJECT-GOVERNANCE-007-V1`
`docs/specs/project-governance/ownership-block.spec.json`	`project-governance.ownership-block`	`proof`	`LAW-PROJECT-GOVERNANCE-009-V1`
`docs/specs/project-governance/single-owner.spec.json`	`project-governance.single-owner`	`proof`	`LAW-PROJECT-GOVERNANCE-003-V1`
`docs/specs/project-governance/single-truth.spec.json`	`project-governance.single-truth`	`proof`	`LAW-PROJECT-GOVERNANCE-002-V1`
`docs/specs/project-governance/source-freeze.spec.json`	`project-governance.source-freeze`	`proof`	`LAW-PROJECT-GOVERNANCE-008-V1`
`docs/specs/project-governance/task-completion.spec.json`	`project-governance.task-completion`	`proof`	`LAW-PROJECT-GOVERNANCE-010-V1`
`docs/specs/project-governance/terminal-state.spec.json`	`project-governance.terminal-state`	`proof`	`LAW-PROJECT-GOVERNANCE-004-V1`

C.6 extractor contract

每个 extractor literal 声明 extractorId、supportedExtensions、surfaceKinds、claimedPathPatterns，并返回 canonical surface records。一个 surface-bearing path 精确一个 extractor；两个 extractor claim 同 path 或零 extractor claim 均 FAIL。extractor 只解析，不决定 Owner、law 或 PASS。

附录 D | 跨仓 contract 与证明包

D.1 provider `cross-repo-contract` 完整 spec

```
{
  "schemaVersion": 1,
  "specId": "browser-authority.browser-control-contract",
  "status": "active",
  "capabilityDomain": "browser-authority",
  "owner": "browser-authority",
  "specKind": "cross-repo-contract",
  "version": 1,
  "lawIds": ["LAW-BROWSER-AUTHORITY-001-V1"],
  "blueprintRefs": ["docs/blueprints/domains/browser-authority/BROWSER-AUTHORITY-BLUEPRINT.md#anchor:law.
 LAW-BROWSER-AUTHORITY-001-V1"],
  "authorityArtifacts": [
    {
      "path": "src/contracts/browser-control.ts",
      "selector": "export:BrowserControlContract",
      "language": "typescript"
    }
  ],
  "contractId": "browser-authority.browser-control",
  "providerRepo": "owner/provider-repo",
  "providerDomain": "browser-authority",
  "authorityArtifact": {
    "path": "src/contracts/browser-control.ts",
    "selector": "export:BrowserControlContract",
    "language": "typescript",
    "sha256": "<64 lowercase hex>"
  },
  "consumerRepos": ["owner/consumer-repo"],
  "surfaces": [],
  "states": [],
  "errorCodes": []
}
```

D.2 consumer `contract-pin` 完整 spec

```
{
  "schemaVersion": 1,
  "specId": "ai-acceptance.browser-control-pin",
  "status": "active",
  "capabilityDomain": "ai-acceptance",
  "owner": "ai-acceptance",
  "specKind": "contract-pin",
  "version": 1,
  "lawIds": ["LAW-AI-ACCEPTANCE-001-V1"],
  "blueprintRefs": ["docs/blueprints/domains/ai-acceptance/AI-ACCEPTANCE-BLUEPRINT.md#anchor:law.LAW-AI-
 ACCEPTANCE-001-V1"],
  "authorityArtifacts": [
    {
      "path": "docs/specs/ai-acceptance/browser-control-pin.spec.json",
      "selector": "json-pointer:/contractId",
      "language": "json"
    }
  ],
  "contractId": "browser-authority.browser-control",
  "providerRepo": "owner/provider-repo",
  "providerDomain": "browser-authority",
  "providerSpecRef": "docs/specs/browser-authority/browser-control-contract.spec.json",
  "providerCommitSha": "<40 lowercase hex>",
  "contractSha256": "<64 lowercase hex>",
  "cachePath": null,
  "surfaces": [],
  "states": [],
  "errorCodes": []
}
```

以上两个对象分别完整命中第 24 节的一个且仅一个 schema 分支。consumer cache 只允许 exact-byte generated projection，并由 pin hash 验证；不得人工编辑、不得含 provider 法案副本。

D.3 candidate-bound proof package

需要留档 `SYSTEM_PROVEN` 时，proof package 只包含 candidateId、law receipt hashes、Owner report hashes、mutant receipt set hash、cross-repo participant hashes 和 compiler/freeze versions；禁止时间字段、自然语言 READY 和规范性新内容。任何 candidate 变化使旧包只具历史追溯意义。